



GAME 기초

Part 1. Tank Game



학습목표

1. 외부의 3D Model 을 Import 하여 활용할 수 있다.
2. 게임오브젝트를 이동하거나, 회전시켜 움직임을 반영할 수 있다.
3. 슈팅 객체를 다룰 수 있다.
4. 프리팹을 다룰 수 있다.
5. 사운드 리소스를 다룰 수 있다.
6. 기본적인 물리 처리를 할 수 있다.
7. 카메라 이동을 다룰 수 있다.
8. 파티클을 다룰 수 있다.
9. 오브젝트 간의 충돌을 판별하고 이벤트를 다룰 수 있다.
10. 게임 인터페이스를 다룰 수 있다.
11. 게임의 화면 전환을 다룰 수 있다.

학습내용

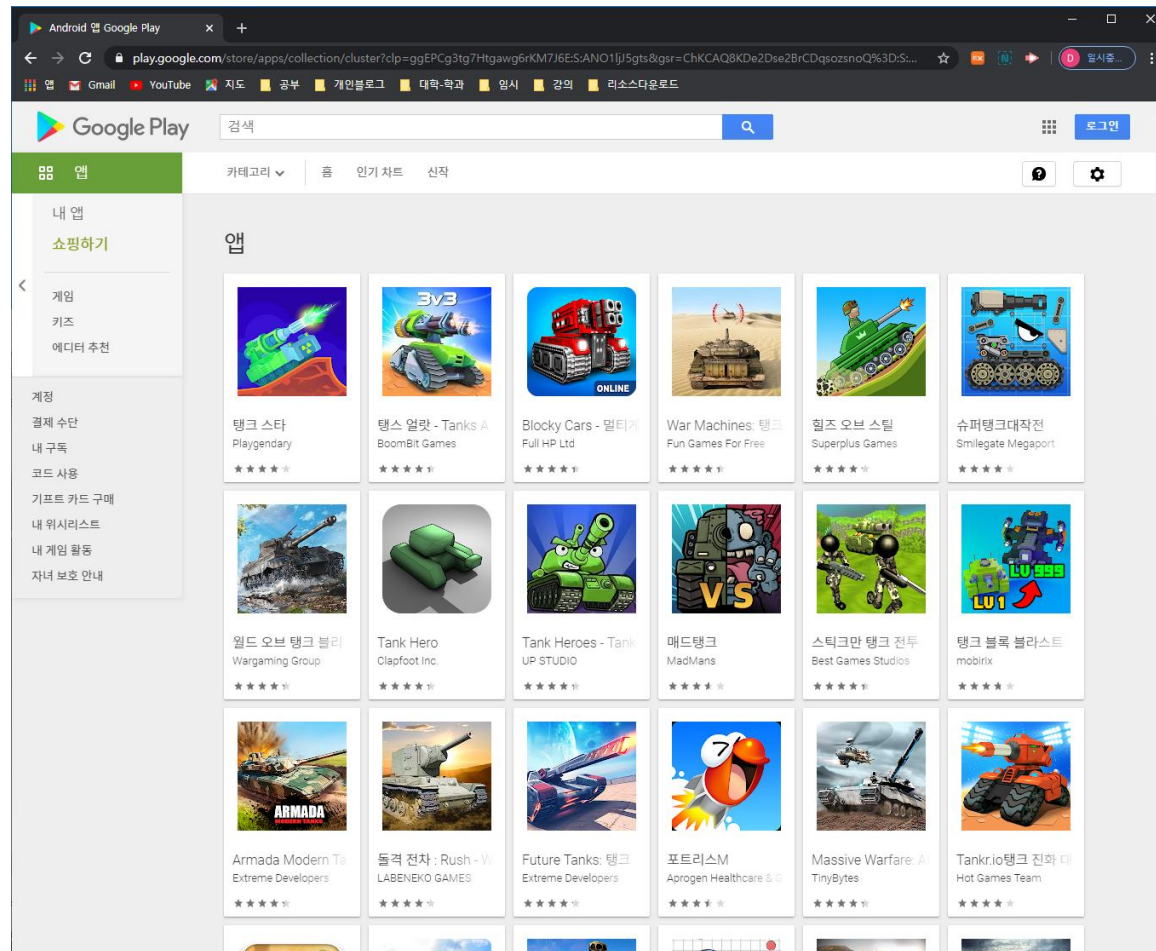
1. 3D 모델과 매터리얼(Material)
2. FPS와 Delta Time
3. 오브젝트 이동 및 회전
4. 슈팅
5. 물리 처리와 사운드 다루기
6. 카메라 트래킹
7. 충돌 판정과 처리
8. 게임 UI 만들기

EXAMPLE OF TANK GAMES

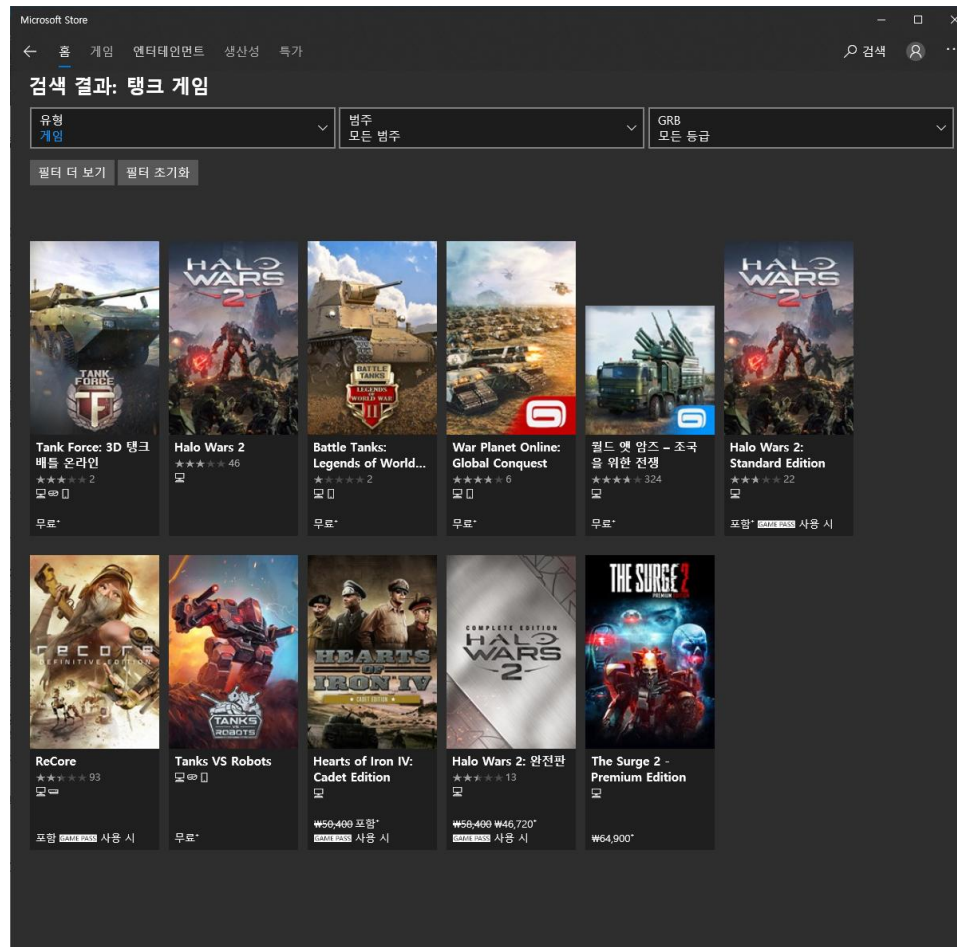
- 간단한 탱크 게임을 만들면서 Unity3D의 세부 기능과 게임 제작에 필요한 기본적인 사항들을 살펴본다



- 구글 스토어 "탱크 게임" 검색

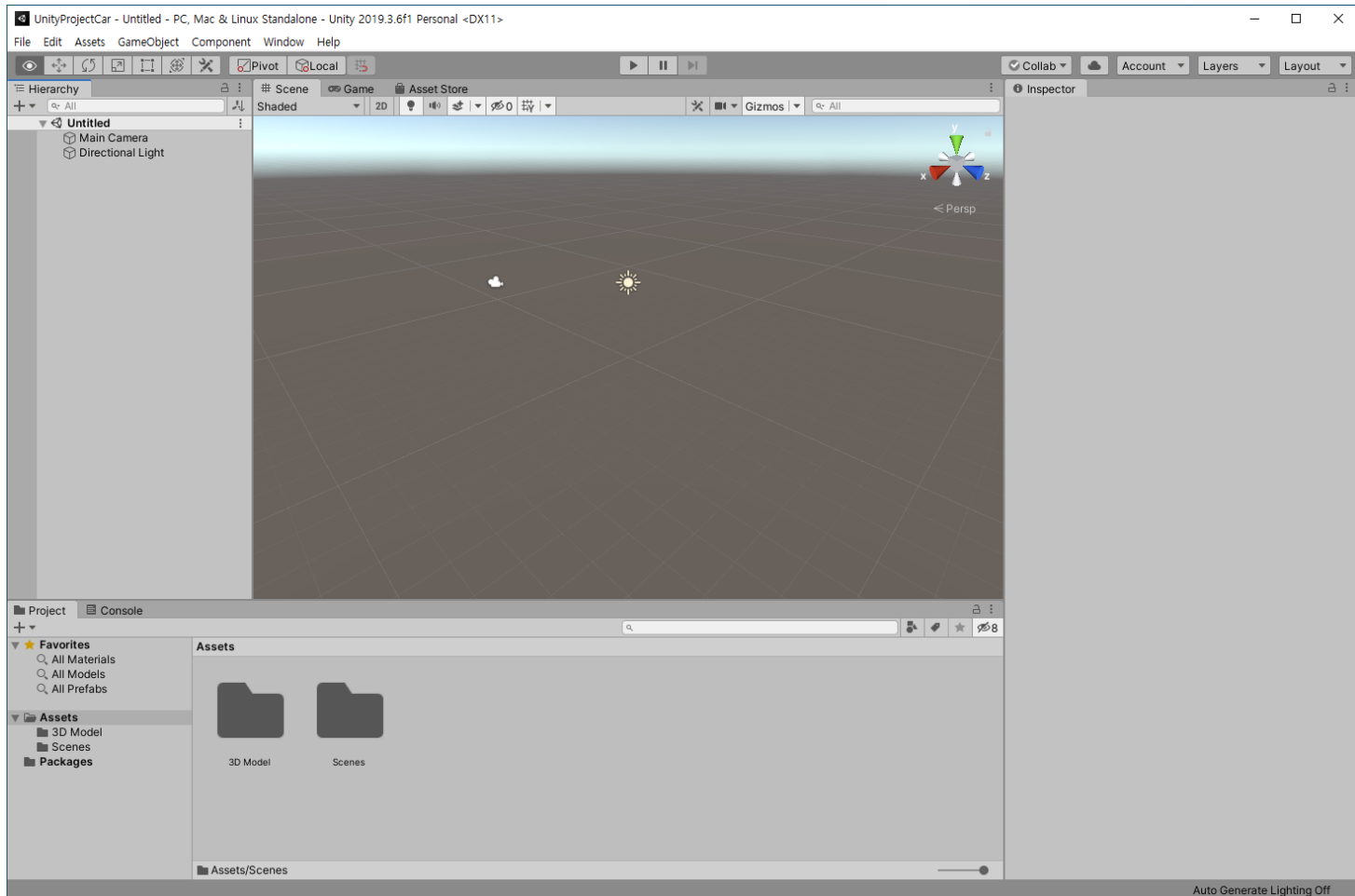


- 마이크로소프트 스토어 "탱크 게임" 검색



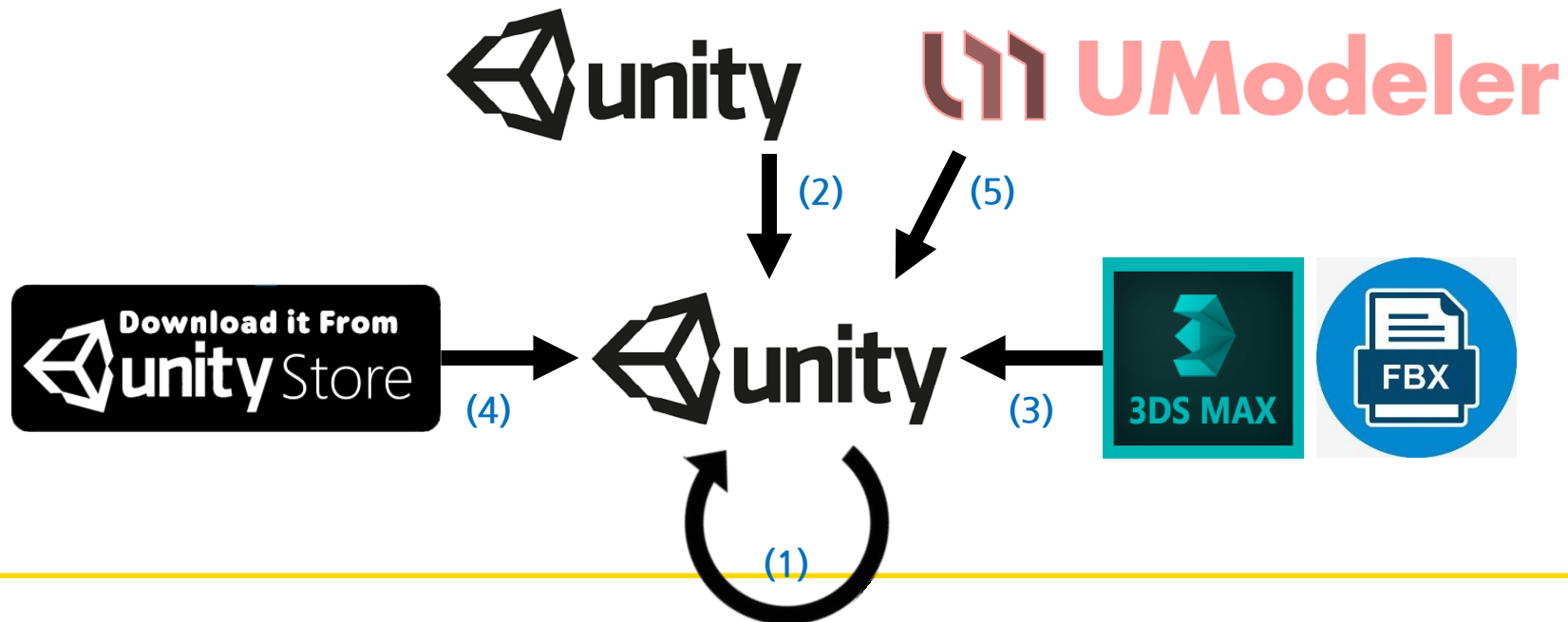


- 새로운 프로젝트 생성



4.1 3D MODEL 다루기

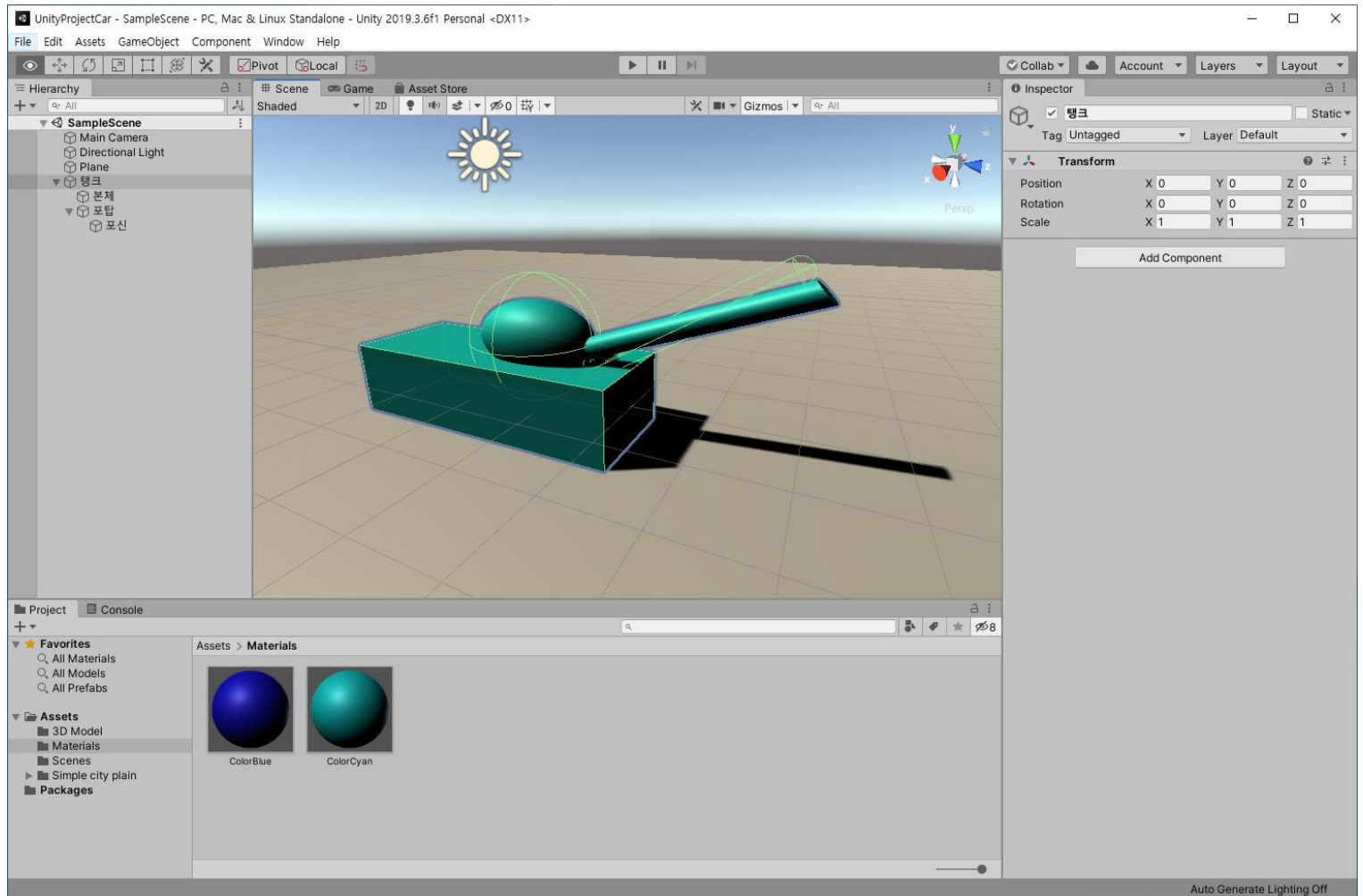
- 게임 오브젝트 생성
 - 1) Unity 에서 제공하는 3D 오브젝트를 활용하여 생성
 - 2) 다른 프로젝트에서 импорт
 - 3) 외부 3D 모델을 импорт
 - 4) Asset Store 에서 다운로드
 - 5) UModeler (3D 모델링을 위한 Unity 플러그인) 에서 제작



탱크 만들기 (1)

- 계층적인 오브젝트
 - 탱크 (Empty Object) : 본체, 포탑, 포신으로 구성
 - 포신 : 포탑과 결합되어 함께 회전
 - 포탑 : 탱크와 함께 이동, 본체와 독립적으로 회전
- 씬 뷰에서 오브젝트 생성

오브젝트	이름	속성	값	비고
Plane	바닥	Position x, y, z Scale x, y, z	0, -0.5, 0 3, 1, 3	매터리얼 적용
Cube	본체	Position x, y, z Scale x, y, z	0, 0, 0 1.5, 1, 4	
Sphere	포탑	Position x, y, z Scale x, y, z	0, 0.75, 0.6 1, 1, 1.7	
Cylinder	포신	Position x, y, z Rotation x, y, z Scale x, y, z	0, 1.1, 2.6 60, 0, 0 0.3, 1.5, 0.3	
Directional Light				탱크가 잘 보이게 적절하게 조정



- 탱크 결합 (하위 객체로 만들기)

- 별도의 빈 오브젝트를 생성 (메뉴 GameObject → Create Empty)

- 빈 오브젝트 속성

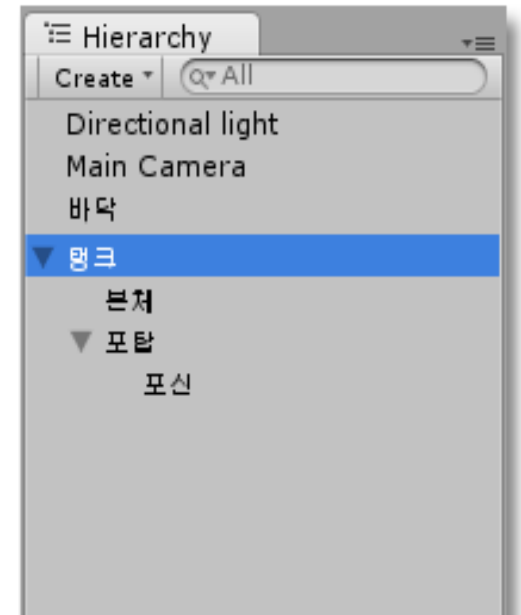
Name(탱크), Position(0, 0, 0)

- 하이라키 창에서 본체를 빈 오브젝트(탱크)로 드래그

- 포신을 포탑으로 드래그

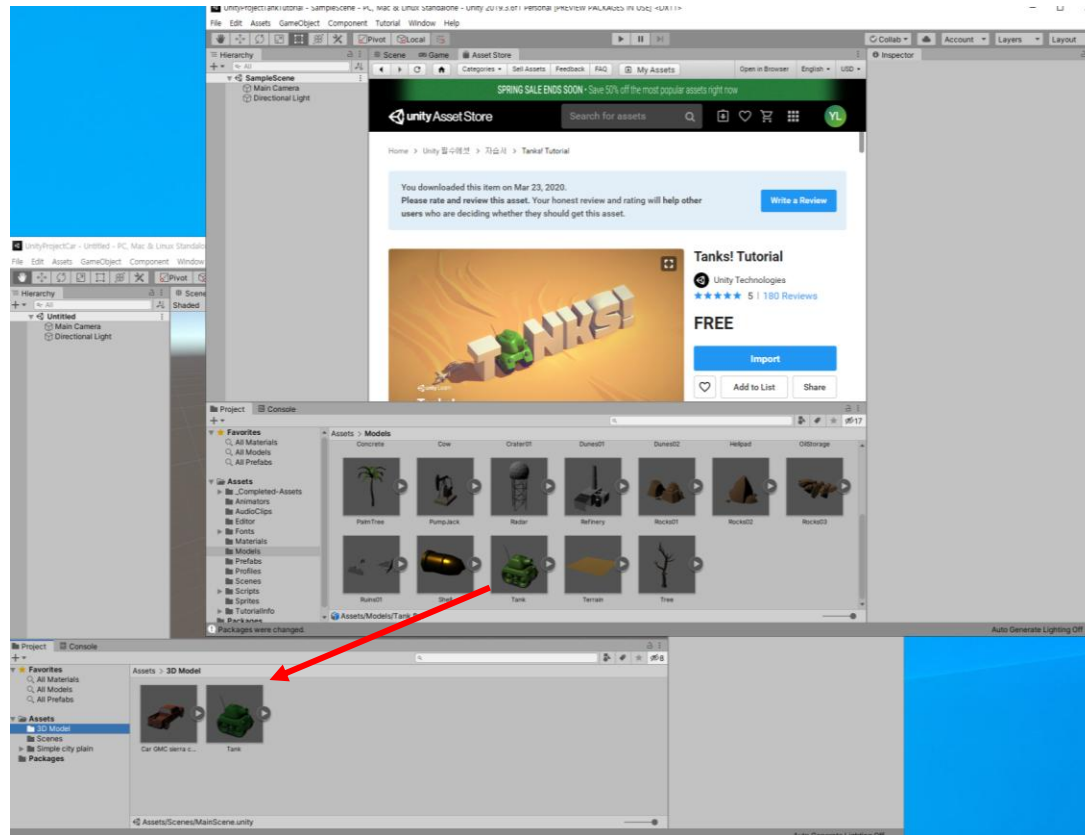
- 포탑을 빈 오브젝트(탱크)로 드래그

- 탱크를 선택하고 오브젝트를 움직여 본다.



탱크 가져오기 (2)

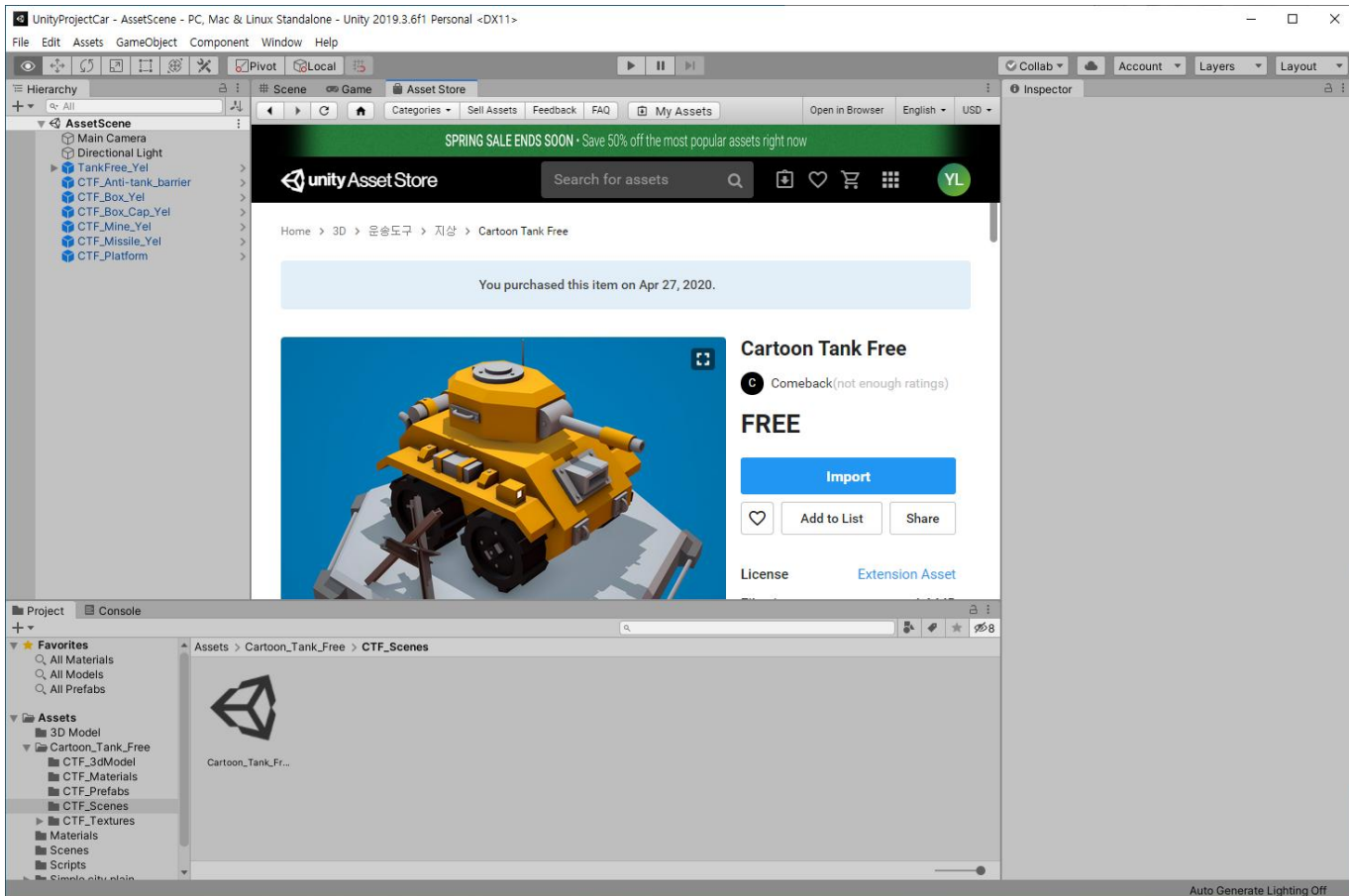
- 다른 프로젝트에서 복사
 - 필요한 오브젝트를 드래그 (또는 Export and Import Package)

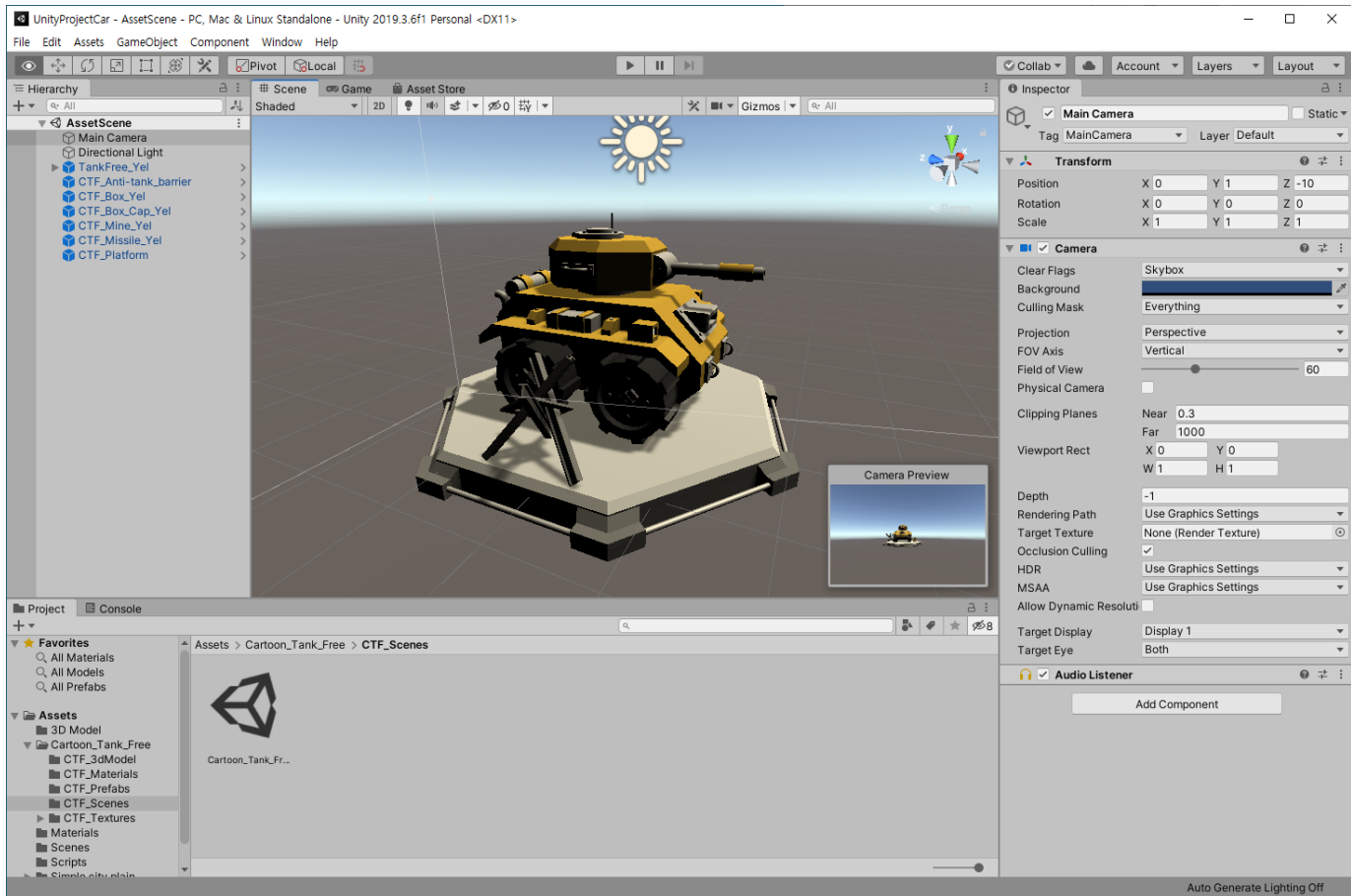




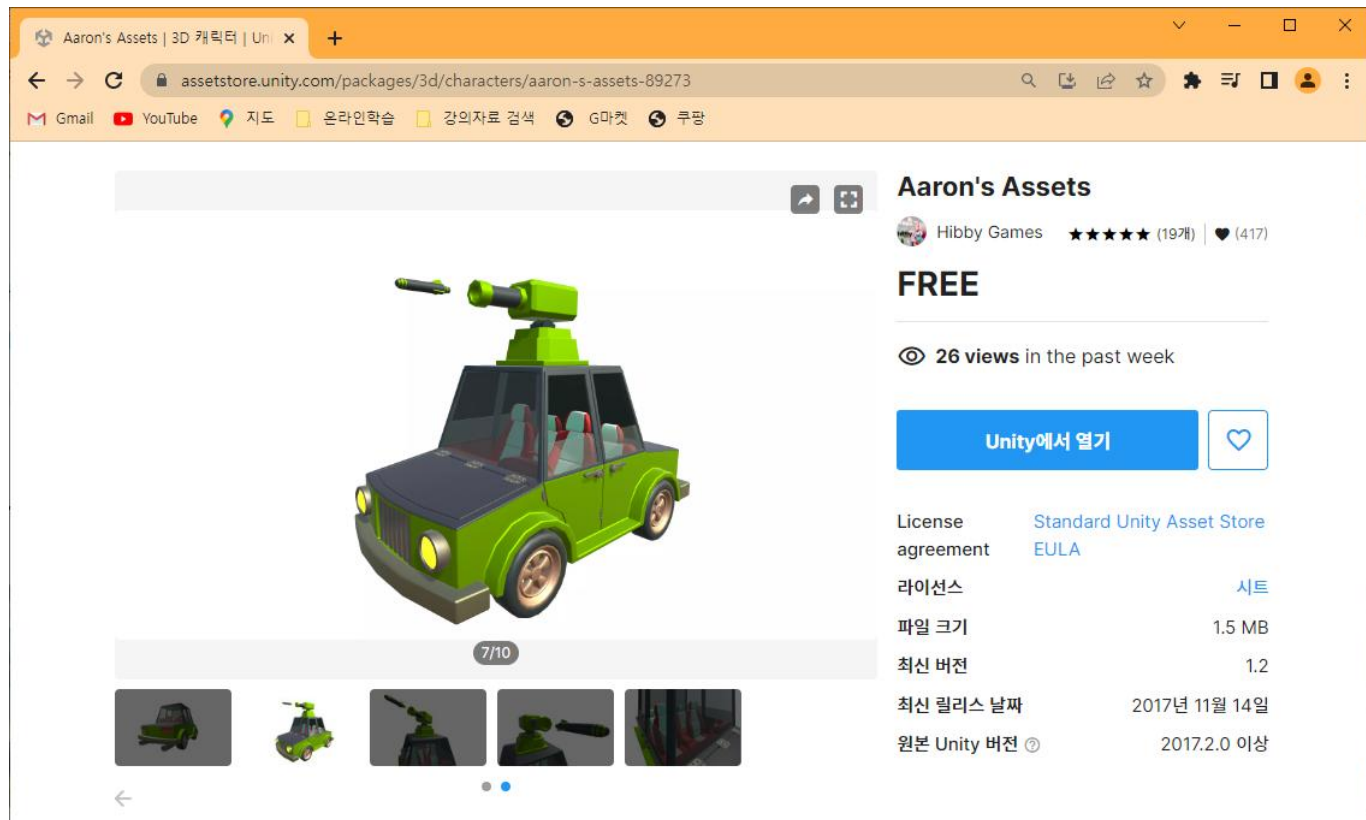
탱크 다운로드 받기 (4)

- Asset Store 에서 다운로드 (무료 또는 유료)
 - 오브젝트를 검색하여, 다운로드 + импорт





– Asset Store 에서 <Car> 등으로 검색

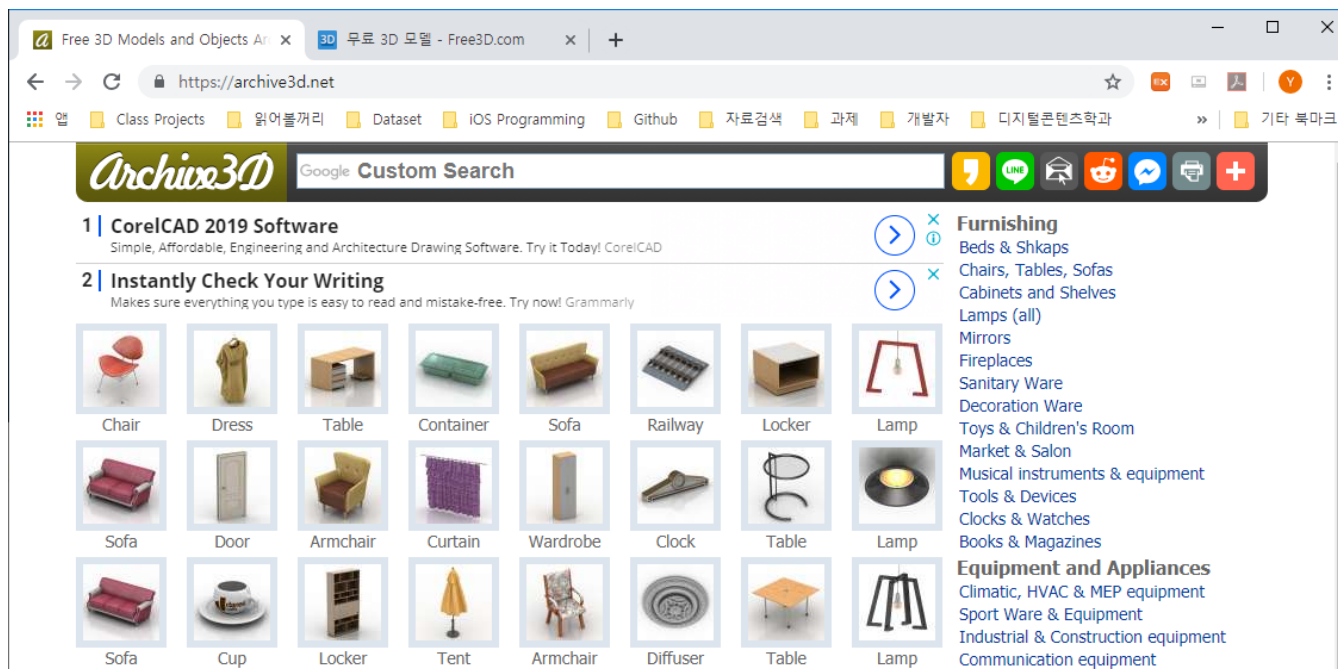


3D MODEL 가져오기 (4)

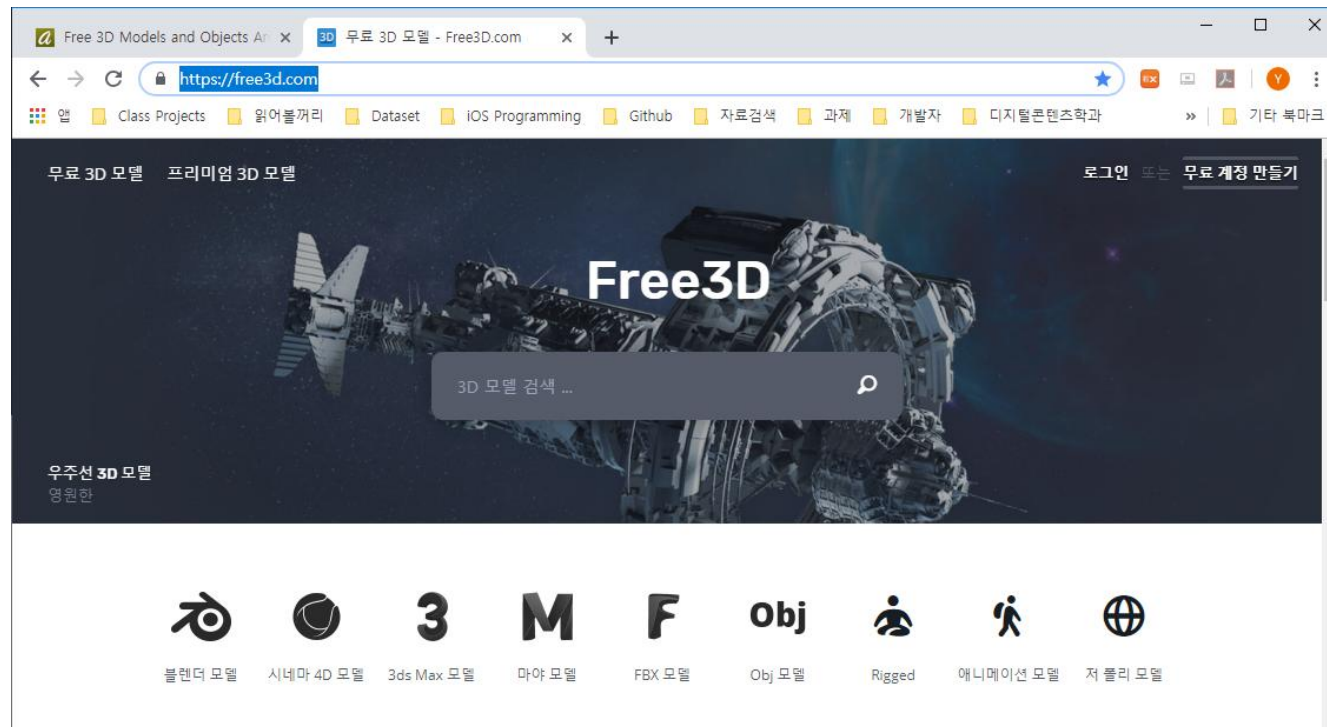
• 4.1.1 3D Model Import

– 무료 다운로드 사이트

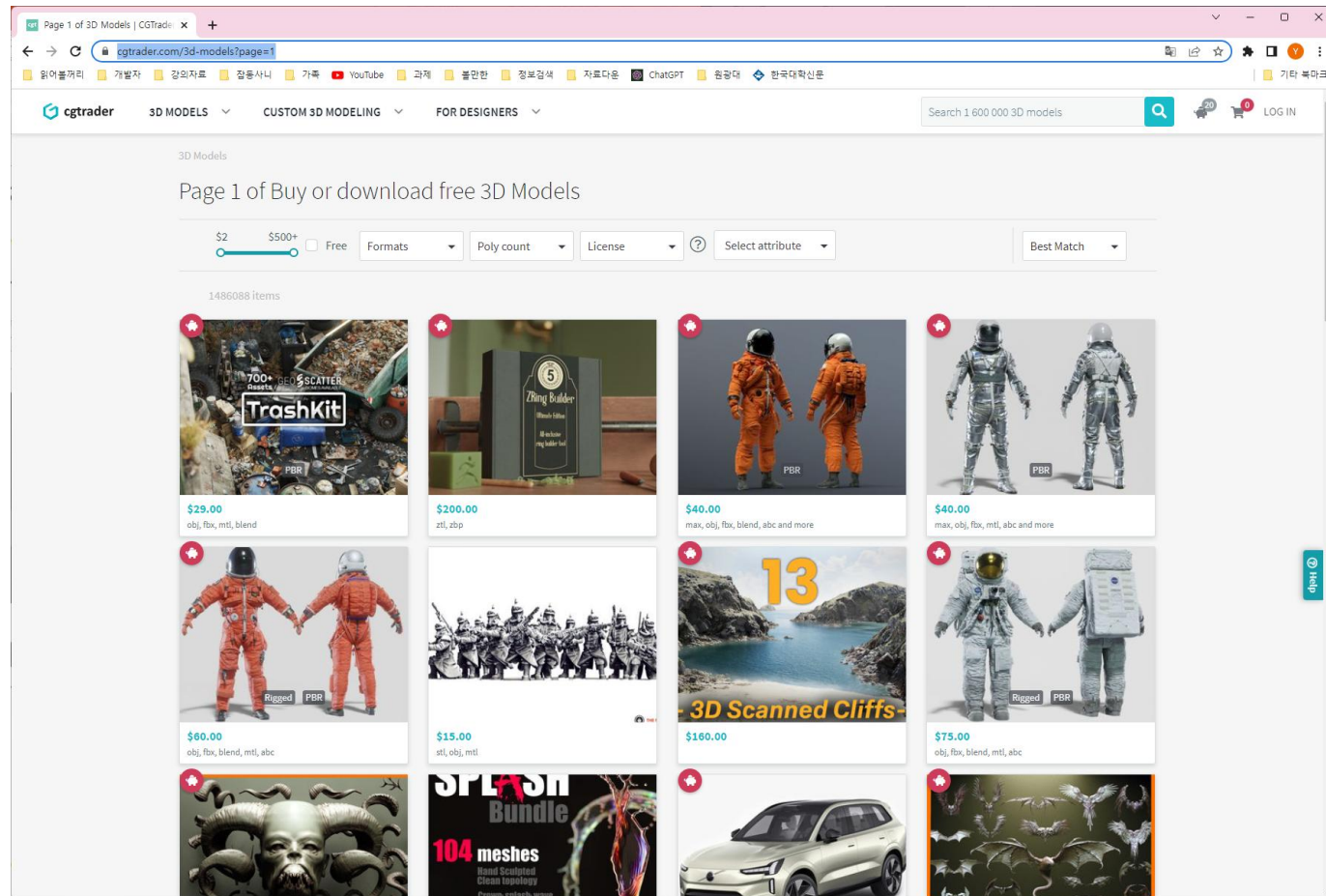
- <https://archive3d.net/> “Car 3D Model” 로 검색



- <https://free3d.com/>



- <https://www.cgtrader.com/>

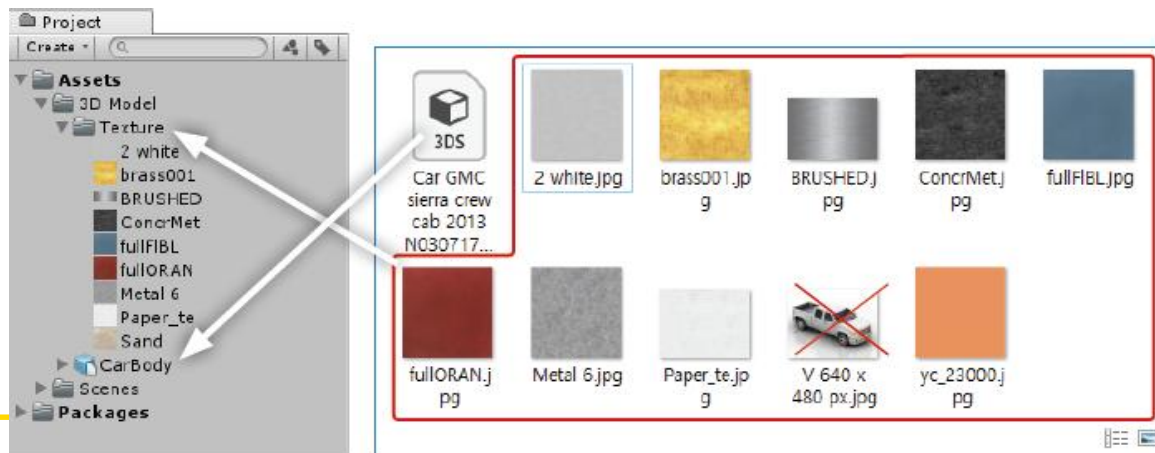


– 3D 모델을 다운로드하고 씬에 추가

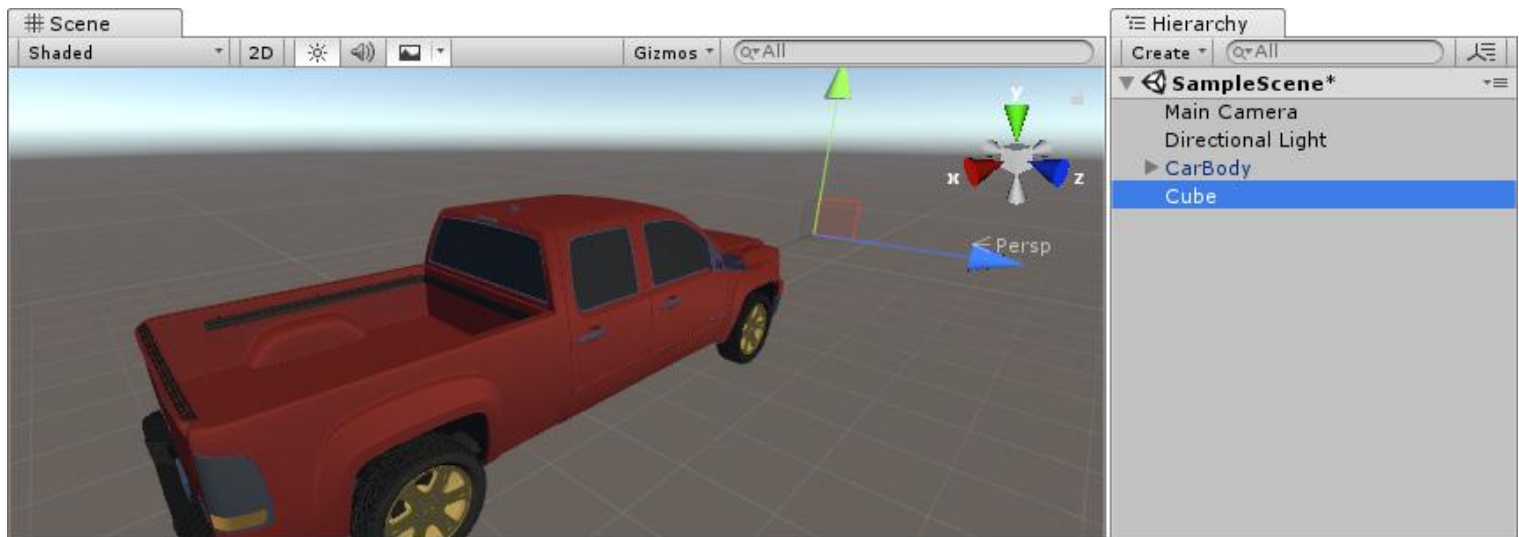
- 적절한 3D 모델 파일을 다운로드



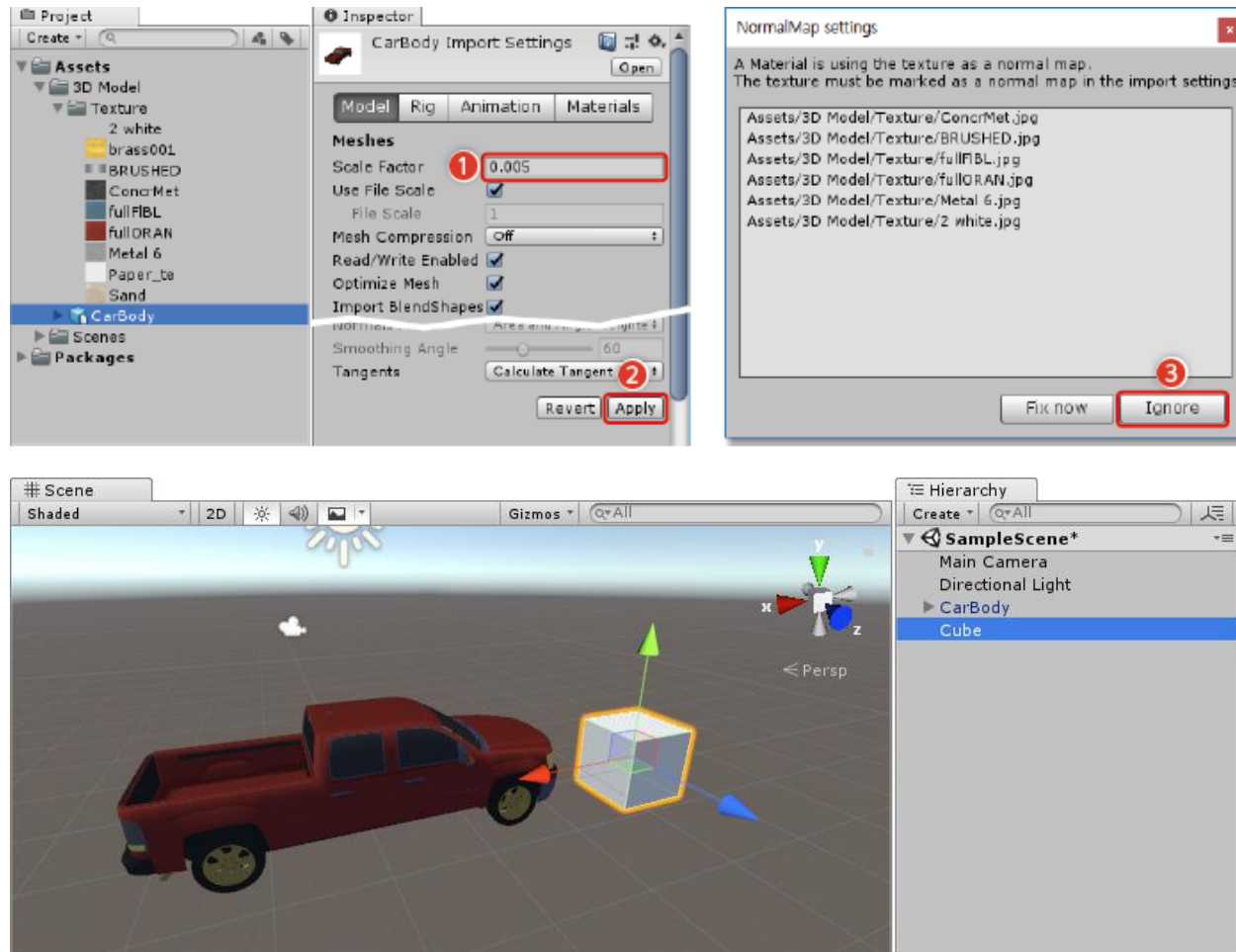
- 압축을 풀고, 이미지와 모델파일을 남기고 삭제 (불필요한 파일 삭제)
- 프로젝트 브라우저에서 3DS파일을 드래그 하여 추가 (이름을 적절하게 변경)
- **Texture** 폴더를 생성하고 이미지 파일들을 드래그 하여 추가



- 3D 모델을 하이라키키로 드래그 하여 씬에 추가



- Cube 모델을 추가하여 적절한 크기로 Scale Factor를 조정 (크기 조정 후, Cube는 삭제)



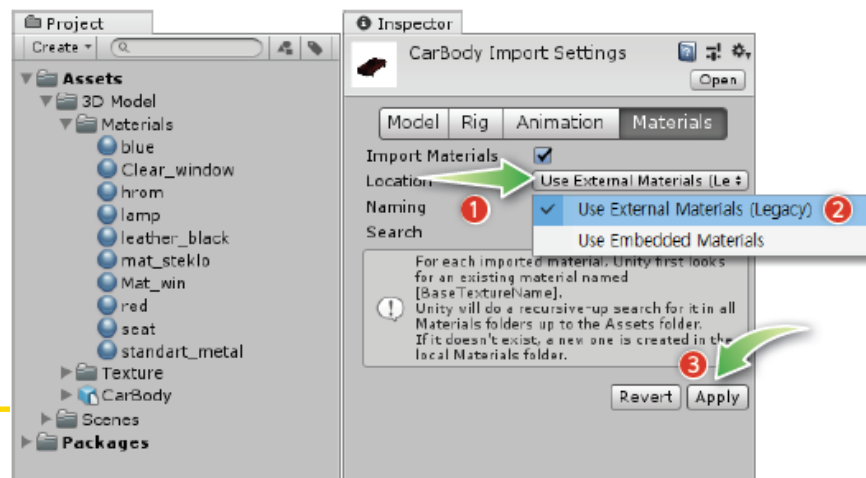
- 유니티에서 사용 가능한 3D 모델
 - 3D 모델링 프로그램에서 생성한 파일은 다양한 종류가 있음
 - 3ds, fbx, dae, dxf, obj, skp 등의 확장자는 유니티에서 사용 가능
 - PC에 Max, Maya, Blender, Cinema4D, Modo, Lightwave 등의 모델링 소프트웨어가 설치되어 있으면, 유니티에서 바로 사용이 가능
 - 해당 소프트웨어에서 수정하면 자동으로 유니티에 반영됨

– 4.1.2 Model의 Material 변경

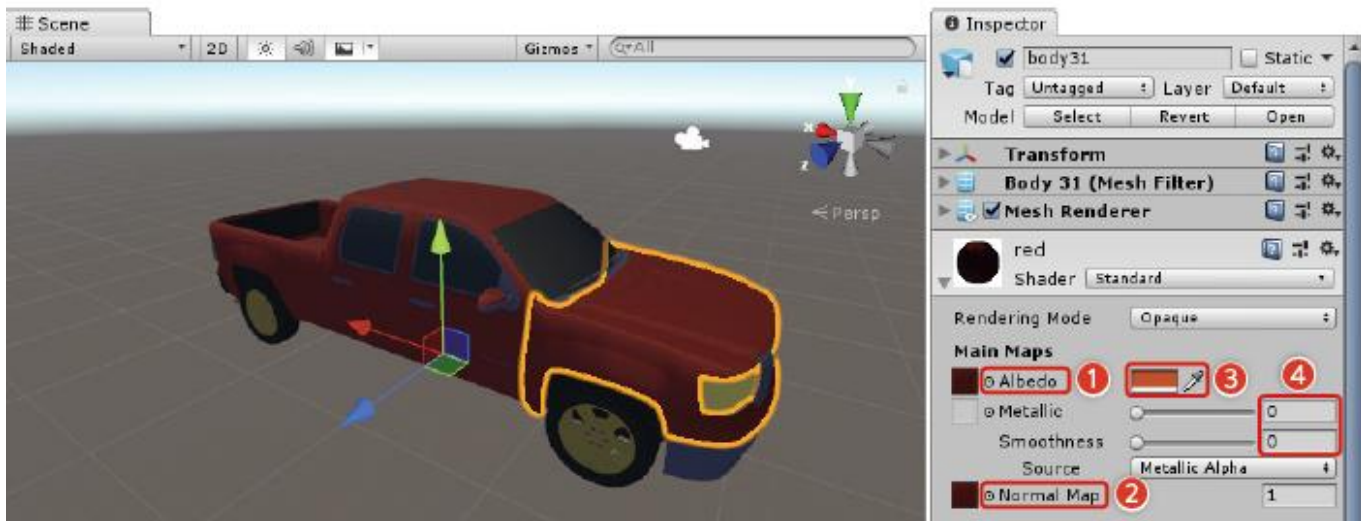
- 다운로드 받은 3D 모델의 색상이 실제의 오브젝트와 다르므로, 모델의 매터리얼을 수정



- 프로젝트 브라우저에서 CarBody를 선택하고, 인스펙터의 Material 탭에서 Location 속성을 Use External Materials (Legacy)로 설정
- [Apply] 버튼을 클릭하면, 프로젝트 브라우저에 Materials 폴더와 모델에 사용된 매터리얼이 생성됨



- CarBody를 선택한 후, 하위 부분을 클릭하면 해당 부위가 선택됨
- 인스펙터에서 매터리얼 속성을 펼치면 매터리얼을 수정할 수 있음

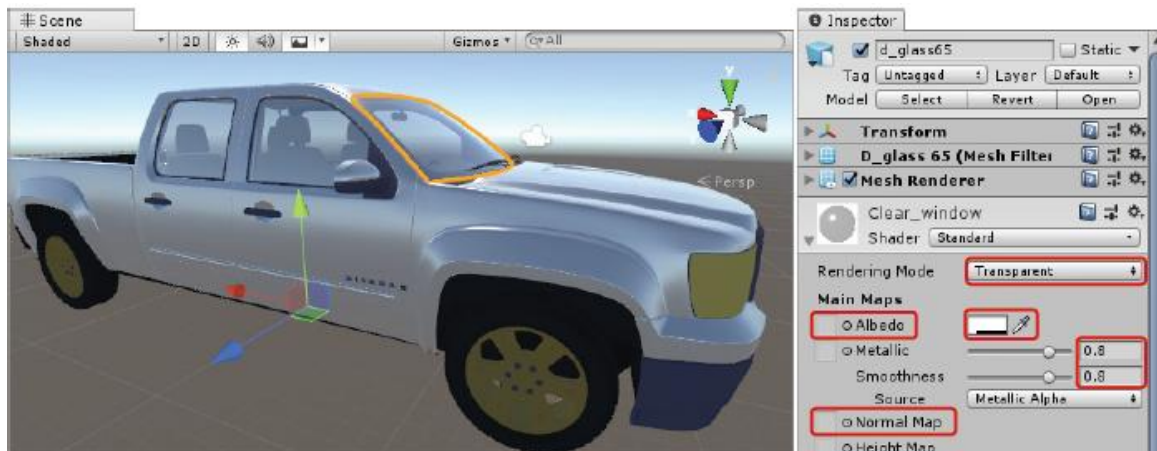


- Albedo : 오브젝트를 매핑할 텍스처
- Normal Map : 요철이 있는 오브젝트의 요철 처리용 텍스처
- Color : 오브젝트 또는 텍스처의 색상
- Metallic : 오브젝트의 금속성(광택), 0=비금속(무광), 1=금속(유광)

- 자동차 색상을 광택이 나는 흰색으로 변경한 예

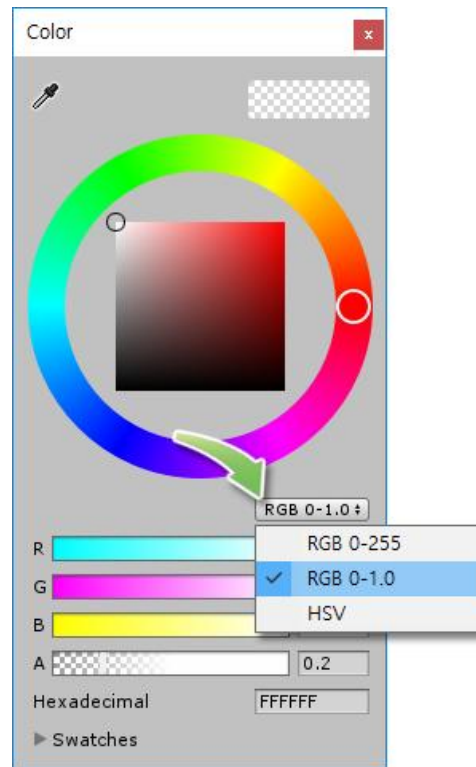


- 유리를 투명하게 설정한 예



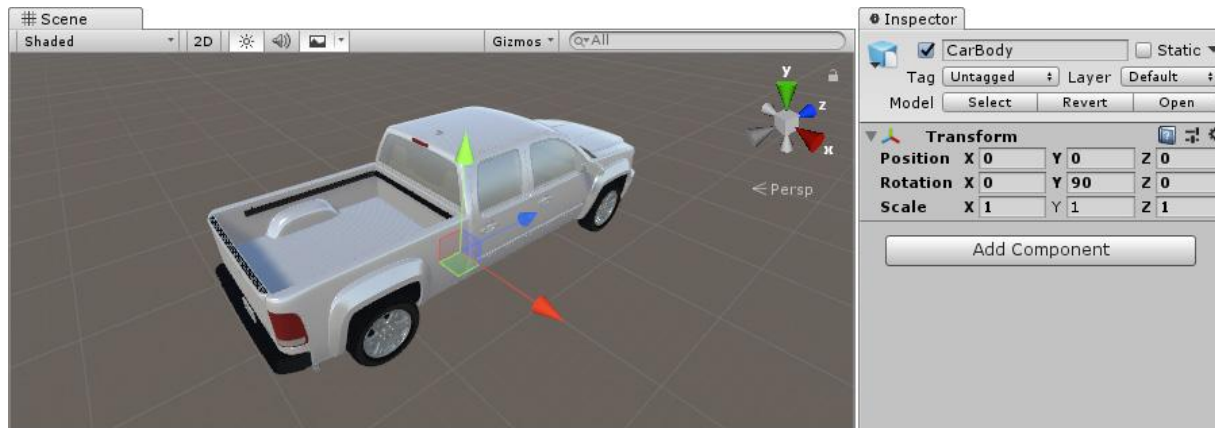
– RGB 칼라 표기법

- RGB 칼라는 0~255로 나타냄
- 칼라 픽커의 표시 방법을 RGB 0~1.0을 설정하면 보다 직관적



- 오브젝트 방향 설정

- 유니티에서 Z축이 게임의 전방향이므로, 플레이어가 조정하는 오브젝트는 Z축을 향하도록 배치해야 함
- CarBody의 Rotation Y을 조정하여 CarBody가 Z축을 향하도록 회전



- 빈 오브젝트 생성
- 오브젝트 이름을 Car로 변경하고 Position을 (0,0,0)으로 설정
- CarBody를 Car 오브젝트로 드래그 하여 하위 오브젝트로 만듦

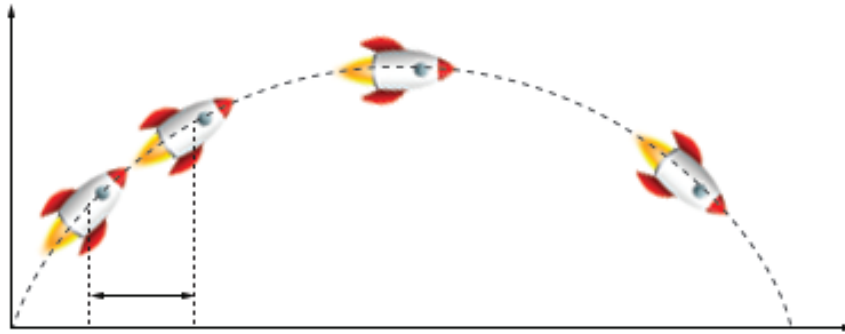
4.2 오브젝트 이동과 속도

- 게임은 일련의 절차를 무한정 수행하는 루프의 연속
- 하드웨어에 의해 수행되는 하나의 루프를 프레임이라고 함

4.2.1 FPS 와 Delta Time

- FPS(Frame Per Second) = 하드웨어가 1초에 프레임을 몇 회 수행하는가를 나타냄
- FPS가 높을수록 하드웨어의 성능이 좋음
- 일반적으로 화면이 끊어지지 않고 부드럽게 인지하는 속도가 초당 60 프레임
게임에서 초당 60 프레임을 수행하려면, 각 프레임의 수행시간이 16ms 이내 이어야 함

- 포물선 운동을 하는 로켓의 궤적



- 로켓의 위치와 회전각은 매 순간 변함
- 물리학에서의 로켓의 이동 경로를 작은 구간으로 나누고 미분방정식을 이용해서 각 구간의 변화율을 구함
- Delta Time은 이렇게 나뉜 구간의 경과 시간
- 유니티에서는 `Time.deltaTime`으로 해당 값을 구함

- 게임 루프는 Delta Time을 이용하여 다음과 같이 구성

```
초속도 = 10 // 오브젝트 이동 속도 (단위:m)
```

```
Loop:
```

```
    현재 프레임에서 이동할 거리 = 초속도 * Time.deltaTime
```

```
    오브젝트를 현재 프레임에서 이동할 거리만큼 이동
```

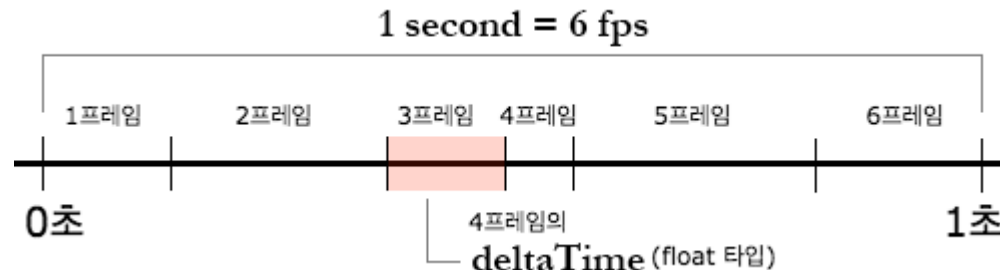
```
EndLoop
```

- 이와 같이 구성할 경우, 하드웨어의 성능 차이와 상관없이 오브젝트가 일정한 속도로 이동할 수 있음 (높은 FPS는 짧은 거리를 자주 이동하고, 낮은 FPS는 긴 거리를 가끔 이동하게 됨)
- 움직임의 부드러움에 차이는 있지만, 1초 후의 이동거리는 동일하게 됨



Time.deltaTime

- The time in seconds it took to complete the last frame. Use this function to make your game frame rate independent (마지막(바로 전) 프레임을 완료하는데 걸리는 시간으로, 게임이 프레임 독립적으로 작동하기 위해 사용)
- 이전 프레임의 시작 시간부터 현재 프레임이 시작 시간까지의 간격 : deltaTime
- 만약 초당 6프레임인 상태에서, 현재 4프레임이라면, Time.deltaTime은 3프레임 (4프레임의 바로 전 프레임) 을 완료하는데 걸리는 시간



- Time.deltaTime 을 사용하는 이유

```
public class ExampleClass : MonoBehaviour {  
    void Update() {  
        //float translation = 10f;  
        float translation = Time.deltaTime * 10;  
        transform.Translate(0, 0, translation);  
    }  
}
```



- Update() 에서 Translate() 메소드를 통해 게임오브젝트를 Z 방향으로 10 유닛 만큼 이동
- translation 변수에는 deltaTime 만큼을 곱하고 있음
- 만약 deltaTime 을 곱하지 않으면 ?
 - 10 fps 로 동작한다면, 1초당 update()가 10번 호출되어, 100 유닛 만큼 이동
 - 60 fps 로 동작한다면, 초당 60번 호출되어, 600 유닛 만큼 이동
- deltaTime 을 곱하면 ?
 - 10 fps 에서, 1초당 update()가 10번 호출되면서 deltaTime (1/10)이 곱해져서, 10번 $\times (1/10) \times 10$ 유닛 만큼 이동
 - 60 fps 에서, 초당 60번이 호출되면서 deltaTime (1/60)이 곱해져서, 60번 $\times (1/60) \times 10$ 유닛 만큼 이동
- FPS에 상관없이 동일한 단위의 이동을 지원하기 위해 (빠른 기기와 느린 기기 간에 동등한 조건을 지원)

- deltaTime 을 곱하므로, 동일한 유닛의 이동을 지원하도록



- 늦은 FPS 에서는 (상대적으로) 큰 deltaTime 을 곱해주고, 빠른 FPS 에서는 (상대적으로) 작은 deltaTime 을 곱해준다
- 이를 통해 (FPS가 빠르건 느리건 상관없이) 동일한 이동 거리를 갈 수 있도록 조정



update() 안에서 deltaTime을 곱하여, 초당 정해진 유닛 만큼 이동하도록 지원하기 위해 deltaTime을 고정 값으로 사용하지 않는 이유는 프레임 간 처리 시간이 항상 일정하지 않기 때문임

– 이동 방향과 Vector

- 3차원 공간에 있는 오브젝트의 위치나 방향을 나타내려면 (x,y,z) 각 축의 값이 필요함
- 유니티에서는 이 값을 Vector3(x,y,z)로 표시하며, 인스펙터의 Position, Rotation, Scale 값이 이에 해당됨
- 2D 게임에서는 z축을 사용하지 않으며, Vector2(x,y)로 표시

RGBA 칼라를 표시할 때는 Color(r,g,b,a) 또는 Vector4(r,g,b,a) 형식으로 사용

- Vector는 오브젝트의 방향 표시에 자주 사용되며, (x,y,z) 값이 각각 0 또는 1로 표시하고 반대방향은 음수로 지정함



– 4.2.3 오브젝트 이동

- 스크립트를 이용하여 자동차를 움직이도록 함
- C# 스크립트를 추가하고 이름을 CarMove로 변경하여 코드를 작성함



```
public class CarMove : MonoBehaviour
{
    float moveSpeed = 10f;  // 이동속도

    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {
        // 현재 프레임에서 이동할 거리
        float amount = moveSpeed * Time.deltaTime;

        // 오브젝트의 전방으로 이동
        transform.Translate(Vector3.forward * amount);
    }
}
```

Translate() 함수는 현재의 위치에서 지정한 방향과 거리만큼 오브젝트를 이동시키는 함수



게임 오브젝트를 이동시키는 방법

- 1) transform의 Position에 직접 접근하여 (값을) 설정해주는 방법
 - 2) transform의 Translate() 함수를 이용하는 방법
 - 3) Rigidbody를 설정한 후, velocity(속도) 값을 설정해주는 방법 (물리 시뮬레이션에서 주로 사용하는 방법)
- (1번) (2번)은 transform을 이용하는 방법으로, 물리 법칙이 적용되지 않는 대상일 때 사용
 - transform을 이용해 강제로 이동하면, 물리 법칙을 무시하고 강제로 절대 위치를 바꿔버리는 것이기에 위치가 옮겨진 후 물리법칙에 문제가 발생할 수 있음
 - (3번)은 Rigidbody 컴포넌트를 이용하는 방법으로, 물리 법칙이 적용되어 있는 대상일 때 사용
 - 물리가 적용되어 있는 Rigidbody 이기 때문에, Rigidbody 컴포넌트를 이용하여 이동하는 것이 좀 더 자연스러운 물리 법칙이 적용될 수 있음



게임 오브젝트를 이동시키는 방법

- transform의 Position에 직접 접근하여 (값을) 설정해주는 방법

```
transform.position += new Vector3(0,0,1);
```

- transform의 Translate() 함수를 이용하는 방법

```
Transform.Translate(new Vector3(0,0,1));
```

- Rigidbody를 설정한 후, velocity(속도) 값을 설정해주는 방법 (물리 시뮬레이션에서 주로 사용하는 방법)

```
// Rigidbody를 설정한 후, 속도값을 설정
```

```
Rigidbody rigidBody = gameObject.GetComponent<Rigidbody>();
```

```
rigidBody.velocity = new Vector3(0,0,1);
```

```
// Rigidbody에 Force를 적용하여 움직임
```

```
rigidBody.AddForce(new Vector3(0,0,1));
```

- Character Controller를 이용하여 움직이는 방법

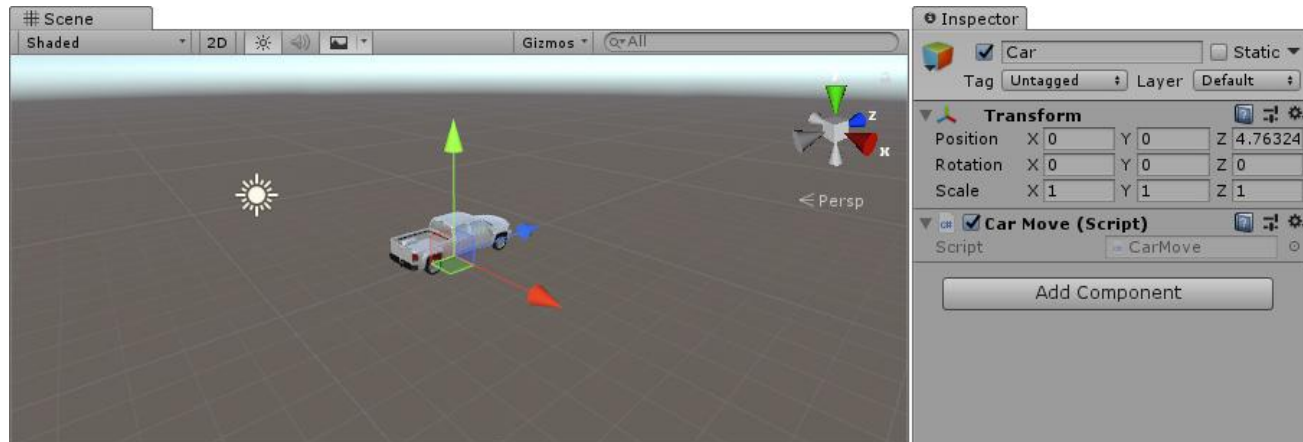
```
CharacterController charController =
```

```
gameObject.GetComponent<CharacterController>();
```

```
charController.Move(new Vector3(0,0,1));
```



스크립트를 하이러키의 Car로 드래그 하여 연결하고 게임을 실행



– 절대좌표 (World좌표)

- 게임 화면을 기준으로 한 좌표
- 화면의 전방이 항상 Z축으로 됨 (X축은 오른쪽)
- 예) “강남역 2번 출구 오른쪽 10m 지점”은 오브젝트의 위치나 방향에 상관없이 항상 같은 곳을 나타낸다

– 상대좌표 (Local좌표)

- 오브젝트를 기준으로 한 좌표
- 예) “오브젝트의 3시 방향 10m 지점”은 오브젝트의 방향과 위치에 따라 수시로 바뀐다
- 게임의 시작위치가 반드시 $\text{Vector3}(0,0,0)$ 일 필요가 없으며, 플레이어의 위치는 항상 바뀌므로, 게임에서는 주로 상대좌표를 더 많이 사용함



Car의 Rotation Y를 변경하여 자동차 객체를 회전시킨 다음, 게임을 실행

- 자동차 객체는 자신의 전방으로 전진함 (=Translate() 함수는 상대좌표를 사용한다는 것을 의미)
(=) Vector3.forward를 오브젝트의 전방으로 처리함
- 자전거 바퀴처럼 x축이나 y축으로 회전하는 물체가 전후방으로 직선운동을 하는 경우, 상대좌표를 사용하면 문제가 발생
 - 오브젝트가 회전하면서 오브젝트의 전방이 지속적으로 바뀌게 되어 직선운동을 하지 못함
- 이럴 경우, 절대좌표를 사용해야 함

```
Translate(Vector3.forward * amount); // 상대좌표  
Translate(Vector3.forward * amount, Space.Self); // 상대좌표  
Translate(Vector3.forward * amount, Space.World); // 절대좌표
```



앞의 스크립트를 Space.World 옵션으로 설정하고 게임을 실행

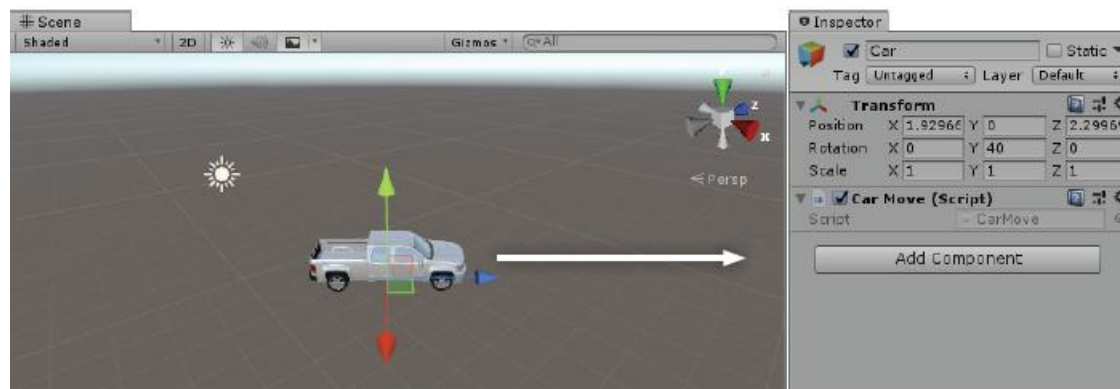


그림 4-18 상대 좌표의 Vector3.forward는 자신의 z축 방향이다.

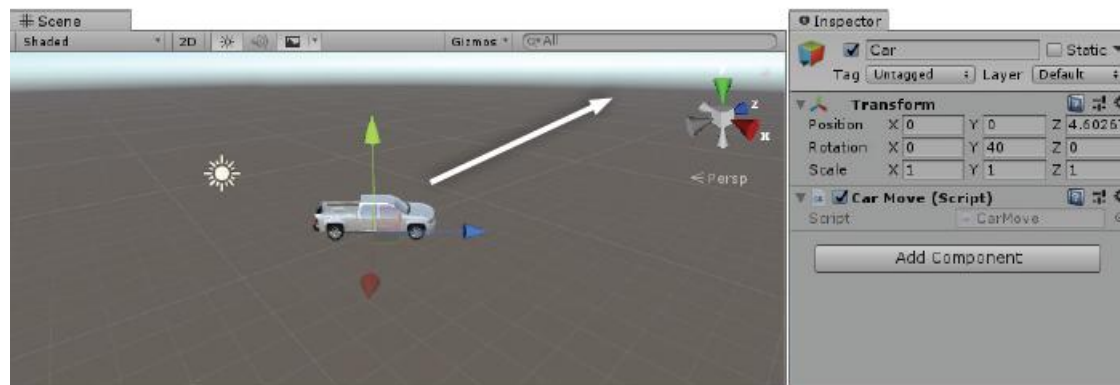


그림 4-19 절대 좌표의 Vector3.forward는 게임 화면의 z축 방향이다.

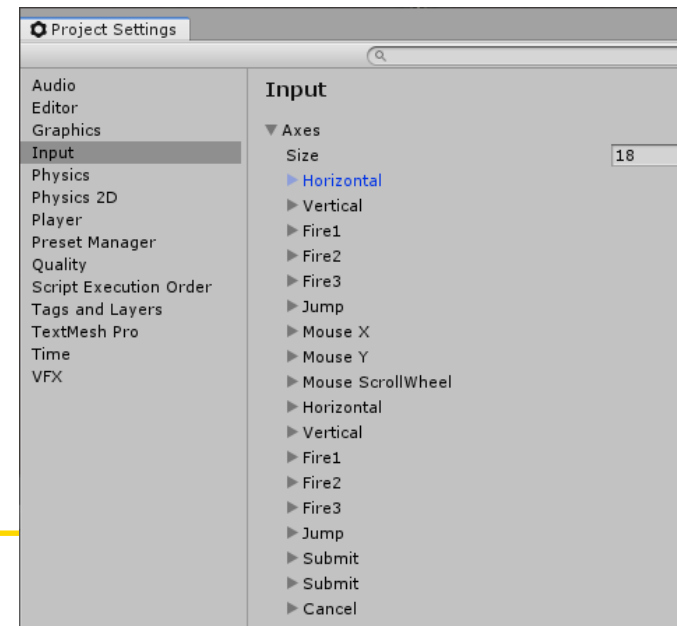
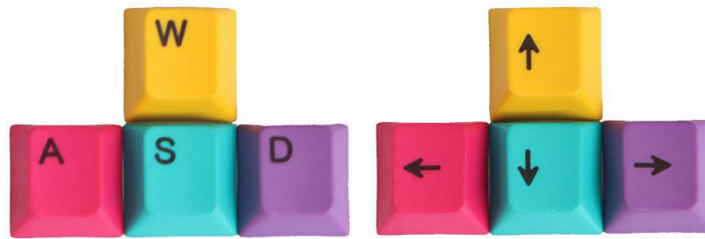


키보드 입력

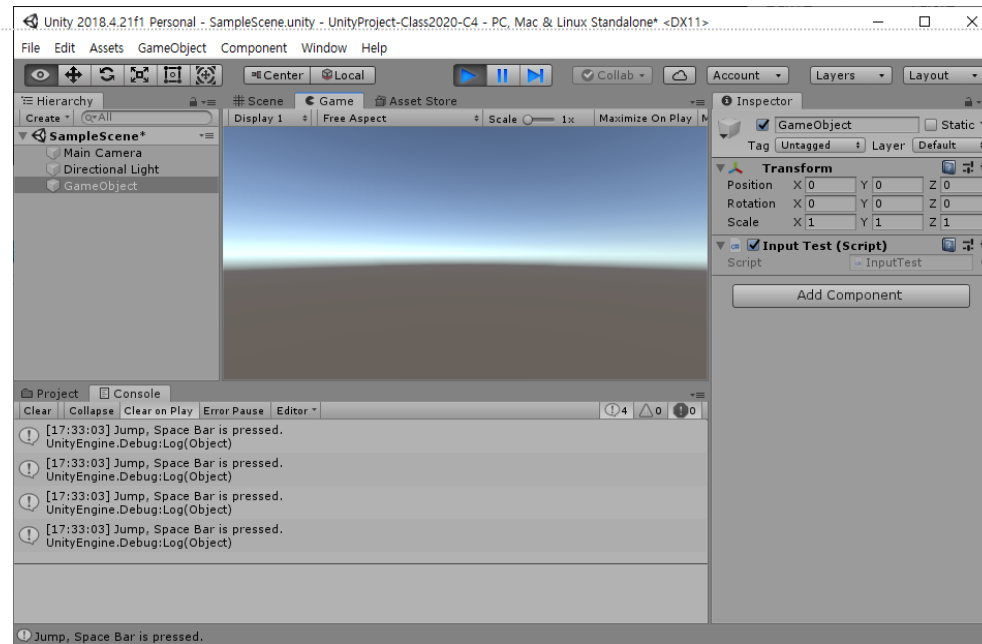
- 미리 설정한 키 조합(버튼) 사용
- 특정 키 입력

(1) 미리 설정한 키 조합(버튼) 사용

- [Edit - Project Setting - Input] 에서 미리 설정된 키 조합 사용 (수정가능)
- GetButton(), GetButtonDown(), GetButtonUp()
- 버튼으로 설정된 이름으로 동작
- 예) `if (Input.GetButton("Fire1")) { ... }`



```
public class InputTest : MonoBehaviour
{
    void Update() {
        if (Input.GetButtonDown("Jump")) {
            Debug.Log("Jump, Space Bar is pressed.");
        }
    }
}
```



GetButton(), GetButtonDown(), GetButtonUp() 을 차례로 실행해 보세요



키보드 입력

(1) 미리 설정한 키 조합(버튼) 사용

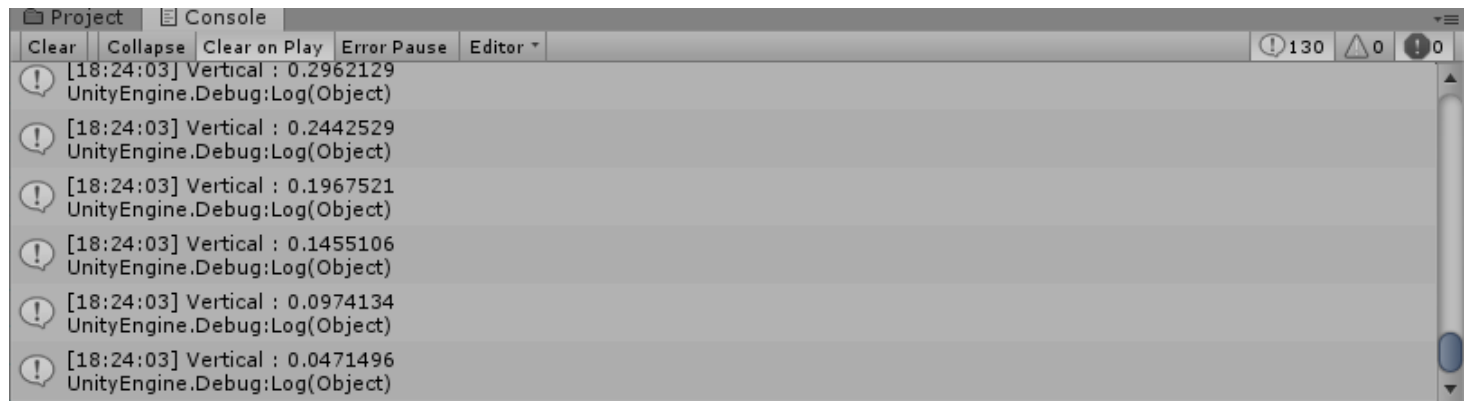
- `Input.GetAxis("키이름")`
 - » "키 이름"에 해당하는 키보드를 누르면 $-1.0f \sim 1.0f$ 까지의 값을 반환하고, 누르지 않으면 $0.0f$ 를 반환
 - » `GetAxis()` : $-1 \sim 1$ 사이 값을 반환 (부드러운 이동이 필요할 때)
 - » `GetAxisRaw()` : $-1, 0, 1$ 값을 반환 (즉시 반응해야 할 때)
- 예) `h = Input.GetAxis("Horizontal");`
 - » `Horizontal` 에 해당하는 키를 누를 시, $-1.0f \sim 1.0f$ 까지의 값을 반환
 - » 오른쪽 화살표(또는 `D`)를 누르면, $0 \sim 1$ 사이의 값을, 왼쪽 화살표(또는 `A`)를 누르면 $-1 \sim 0$ 사이의 값을 리턴

```

public class InputTest : MonoBehaviour
{
    float v;

    void Update() {
        v = Input.GetAxis("Vertical");
        if ( h != 0.0 ) {
            Debug.Log("Vertical : " + v);
        }
    }
}

```



GetAxis(), GetAxisRaw() 을 차례로 실행해 보세요



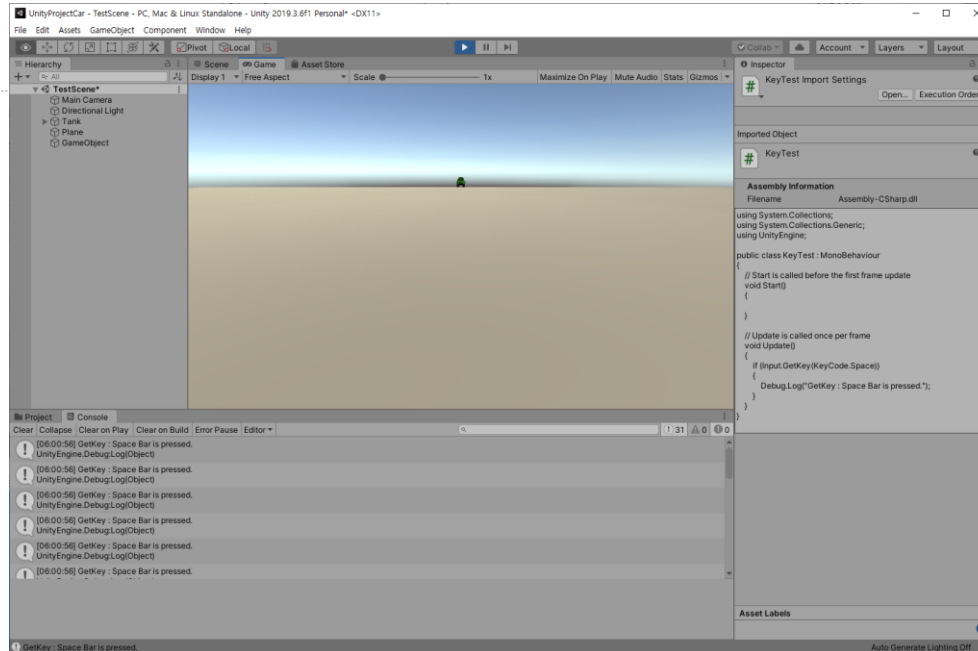
키보드 입력

(2) 특정 키 입력

- GetKey(), GetKeyDown(), GetKeyUp()
- 키보드의 키코드를 인식하여 동작
 - » 키 입력은 [KeyCode.특정키]
 - » 방향 키는 UpArrow, DownArrow, RightArrow, LeftArrow
- 예) if (Input.GetKeyDown(RightArrow)) { ... }
 - » 오른쪽 방향키를 한번만 눌러 실행하고자 할 때



```
public class InputTest : MonoBehaviour
{
    void Update() {
        if (Input.GetKeyDown(KeyCode.Space)) {
            Debug.Log("Getkey : Space Bar is pressed.");
        }
    }
}
```



GetKey(), GetKeyDown(), GetKeyUp() 을 차례로 실행해 보세요

- 키보드로 오브젝트 이동

- 플레이어가 움직임을 제어하기 위해, 키보드를 통해 게임 오브젝트를 이동
키보드로부터 입력받은 값을 새로운 방향 벡터로 설정



```
public class CarMove : MonoBehaviour
{
    float moveSpeed = 10f; // 이동속도

    // Start is called before the first frame update
    void Start()
    {

    }

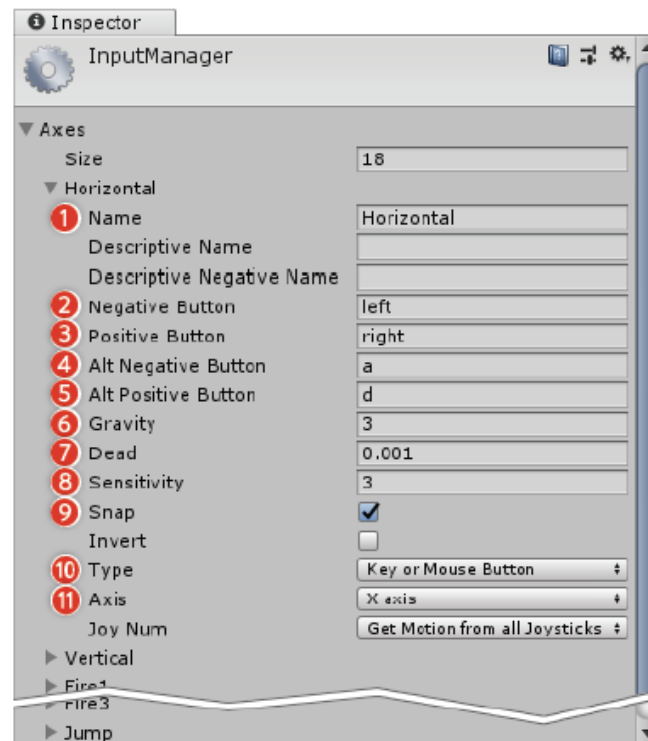
    // Update is called once per frame
    void Update()
    {
        // 현재 프레임에서 이동할 거리
        float amount = moveSpeed * Time.deltaTime;

        // 전후(Vertical) 좌우(Horizontal) 이동키를 받음
        float vert = Input.GetAxis("Vertical");
        float horz = Input.GetAxis("Horizontal");

        // 오브젝트의 전방으로 이동
        transform.Translate(new Vector3(horz,0,vert) * amount); // 상대좌표
    }
}
```

Input.GetAxis() 함수는 키보드, 마우스 등의 입력 신호를 -1.0~1.0 사이의 아날로그적인 값으로 구하는 함수

- 입력장치의 속성으로 확인
 - 메뉴 [Edit - Project Settings - Input] 클릭하여 설정 확인



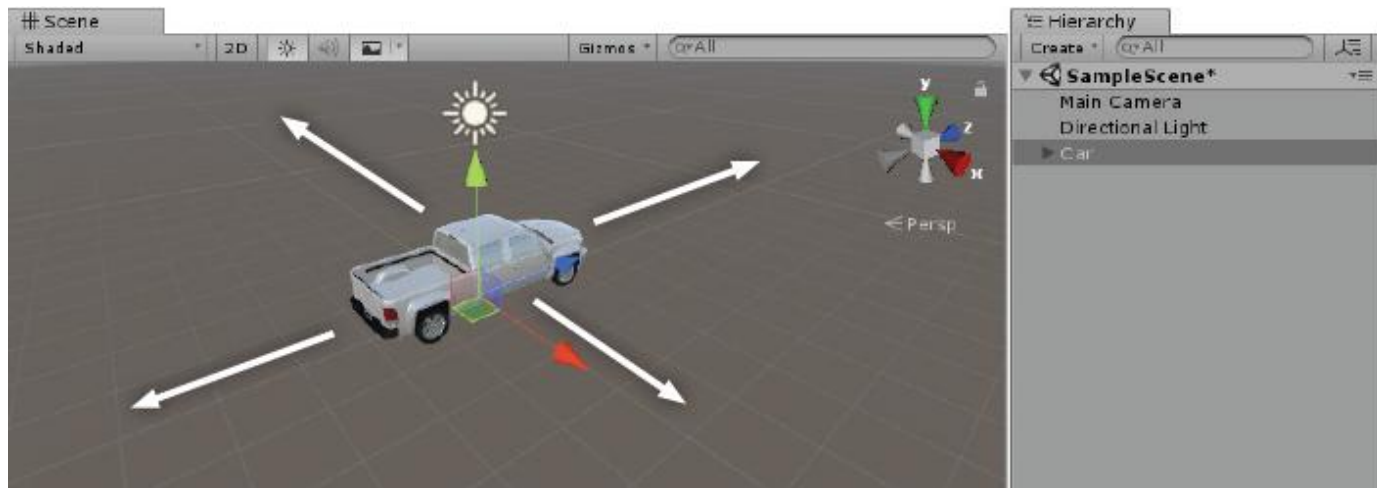
- Input Manager 속성

Name	입력 장치의 이름 (공백을 포함할 수 있음)
Negative Button	음(-)의 방향으로 이동하는 키
Positive Button	양(+)의 방향으로 이동하는 키
Alt Negative Button	음(-)의 방향으로 이동하는 다른 보조키
Alt Positive Button	양(+)의 방향으로 이동하는 다른 보조키
Gravity	키의 복구성 - 값이 클수록 복구 속도가 빠름
Dead	조이스틱과 같은 아날로그 장치의 유격
Sensitivity	키의 민감성 - 값이 클수록 반응 속도가 빠름
Snap	반대 방향의 키를 누를 때 즉시 반응하는지 여부
Type	입력에 사용하는 장치
Axis	입력에 사용할 축



게임을 실행하여 동작 여부를 확인

- 키의 반응은 게임뷰에서 동작



- 스크립트를 수정



```
public class CarMove : MonoBehaviour
{
    float moveSpeed = 10f; // 이동속도
    float rotateSpeed = 60f; // 회전속도(초속 60)

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update()
    {
        // 현재 프레임에서 이동할 거리
        float amount = moveSpeed * Time.deltaTime;
        float amountRotate = rotateSpeed * Time.deltaTime;

        // 전후(Vertical) 좌우(Horizontal) 이동키를 받음
        float vert = Input.GetAxis("Vertical");
        float horz = Input.GetAxis("Horizontal");

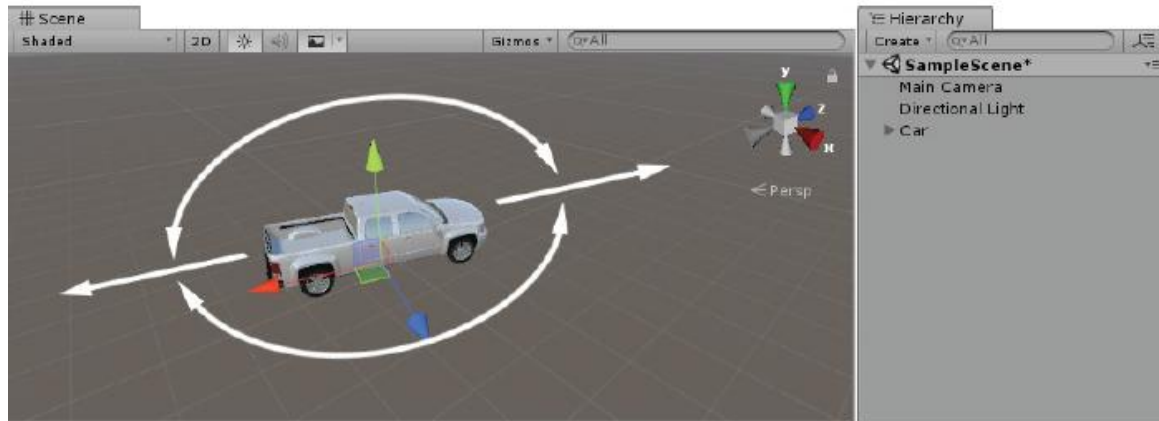
        // 오브젝트의 전방으로 이동 (전진)
        transform.Translate(Vector3.forward * amount * vert);

        // 좌우회전
        transform.Rotate(Vector3.up * amountRotate * horz);
    }
}
```

Transform.Rotate() 함수는
오브젝트를 회전시키도록 하며,
회전 축에 각도를 곱하여 회전각
을 얻는다



스크립트를 수정한 다음, 게임을 실행



이동/회전 속도 조절

- 특정(예, Shift) 키를 누른 상태에서 이동 및 회전을 할 경우, 이동 속도와 회전 속도를 보다 빠르게 수행하도록 코드를 변경해 봅니다



```
public class CarMove : MonoBehaviour
{
    const float runMode = 2.0f;

    ...

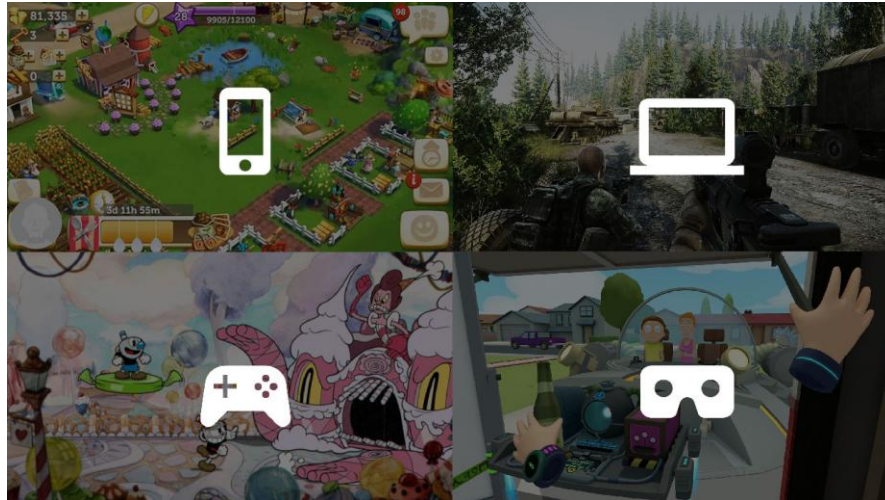
    void Update()
    {
        ...

        if (Input.GetKey(KeyCode.LeftShift)) {
            amount *= runMode;
            amountRotate *= runMode;
        }

        // 오브젝트의 전방으로 이동 (전진)
        transform.Translate(Vector3.forward * amount * vert);
        transform.Rotate(Vector3.up * amountRotate * horz);
    }
}
```

- New Input System

- 2020.1 버전부터 사용 편의성, 플랫폼 간 일관성을 고려하여 새로 제공될 예정
(2019.1 베타 버전에서 테스트 중이며, 이전의 입력시스템에 변화가 생길 듯)
- 변경 이유 : 처음 유니티를 설계할 당시, 다양한 입력 시스템, 플랫폼과 기기 지원을 염두에 두지 않았음 (현재의 Input Manager 지원 중단 계획은 미확정)



새로운 기술로 인해, 최근 미디어 소비 방식에 변화가 생겼으며,
기술 발전과 함께 새로운 기기와 컨트롤, 인터페이스 확장이 요구되고 있음

학습내용

1. 3D 모델과 매터리얼(Material)
2. FPS와 Delta Time
3. 오브젝트 이동 및 회전
4. 슈팅
5. 물리 처리와 사운드 다루기
6. 카메라 트래킹
7. 충돌 판정과 처리
8. 게임 UI 만들기

4.3 SHOOTING

- 자동차에 총을 장착하거나 혹은 탱크의 포신으로부터 총알을 발사

4.3.1 기관총 만들기

- 자동차에 총을 장착(Capsule, Cylinder)하고 속성을 변경

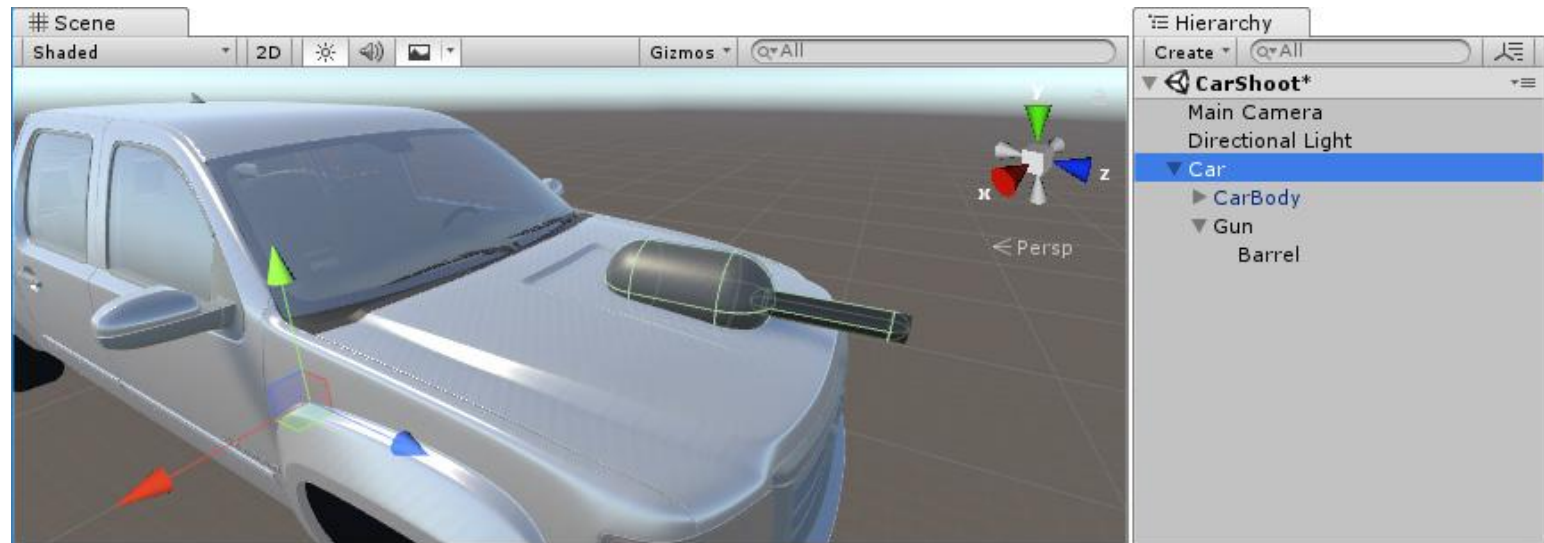
표 4.1 오브젝트의 속성

Object	Name	Rotation	Scale	Position
Capsule	Gun	90, 0, 0	0.2, 0.2, 0.2	0, 0.9, 1.7
Cylinder	Barrel	90, 0, 0	0.06, 0.15, 0.06	0, 0.9, 2

표 4.2 Material의 속성

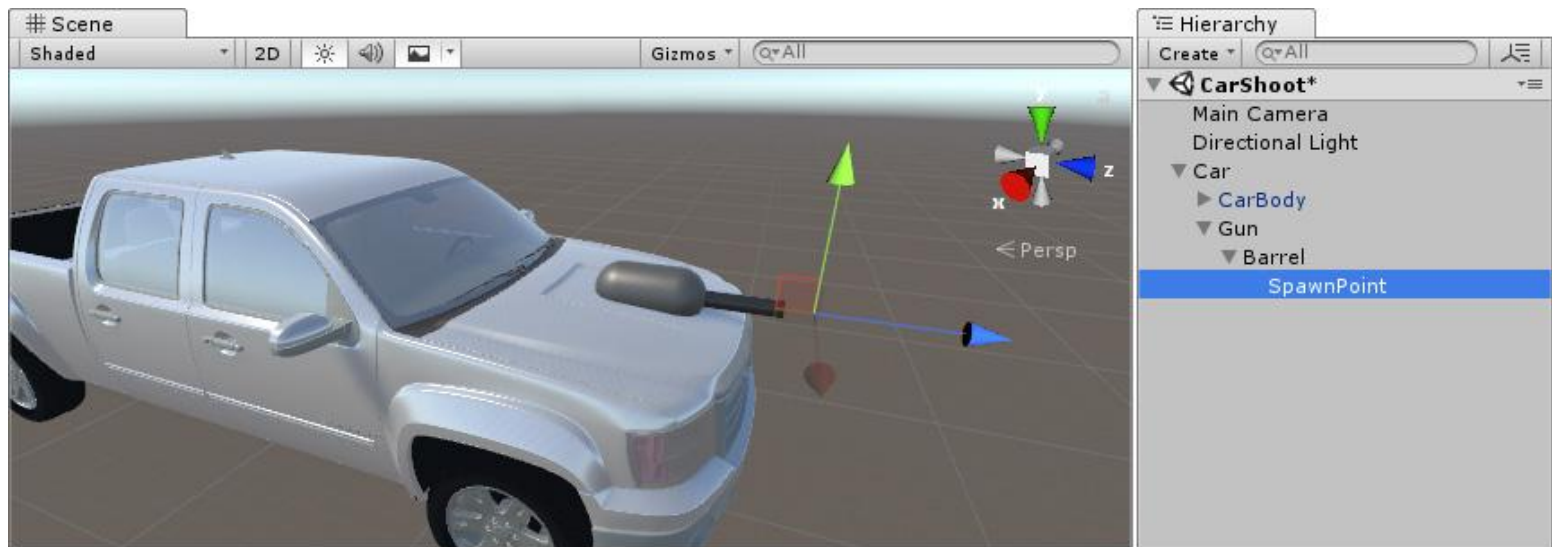
Material	적용 대상	Color	Metallic	Smoothness
LightGray	Gun	0.5, 0.5, 0.5, 1	0.6	0.5
DarkGray	Barrel	0.3, 0.3, 0.3, 1	0.8	0.8

- 하이라리키에서 (생성한) 객체의 계층구조를 조정



4.3.2 스파운포인트(Spawn Point) 만들기

- 오브젝트가 나타나는 위치를 의미
- 예) 머신건의 경우, 총알이 발사되는 총구 (자동차가 수시로 이동하기 때문에, 절대좌표 사용 X)
 - 총구 앞에 빈 오브젝트를 추가하여 스파운포인트로 삼고, 총알을 발사할 때마다 스파운포인트의 위치를 참조하여 총알을 생성하여 발사함
 - 위치는 총구 바로 앞으로 : Position (0, 0.9, 2.24)
 - 계층은 총구의 하위 오브젝트로 설정



4.3.3 프리팹(Prefab) 만들기

- 총알이나 화살과 같이 게임에서 반복적으로 사용하는 오브젝트는 미리 저장해두고, 필요할 때마다 꺼내서 사용하면 매번 오브젝트를 만들지 않아도 됨
- 프리팹 = 미리 저장해 둔 오브젝트

(1) 총알 만들기

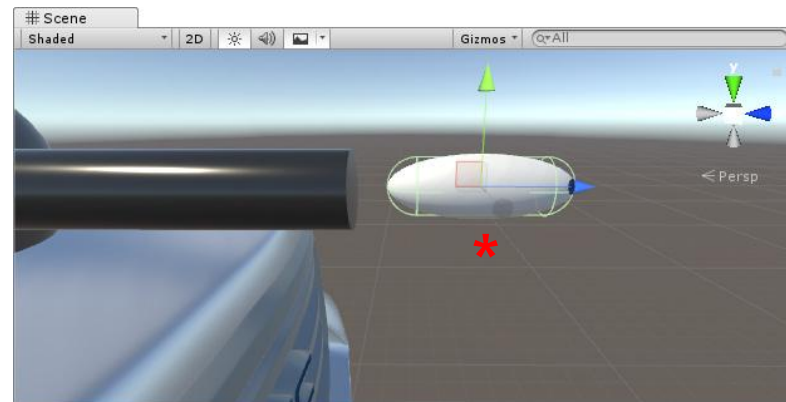
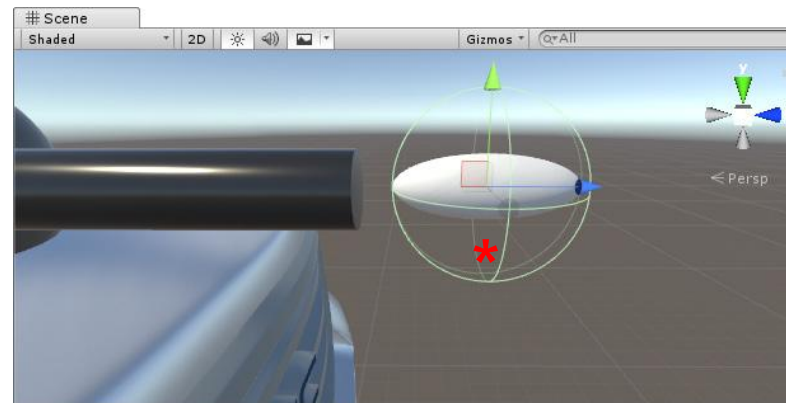
- 프리팹은 오브젝트 원형이 있어야 하므로, 총알과 스크립트 등을 만들고, 나중에 프리팹으로 등록
- 하이라리키에 Sphere를 추가로 생성
 - Scale은 (0.07,0.07,0.2)로, 이름은 Bullet로 변경

프리팹(Prefab)

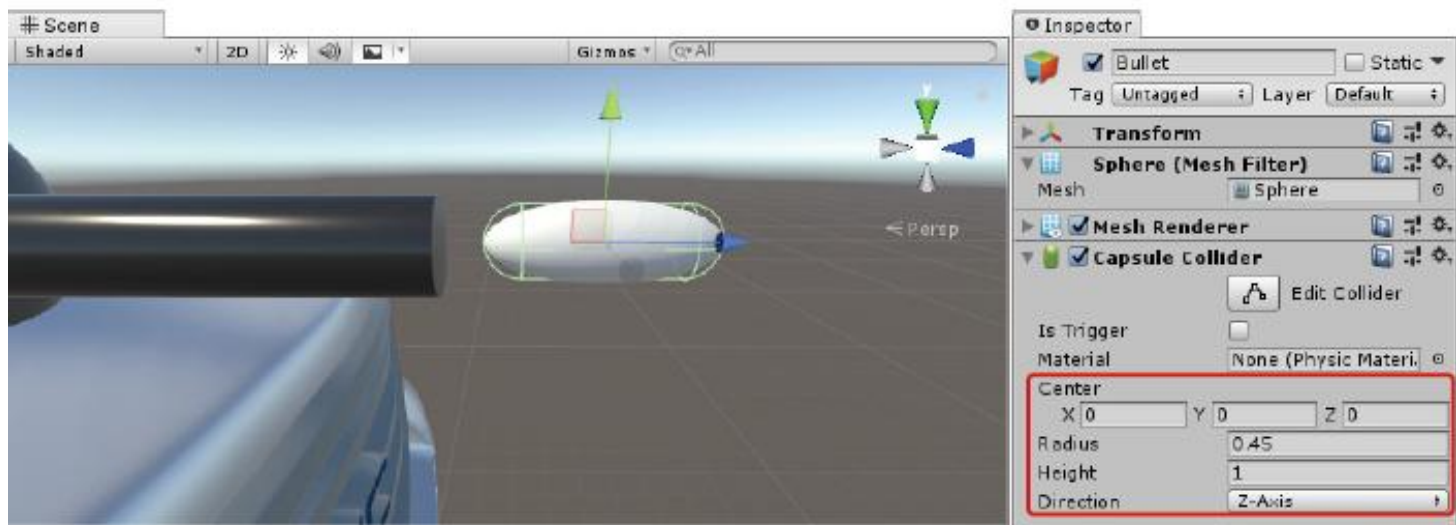
- 프리팹 총알을 썬에 여러 개 만들어서 속성(예, 칼라) 변경의 장점을 살펴봅니다



- Bullet의 Collider 설정
 - Sphere Collider를 제거하고 Capsule Collider를 추가
(총알의 충돌 처리에 보다 적합한 모양으로 수정)
 - * 위치에서의 충돌 처리 ?

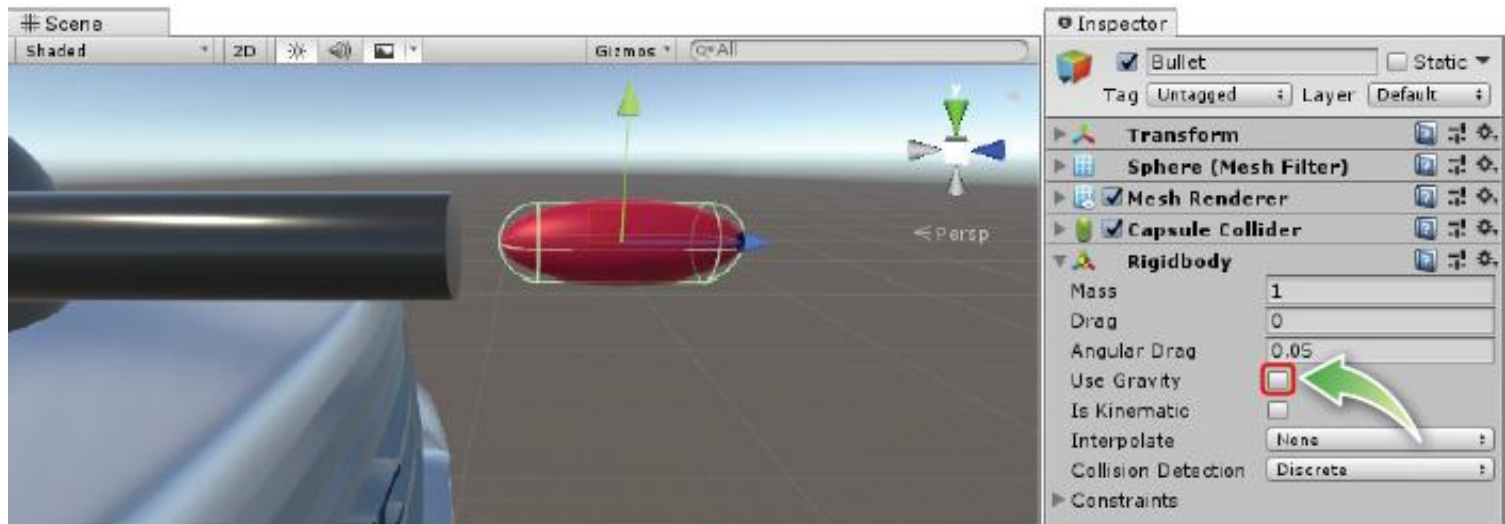


- Bullet의 Collider 설정
 - Sphere Collider를 제거하고 Capsule Collider를 추가
(총알의 충돌 처리에 보다 적합한 모양으로 수정)
 - 총알은 축방향(z축)과 일치하도록 Direction을 정하고 반지름과 높이를 설정



- 총알의 매터리얼
 - 매터리얼을 생성하고 적당한 색을 설정한 후, 총알에 적용

- Bullet에 Rigidbody 추가
 - 충돌 판정을 하려는 오브젝트는 리지드바디가 존재해야 한다
 - 리지드바디를 추가하고, 중력의 영향을 받지 않도록 Use Gravity 옵션을 뺌



(2) Bullet에 Script 추가

– 오브젝트 발사 처리

(2-1) 물리적 충격으로 발사시킴 (예, 총알 또는 화살)

발사되는 오브젝트의 리지드바디에 힘(Force)를 가해서 그 충격으로 오브젝트를 튕겨냄. 발사한 후에는 오브젝트를 제어할 수 없으며, 발사 속도는 발사하는 오브젝트(총)에서 설정함

(2-2) 오브젝트 자체의 동력으로 이동시킴 (예, 미사일)

오브젝트에 스크립트를 추가해서 스스로 이동하게 함. 오브젝트가 이동하는 도중에 속도, 방향 등을 자유롭게 제어할 수 있으며, 발사 속도는 발사되는 오브젝트(총알)에서 설정함

- 게임에서는 (2번) 방법을 주로 사용 : 발사되는 오브젝트를 정교하게 제어할 수 있기 때문

- (1번)은 충격만 주면 나머지는 물리엔진이 알아서 처리하는 경우에 효과적
- 캐릭터가 점프할 때 자주 활용됨. 캐릭터의 y축으로 힘만 가하면, 캐릭터의 상승과 착지는 물리엔진에서 알아서 처리해 줌
- 스크립트를 작성한 후, Bullet로 저장하고, 하이라리키에서 Bullet에 연결
- 오브젝트를 전방으로 이동하는 일반적인 코드

```
public class Bullet : MonoBehaviour
{
    float speed = 30f;

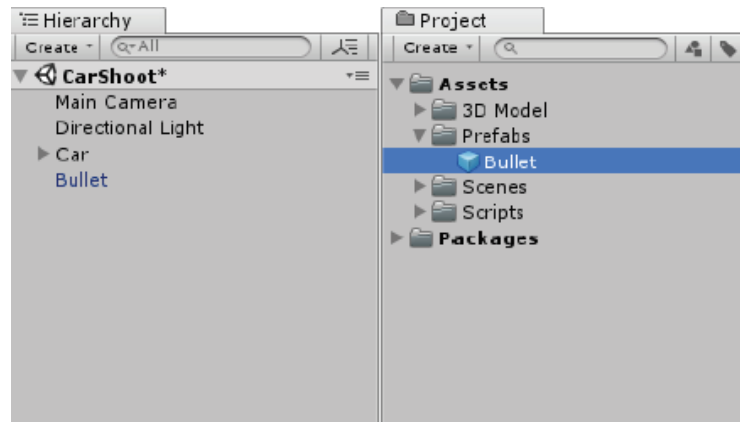
    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update()
    {
        float amount = speed * Time.deltaTime;
        transform.Translate(Vector3.forward * amount);
    }
}
```



(3) 프리팹 만들기

- 하이라키의 오브젝트를 프로젝트 브라우저에 드래그하면 자동으로 프리팹이 만들어짐 (오브젝트 아이콘이 변경됨을 확인)
- 프로젝트 브라우저에 Prefabs 폴더를 만들고, 하이라키의 Bullet를 프로젝트 브라우저의 Prefabs 폴더로 드래그하여 프리팹을 만듦
(프리팹으로 등록된 오브젝트는 하이라키에 짙은 파란색으로 표시됨)



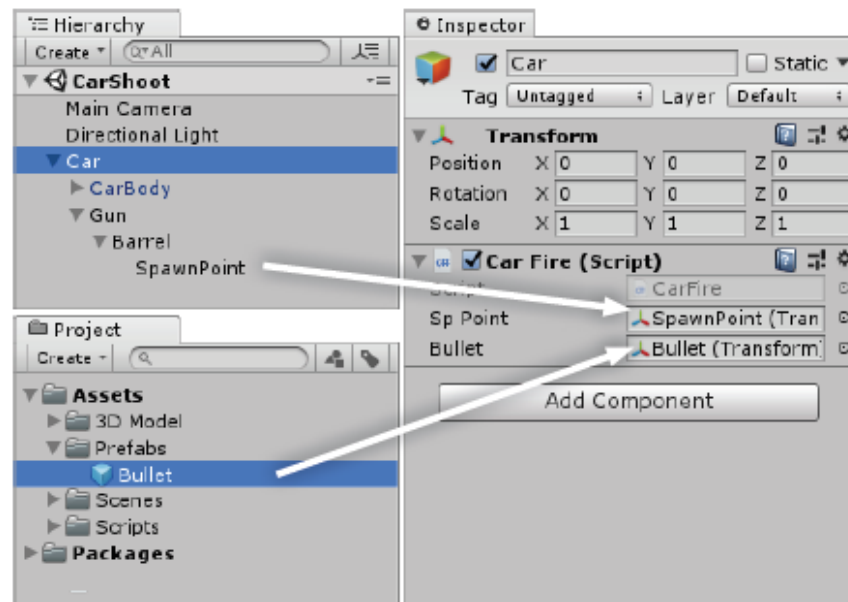
- 프리팹으로 등록되면 원본 오브젝트는 없어도 되므로, (하이라키에서) 총알 오브젝트를 삭제



4.3.4 단발 사격

- 발사버튼을 누를 때마다 한 발씩 총을 쏘는 기능
- Button/Key 상태를 조사하는 함수
 - » GetButtonDown() : 버튼을 누르면 True (1회 호출)
 - » GetButtonUp() : 버튼을 놓으면 True (1회 호출)
 - » GetButton() : 버튼을 누르면 True (연속 호출)
 - » GetKeyDown() : 키를 누르면 True (1회 호출)
 - » GetKeyUp() : 키를 놓으면 True (1회 호출)
 - » GetKey() : 키를 누르면 True (연속 호출)
- ❖ 모두 Input Class에 속하며, 버튼은 "Fire1", "Fire2" .. 와 같이 큰따옴표로 묶어서 표시함

- C# 스크립트를 생성하고, 이름을 CarFire로 저장하고, 스크립트를 작성 (다음 슬라이드 또는 Page.147)
- 작성한 스크립트를 자동차에 연결하고, 기존 스크립트(CarMove)는 연결을 제거
- public 변수에 오브젝트를 설정 (Spawn Point, Bullet Prefab)

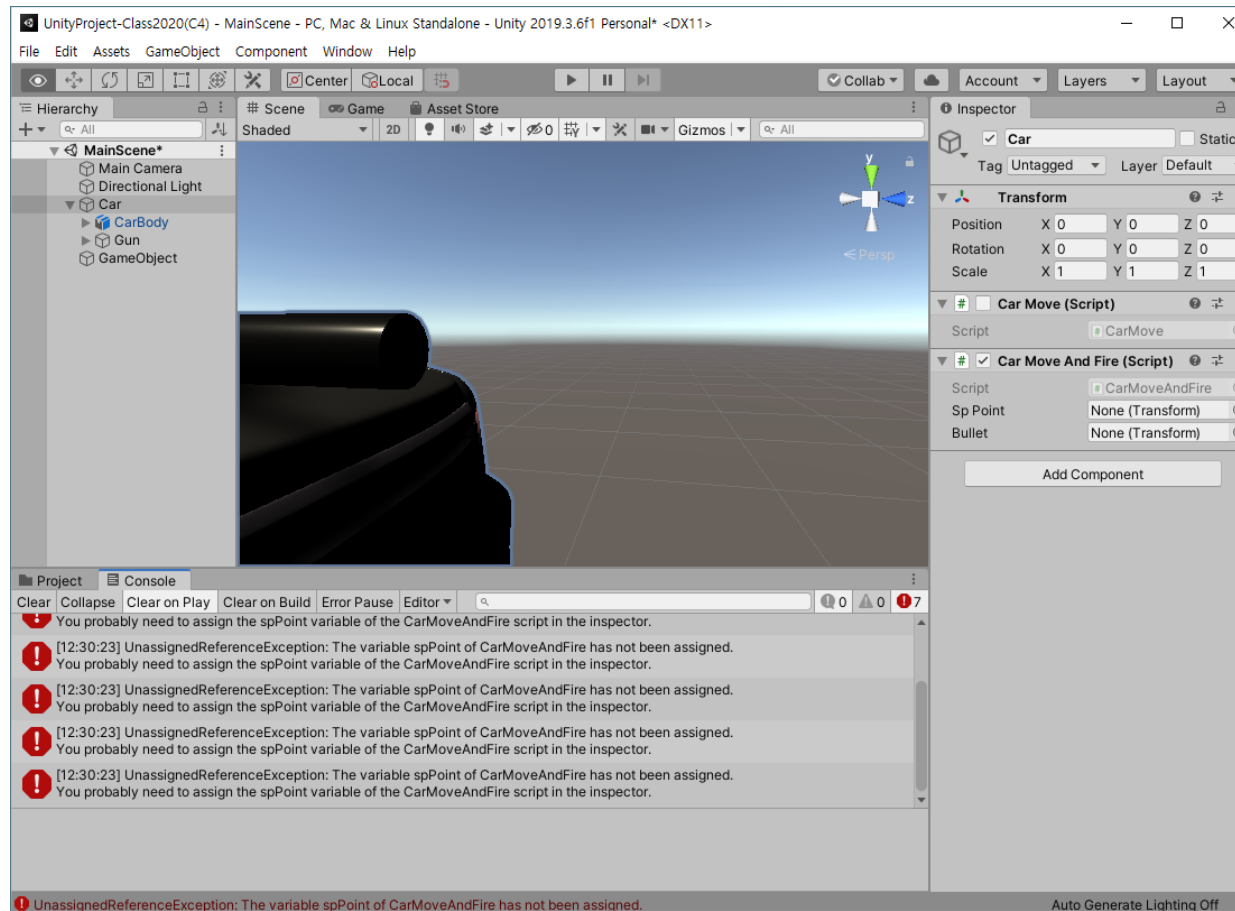


오류 나기 쉬운 예



오류 확인

public으로 선언된 변수에 오브젝트를 할당하지 않고 코드를 실행한 경우



```
public class CarFire : MonoBehaviour
{
    public Transform spPoint; // SpawnPoint
    public Transform bullet; // Bullet 프리팹

    float moveSpeed = 10f; // 이동속도
    float rotateSpeed = 60f; // 회전속도 (초속 60)

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update()
    {
        // 현재 프레임에서 이동할 거리
        float amount = moveSpeed * Time.deltaTime;
        float amountRotate = rotateSpeed * Time.deltaTime;

        // 전후 (Vertical) 좌우 (Horizontal) 이동키를 받음
        float vert = Input.GetAxis("Vertical");
        float horz = Input.GetAxis("Horizontal");

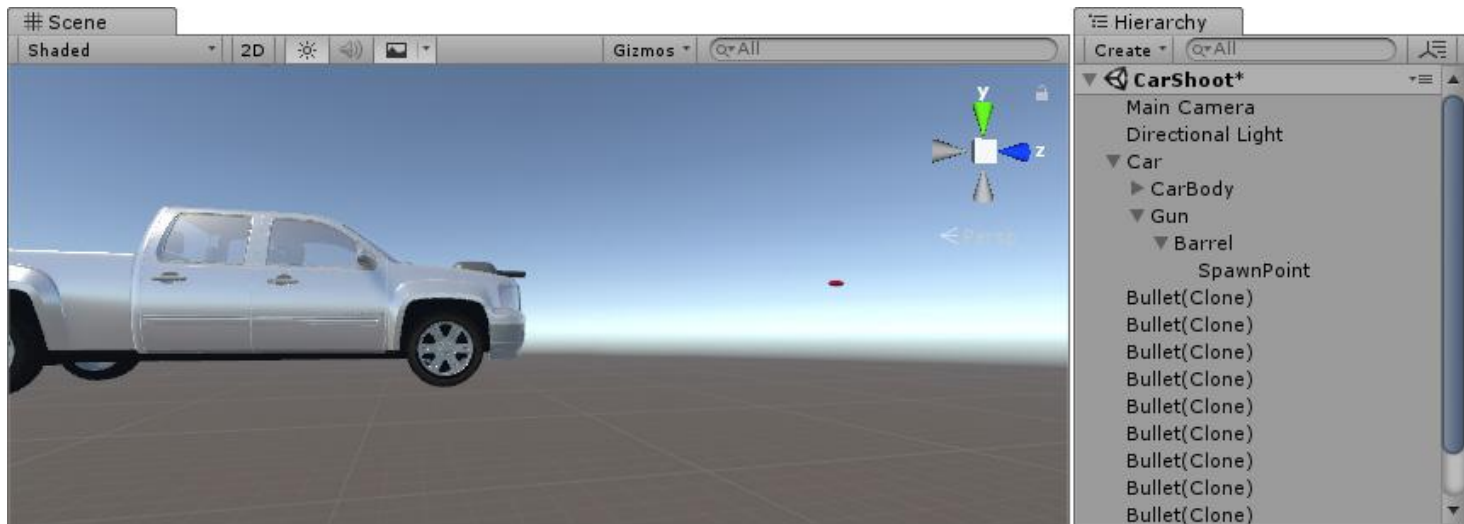
        // 오브젝트의 전방으로 이동 (전진)
        transform.Translate(Vector3.forward * amount * vert);

        // 좌우회전
        transform.Rotate(Vector3.up * amountRotate * horz);

        // 단발사격
        if (Input.GetButtonDown("Fire1")) { SingleShut(); }
    }
    void SingleShut() { Instantiate(bullet, spPoint.position, spPoint.rotation); }
}
```



게임을 실행하여 총알 발사를 확인





스크립트 수정 후, 게임을 재 실행



```
public class CarFire : MonoBehaviour
{
    public Transform spPoint; // SpawnPoint
    public Transform bullet; // Bullet 프리팹

    .....

    // Start is called before the first frame update
    void Start() {
        // 시작할 때 SpawnPoint의 transform 읽기
        spPoint = GameObject.Find("SpawnPoint").transform;
    }

    // Update is called once per frame
    void Update()
    {
        .....

        // 단발사격
        if (Input.GetButtonDown("Fire1")) { SingleShut();}
    }

    void SingleShut() { Instantiate(bullet, spPoint.position, spPoint.rotation); }
}
```

GameObject.Find()는 하이라리 키 뷰에서 오브젝트를 검색한다. 리턴 타입은 GameObject. 동일한 이름의 오브젝트가 여러 개 있는 경우, 첫번째로 발견한 오브젝트를 반환

Instantiate()는 프리팹으로 선언한 오브젝트를 복제하는 함수

참조, INstantiate()

– Object.Instantiate()

- 사용법

- Instantiate(<프리팍>, <위치>, <방향>);

지정한 위치에 프리팍을 만든다

- Transform obj = Instantiate(<프리팍>, <위치>, <방향>) as Transform;

지정한 위치에 프리팍을 만들고 변수 obj에 할당한다

```
public class CarFire : MonoBehaviour
{
    .....
    void Update() {
        .....

        // 사격
        if (Input.GetButtonDown("Fire1")) { FireBullet();}
    }

    void FireBullet() {
        // 프리팍 만들기
        Transform obj = Instantiate(bullet, spPoint.position, spPoint.rotation) as Transform;

        // 발사
        obj.rigidbody.AddForce(spPoint.forward * power);
    }
}
```



- 발사된 총알 제거
 - 총알이 발사될 때마다 하이러키에 Bullet(Clone)과 같은 오브젝트가 계속적으로 추가됨
 - 추가된 총알은 화면을 벗어나더라도 사라지지 않고 계속 쌓이게 되며, 적당한 시점과 일정한 조건에서 제거할 필요가 있음
 - 총알 오브젝트를 제거
 - ① 화면을 벗어나면, 제거
 - ② 다른 오브젝트와 충돌하면, 제거
 - ③ 발사 후 일정한 시간이 지나면, 제거

- 일정한 시간이 지나면 제거하도록 스크립트를 수정



```
public class Bullet : MonoBehaviour
{
    // Start is called before the first frame update
    void Start() {
        //
        Destroy(gameObject, 2);
    }
}
```



스크립트를 저장하고 게임을 다시 실행

발사된 총알이 2초 후에 자동 제거되는 것을 확인

참조, **DESTROY()**

- Destroy()
 - 사용법

• 4.3.5 연발 사격

- 버튼이나 키를 누르면 연속해서 발사하는 기능
 - 연속 발사 사격은 버튼이나 키를 누르고 있으면 계속적으로 이벤트가 발생하는 함수를 이용하여 기능 구현
 - Button/Key 상태 조사 함수 (151페이지 참조)
 - » 모두 Input 클래스에 포함되어 있으며, 버튼 정의는 메뉴 [Edit - Project Settings - Input]에서 확인 및 변경 가능
 - » Key는 "a", "b", "return", "space", "right ctrl" 등과 같이 큰따옴표로 묶어서 문자열로 표기하거나 KeyCode 상수를 이용
- 키들은 각각 KeyCode.A, KeyCode.B, KeyCode.Return, KeyCode.Space, KeyCode.RightControl 등으로 정의되어 있음

– 연발 사격 함수 추가

- » GetButton()나 GetKey()와 같이 누르고 있으면 계속적인 이벤트가 발생하는 함수를 이용하여 스크립트를 작성

```
public class CarFire : MonoBehaviour
{
    .....

    // Update is called once per frame
    void Update()
    {
        .....

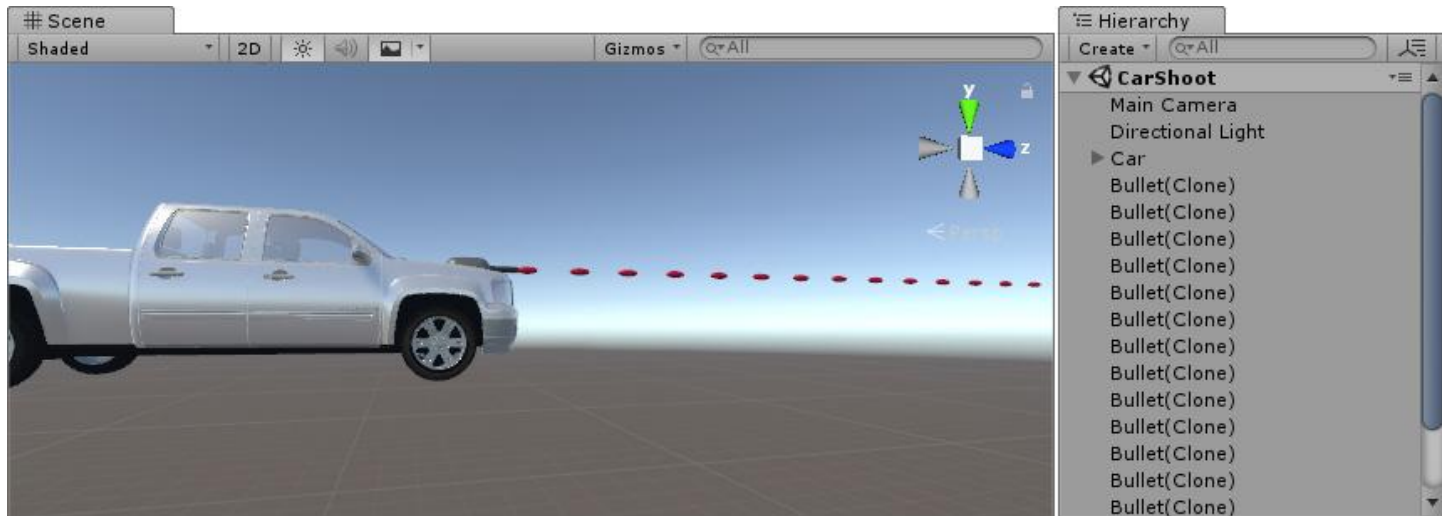
        // 연발 사격 (Left Alt or Right Button)
        if (Input.GetButton("Fire2")) { AutoFire(); }
    }

    void SingleShut() { Instantiate(bullet, spPoint.position, spPoint.rotation); }
    void AutoFire() { Instantiate(bullet, spPoint.position, spPoint.rotation); }
}
```





게임을 실행하여, 왼쪽 Alt 키를 누르면 총알이 연속해서 발사되는 것을 확인



– 발사 간격 설정

- 앞의 스크립트는 매 프레임마다 발사하므로, 발사 간격을 지연시킬 필요가 있음
- 발사 시간을 계산하는 변수를 추가하고 연발 사격 함수를 수정

```
public class CarFire : MonoBehaviour
{
    .....
    float delayTime = 0.1f;    // 발사지연시간

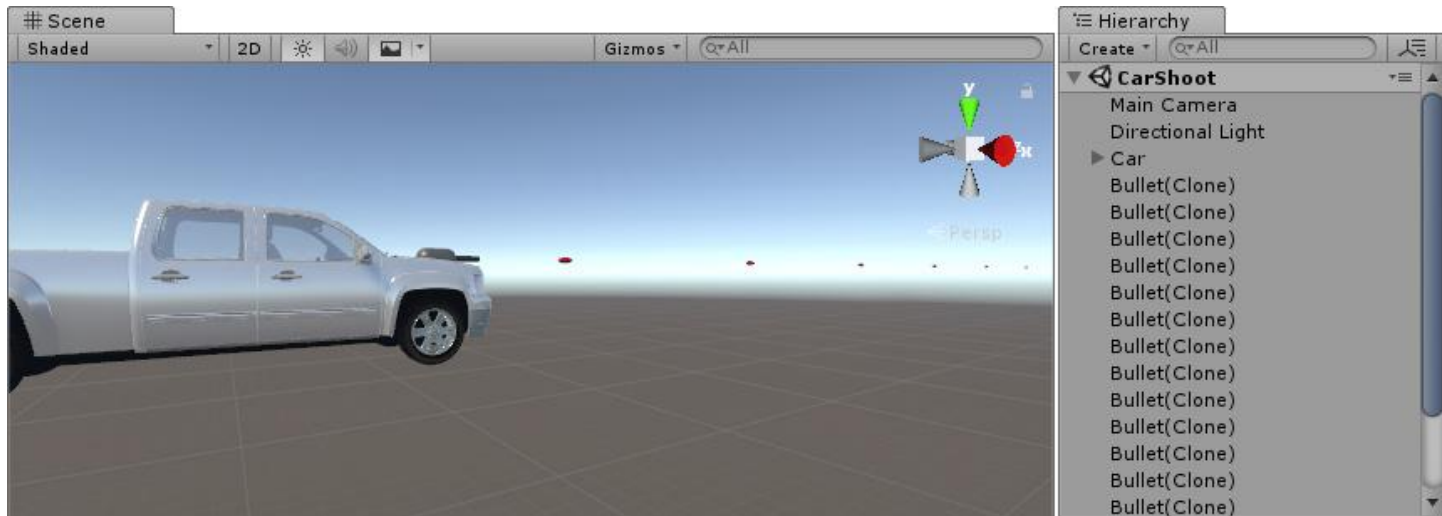
    // Update is called once per frame
    void Update()
    {
        .....

        // 연발 사격 (Left Alt or Right Button)
        if (Input.GetButton("Fire2")) { AutoFire(); }
    }
    void AutoFire() {
        // 발사 지연시간 계산
        delayTime += Time.deltaTime;
        if (delayTime >= 0.1f) {
            delayTime = 0;
            Instantiate(bullet, spPoint.position, spPoint.rotation);
        }
    }
}
```





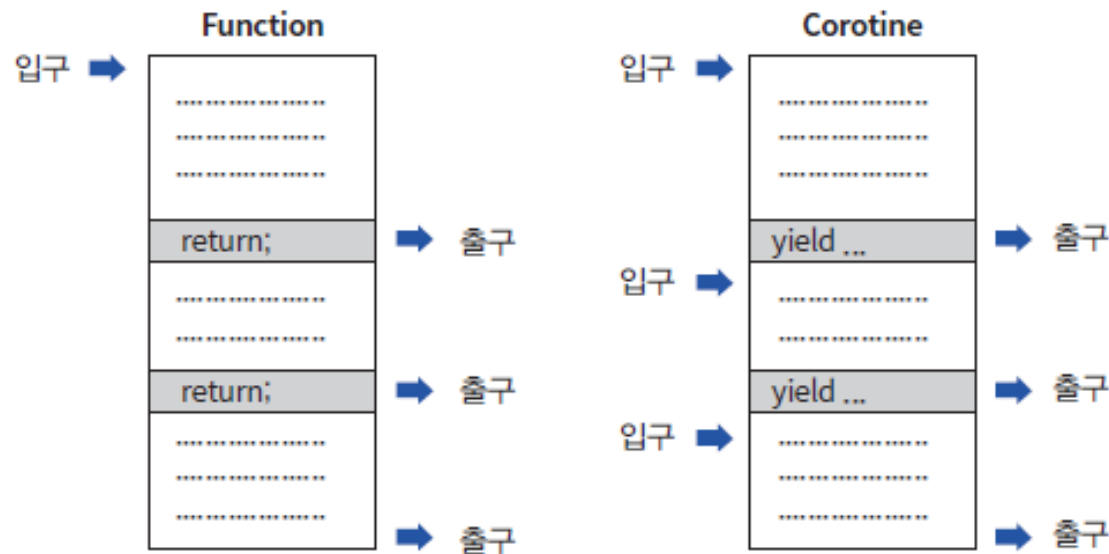
게임을 다시 실행하여, 총알이 지연된 시간 간격으로 발사되는 것을 확인



COROUTINE 활용

– Coroutine 활용

- 코루틴(Coroutine)은 함수(Function or Method)의 특수한 형태
- 함수가 Single Input – Multiple Output 구조를 갖는 반면, 코루틴은 Multiple Input – Multiple Output 구조를 가짐



- 코루틴은 yield문을 만나면 일단 리턴한 다음, yield문에서 지시한 지연시간 후에 다시 (루틴에) 진입하여 yield 다음 부분을 실행한다

일종의 함수 지연 수행 또는 분할 수행과 유사한 패턴

- yield문 사용 형식

- `yield break;` // 코루틴 완전 종료 (이후 문장은 수행하지 않음)
- `yield return 0;` // 현재 프레임이 완료된 후
- `yield return null;` // 현재 프레임이 완료된 후
- `yield return new WaitForFixedUpdates;` // 물리 처리가 끝난 후
- `yield return new WaitForSeconds(지연시간);` // 지정시간이 경과한 후
- `yield return new WWW(url);` // 링크 접속이 완료된 후
- `yield return StartCoroutine(Coroutine);` // 다른 코루틴이 완료된 후

- 코루틴 호출 형식

- `StartCoroutine("AutoFire2");` // 코루틴 이름을 문자열로 지정
 - `StartCoroutine("AutoFire2", parameters);` // 코루틴에 매개변수 전달
 - `StartCoroutine(AutoFire2());` // 코루틴을 함수 형식으로 호출
 - `StartCoroutine(AutoFire2(parameters));` // 코루틴에 매개변수 전달

- 문자열 형식의 코루틴 호출은 함수명을 잘못 기재하는 경우, 컴파일 타임에서는 오류를 걸러내지 못함
- 매개변수를 하나만 전달할 수 있음

– 연발 사격 함수를 코루틴으로 작성하면

```
public class CarFire : MonoBehaviour
{
    .....
    bool canFire = true; // 총을 발사할 수 있는가?

    void Update() {
        ....
        // 연발사격
        if(Input.GetButton("Fire2")) {
            // AutoFire();
        }
        // 연발사격 코루틴
        if(Input.GetButton("Fire2") && canFire) {
            StartCoroutine(AutoFire2());
        }
    }

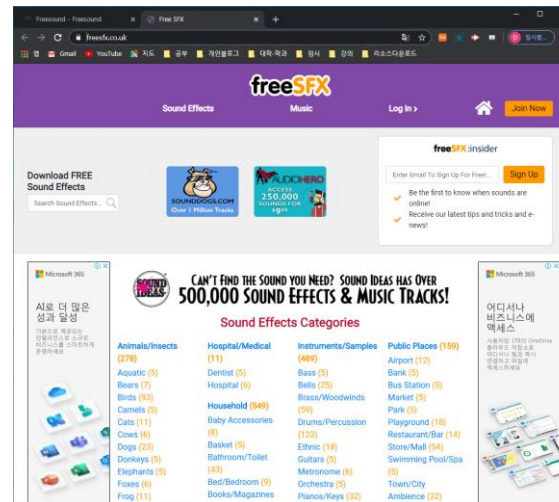
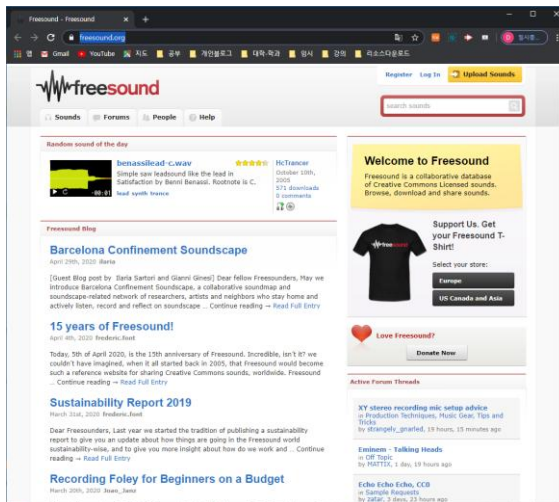
    // 연발사격 코루틴
    IEnumerator AutoFire2() {
        Instantiate(bullet, spPoint.position, spPoint.rotation);
        canFire = false;

        yield return new WaitForSeconds(0.1f);
        canFire = true;
    }
}
```



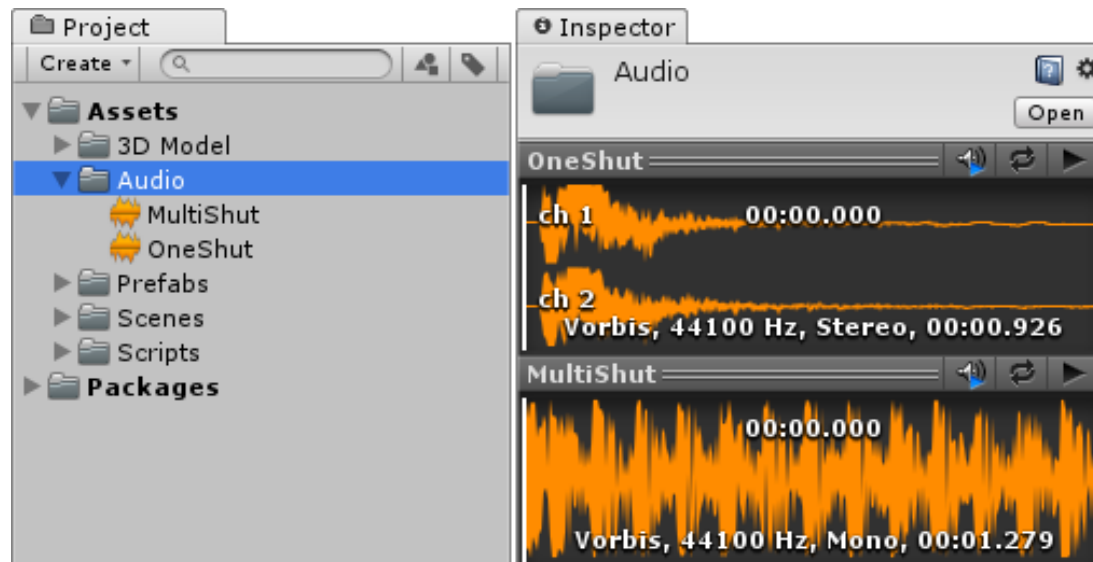
4.4 사운드(SOUND) 다루기

- 게임에 사용할 사운드 파일을 다운로드
- 사운드의 일부분만 추출하거나, 피치나 볼륨 등을 변경해서 게임에 최적화시켜야 하는 경우가 많으며, 이럴 경우 별도의 사운드 편집 툴 사용을 권장
 - <https://freesound.org/>
 - <https://www.freesfx.co.uk/>
 - <http://soundbible.com/> 등



4.4.1 단발 사격의 사운드

- 인터넷에서 “gun fire” 등으로 검색하여 적당한 파일을 다운로드 받음
- 프로젝트 브라우저에 드래그하여 추가
- 효과음 사운드는 즉시 재생되어야 하므로, 비압축 형식인 wav 파일이 적당함



- 단발 사격용 오디오 클립을 하이라키의 Car로 드래그하여 AudioSource를 추가
- Play On Awake 속성을 해제
- 총을 발사할 때마다, 발사음이 재생되도록 스크립트를 수정

```
public class CarFire : MonoBehaviour
{
    .....
    AudioSource gunSound; // AudioSource 변수선언

    void Start() {
        ....
        gunSound = GetComponent(); // 컴포넌트 읽기
    }
    void SingleShut() {
        gunSound.Play(); // AudioClip 재생
    }
}
```

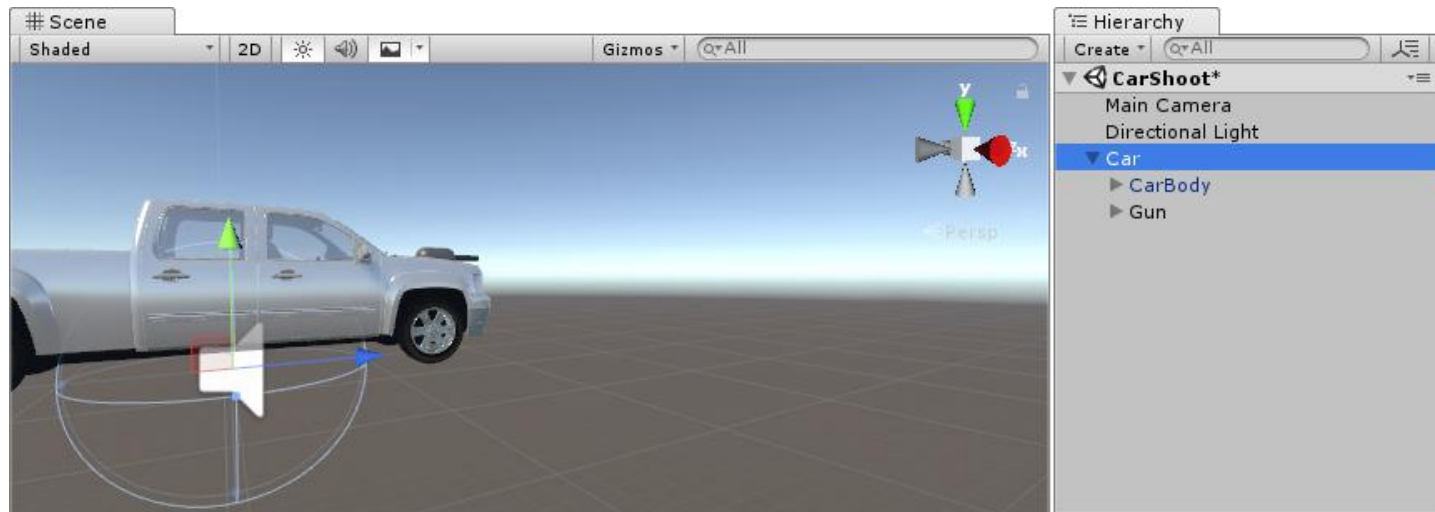


효과음 처리 (페이지 75~78 참조)

- (1) AudioSource 처리를 위한 변수 선언
- (2) AudioSource 컴포넌트 읽어오기
- (3) AudioClip 재생

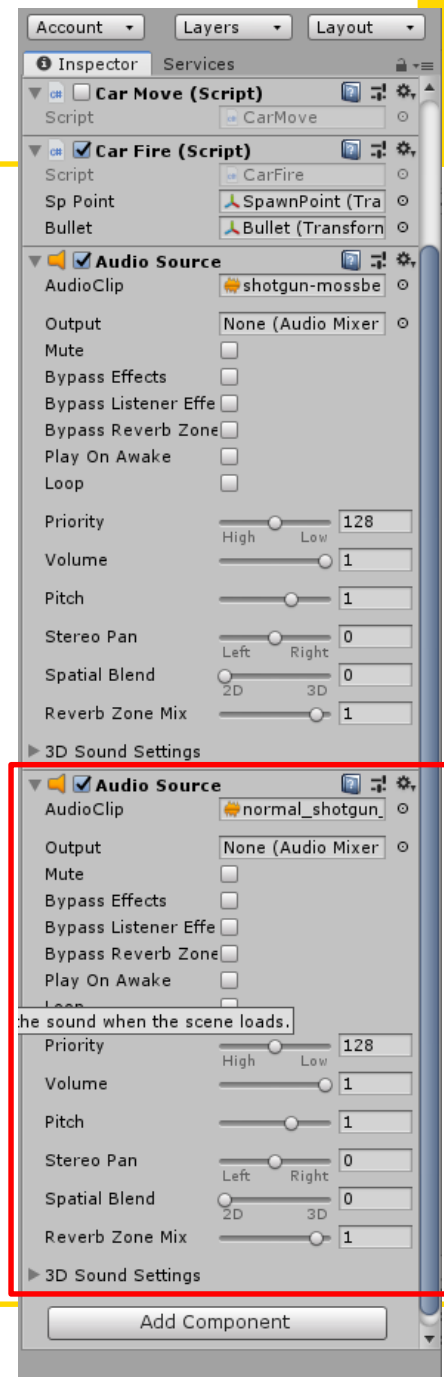
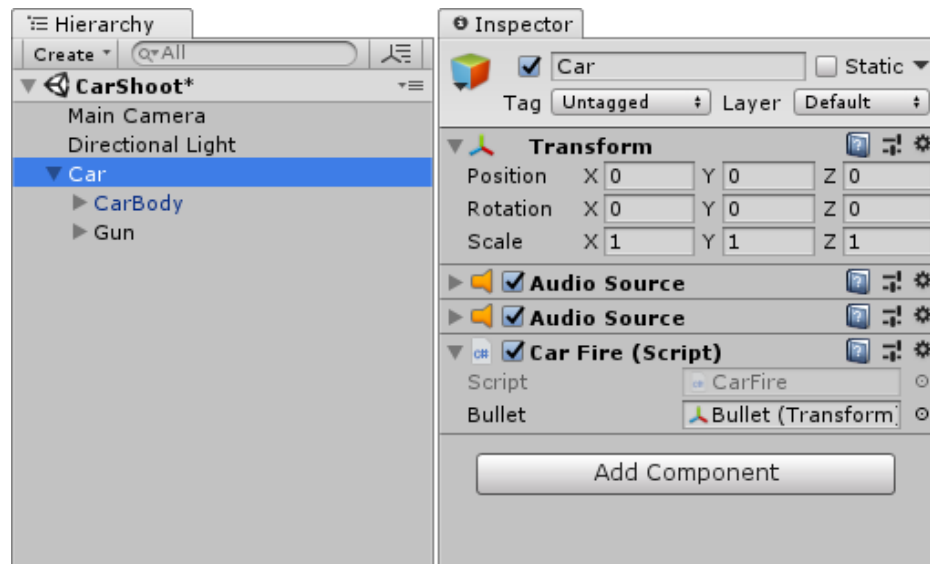


게임을 실행하여, 총알을 (단발로) 발사할 때 오디오 클립 재생 여부를 확인

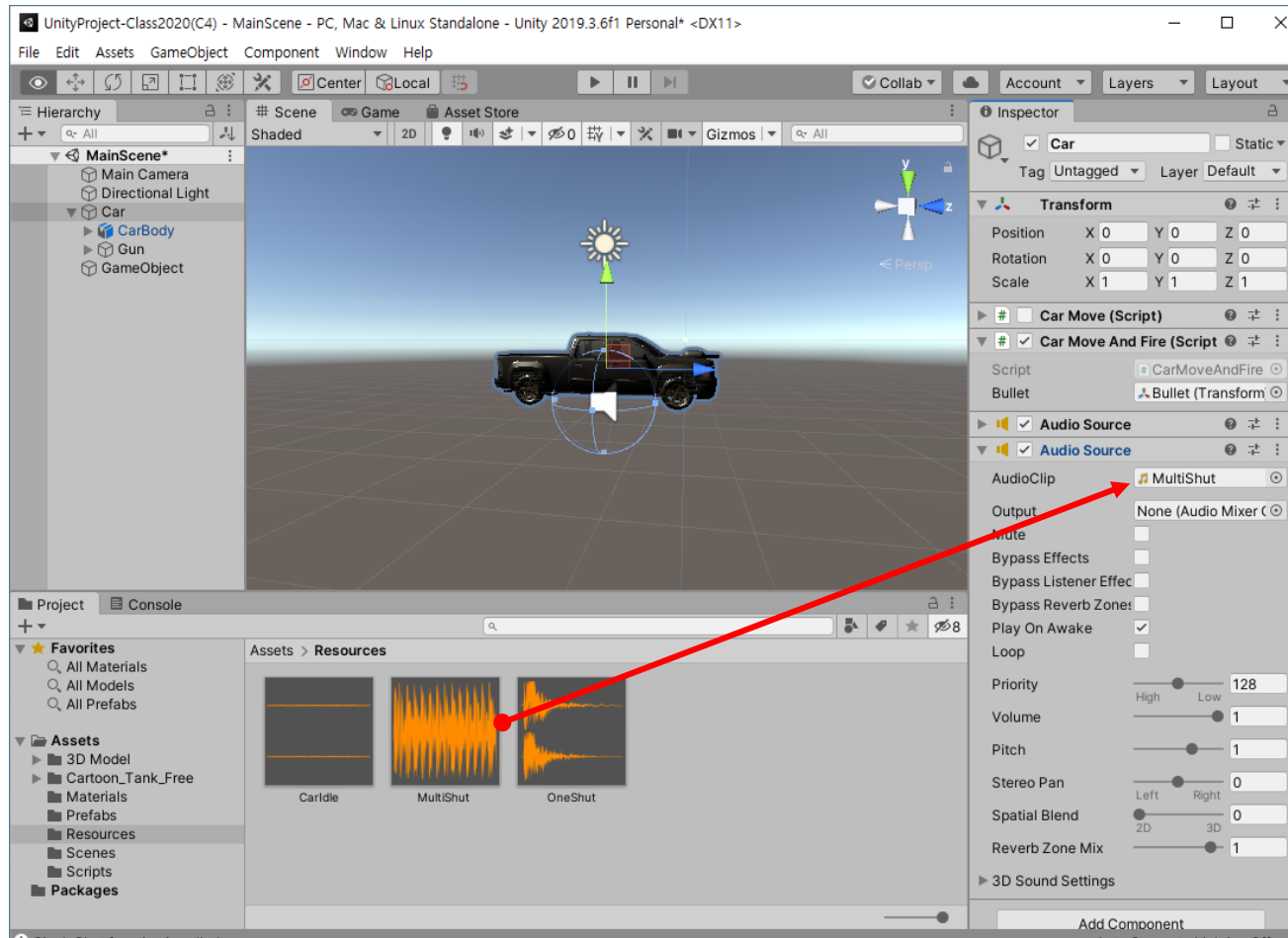


4.4.2 연발 사격의 사운드

- 하나의 AudioSource로 여러 종류의 오디오 클립을 재생하려면, 각각의 오디오 클립을 변수 또는 배열로 저장하고 필요할 때마다 오디오 클립을 설정
- [Add Component - Audio - Audio Source]를 클릭하여 Car에 AudioSource를 하나 더 추가



- 연발 사격용 오디오 파일을 인스펙터에서 AudioClip으로 드래그하여 연결



- 연발 사격 시에 오디오 재생을 위해 스크립트를 수정

```
public class CarFire : MonoBehaviour
{
    .....
    AudioSource[] gunSound; // AudioSource 변수선언 (배열로)

    void Start() {
        ....
        gunSound = GetComponent<AudioSource>(); // 컴포넌트 읽기
    }
    void SingleShut() { gunSound[0].Play(); }
    void AutoFire() {
        ....
        gunSound[1].Play();
    }
}
```

- 앞의 스크립트에서, 연발 사격을 중지하더라도 사운드는 오디오 클립의 끝까지 재생되어 불필요한 발사음이 지속되는 문제가 발생
- 키나 버튼을 놓으면, 사운드를 중지하도록 스크립트를 수정

```
public class CarFire : MonoBehaviour
{
    .....

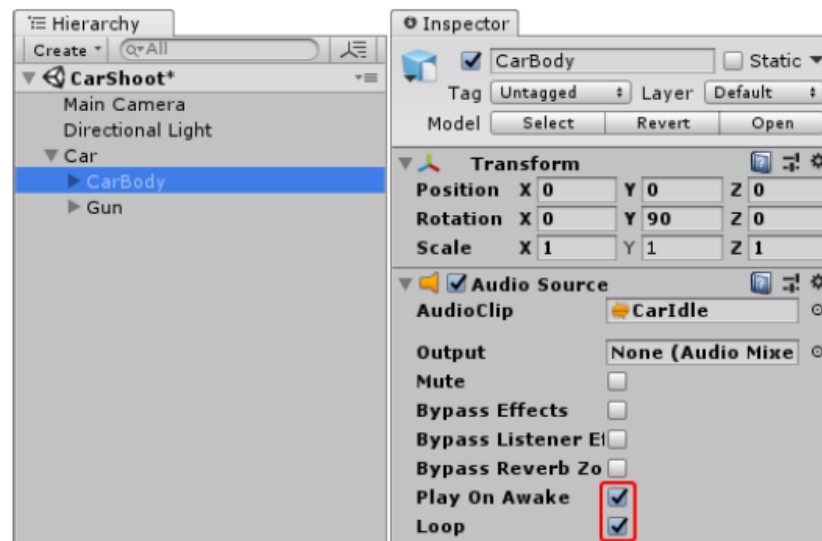
    void Update() {
        ....
        // 버튼을 놓으면 사운지 정지
        if (!Input.GetButton("Fire2")) { gunSound[1].Stop(); }
    }
}
```



게임을 실행하여 키를 놓았을 때의 사운드 재생 여부를 확인

4.4.3 자동차 엔진 사운드 처리

- 인터넷에서 “car idle sound” 등을 적당한 파일을 검색하여 다운로드 (필요에 따라 오디오 파일의 편집을 권장)
- (적용 예) 자동차의 공회전 엔진 소리를 재생하고, 자동차가 달리기 시작하면서 속도가 빨라지면 엔진소리를 높인다
- 현재 Car에는 2개의 AudioSource가 연결되어 있으므로, 실습에서는 CarBody로 연결하고 처리하는 스크립트를 작성하여 연결 (자동차 속도에 맞춰 사운드를 변경함)





```
public class CarEngine : MonoBehaviour
{
    AudioSource soundEngine;

    // Start is called before the first frame update
    void Start()
    {
        soundEngine = GetComponent();
    }

    // Update is called once per frame
    void Update()
    {
        // 이동 및 회전 키의 입력상태를 조사
        float vert = Mathf.Abs(Input.GetAxis("Vertical"));
        float horz = Mathf.Abs(Input.GetAxis("Horizontal"));

        // 큰값을 pitch로 설정
        float pitch = Mathf.Max(vert, horz);

        // 피치는 -3.0~3.0, 볼륨은 0~1.0 으로 지정
        soundEngine.pitch = pitch + 1; // 1.0~2.0
        soundEngine.volume = soundEngine.pitch * 0.6f; // 0.6~1.2
    }
}
```



게임을 실행하여 키를 눌렀을 때의 사운드 재생 여부를 확인



GAME 기초

Part 2.

학습내용

1. 3D 모델과 매터리얼(Material)
2. FPS와 Delta Time
3. 오브젝트 이동 및 회전
4. 슈팅
5. 물리 처리와 사운드 다루기
6. 카메라 트래킹
7. 충돌 판정과 처리
8. 게임 UI 만들기

4.5 오브젝트의 물리 처리

- 현재까지 작업에서, 자동차는 물리 엔진이 적용되어 있지 않으므로, 무중력 상태에 있는 것과 동일
- 기초적인 물리 처리를 학습

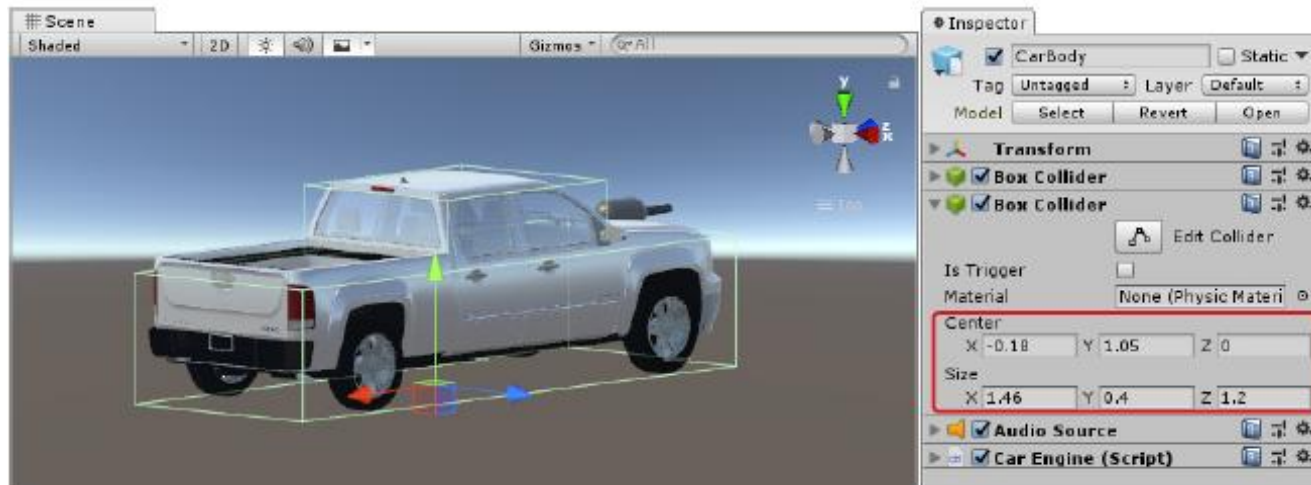
4.5.1 콜라이더(Collider)와 리지드바디(Rigidbody)

- 물리처리에서 매우 중요한 요소로, 충돌 판정에 반드시 필요

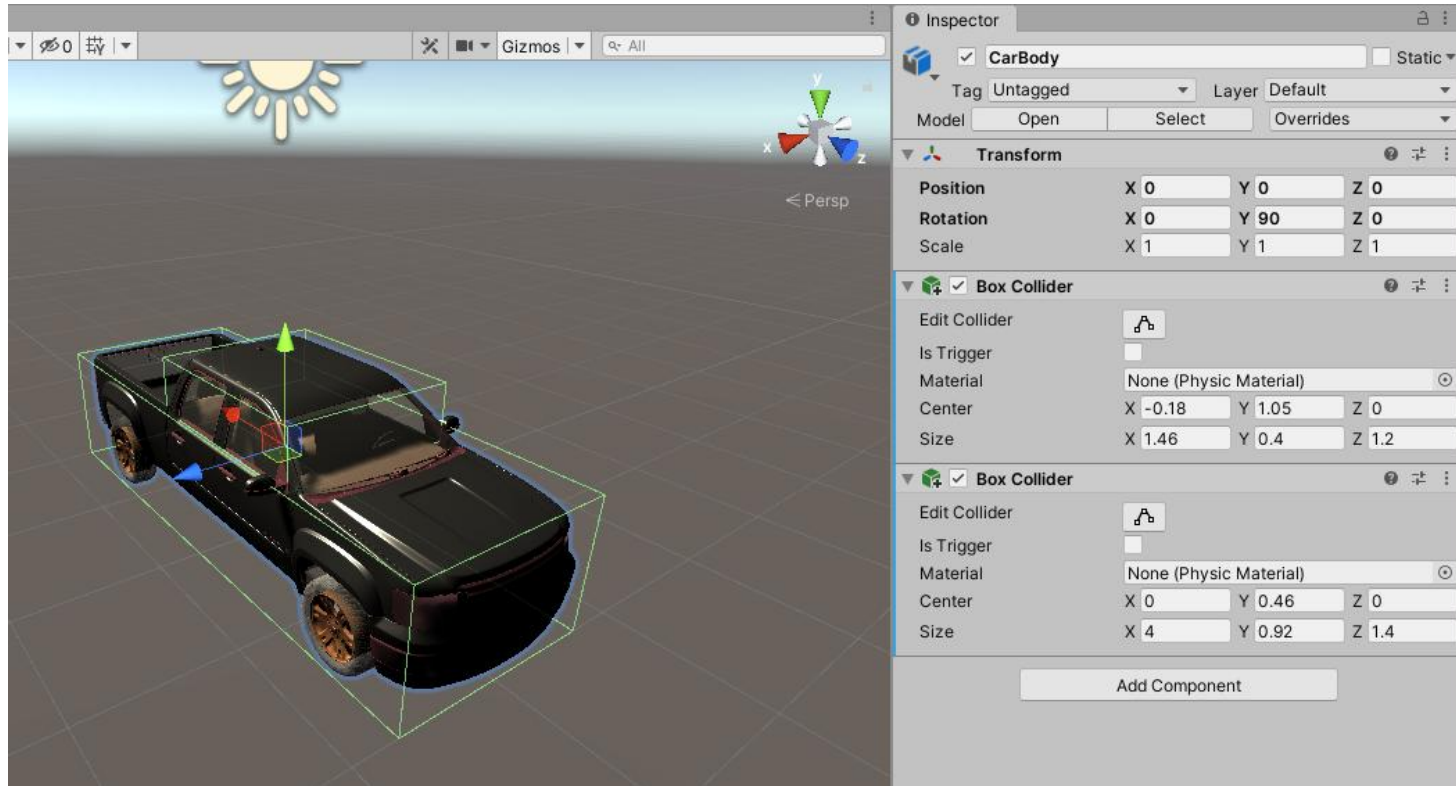
(1) CarBody에 BoxCollider 추가

- 콜라이더는 오브젝트의 충돌 감지를 위해 설정한 영역
- Cube, Sphere 등 유니티에서 제공하는 오브젝트는 기본적으로 설정되어 있지만, 외부에서 가져온 모델은 콜라이더가 없음
- 콜라이더는 모델의 외형과 똑같이 만드는 것이 이상적이지만, 콜라이더의 모든 모서리에 대해 충돌 판정이 필요하므로 형태가 복잡할수록 성능은 떨어짐
- 콜라이더의 Vectex는 255까지 허용하지만, 러프한 콜라이더를 사용해도 충돌은 순식간에 이뤄지므로 플레이어는 의식하지 못할 정도임
- 하위 오브젝트의 충돌을 감지하므로, 콜라이더를 부모에게 설정할 필요는 없음

- CarBody를 선택하고 [Component – Physics – Box Collider]를 클릭하여 콜라이더를 설치
- 콜라이더가 자동차의 아래 부분을 완전히 감싸도록 Size와 Center를 조절
- Box Collider를 하나 더 추가하여 자동차 위 부분을 감싸도록 설정

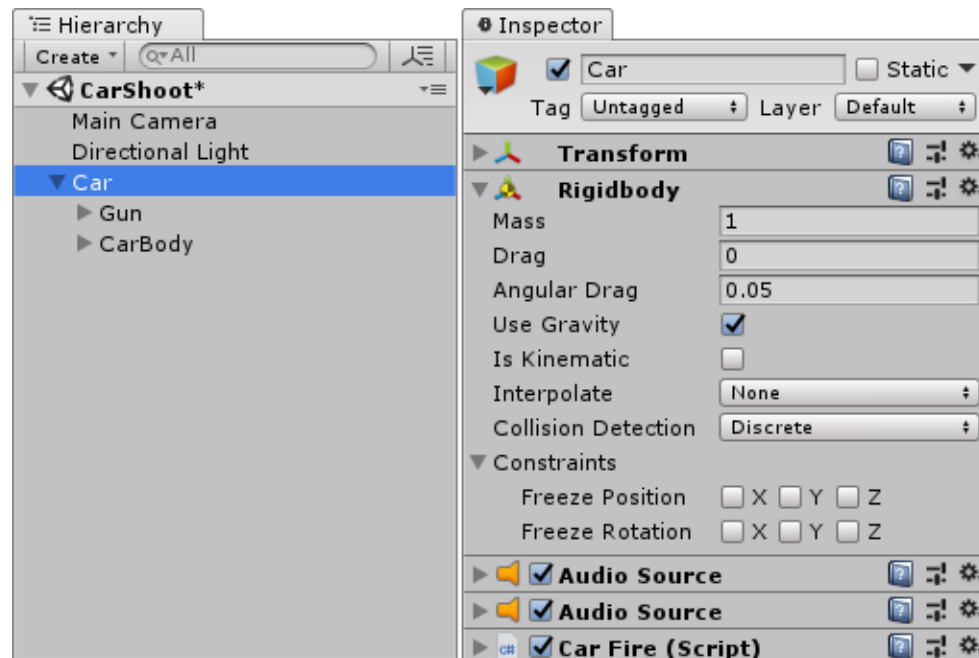


- Center 와 Size 참조



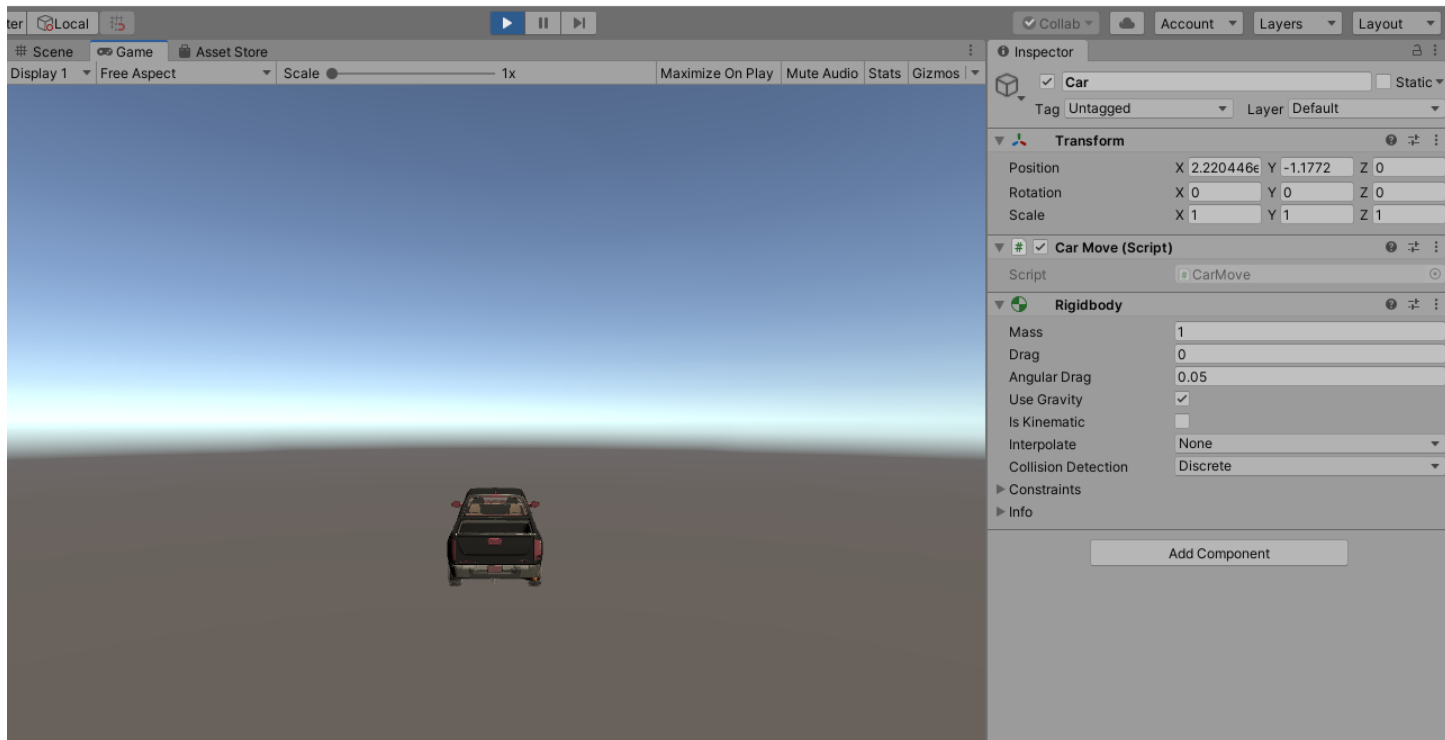
(2) Car에 Rigidbody 추가

- Car를 선택하고 [Component – Physics – Rigidbody]를 클릭하여 리지드바디를 추가
- 충돌이벤트는 리지드바디가 있는 오브젝트에 발생하므로 Car에 추가





리지드바디를 부여한 다음, 게임을 실행하여 확인

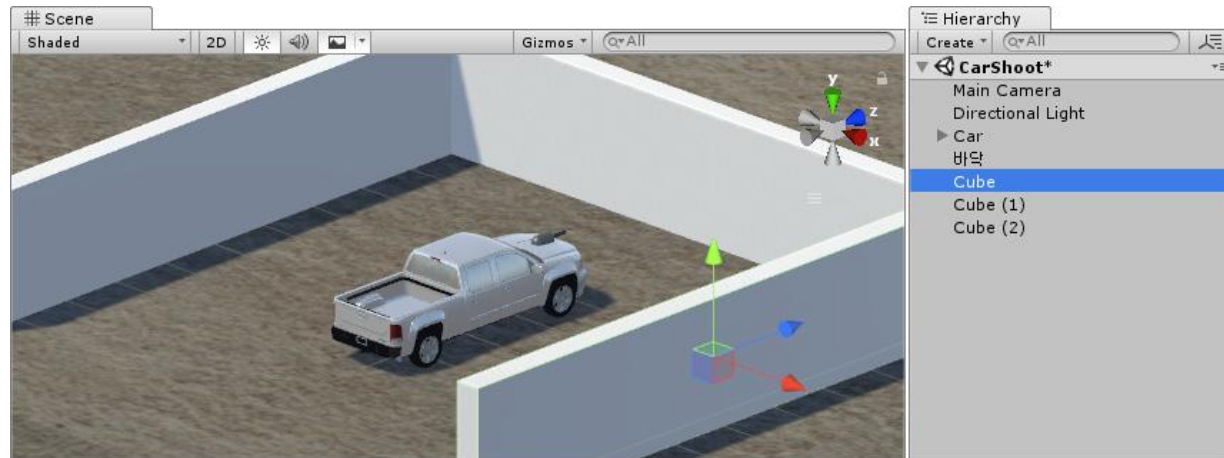


4.5.2 장애물 설치

- 자동차의 물리엔진을 테스트하기 위해 바닥과 장애물을 설치함
- [GameObject - 3D Object - Plane]을 클릭하여 Plane을 추가하고 속성을 조정
Position (0,0,0), Scale(20,1,20), Material 적용



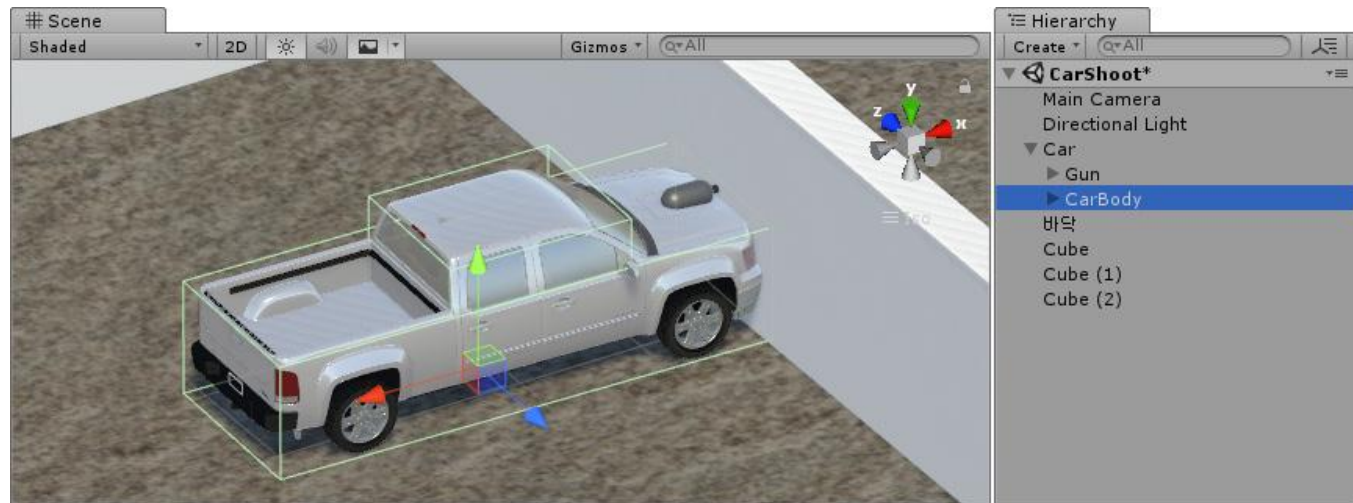
- 장애물 설치
 - [GameObject – 3D Object – Cube]을 클릭하여 Cube를 추가하고 속성을 조정
Scale (0.4, 2, 10), Material 적용
 - 객체를 복사하여 여러 개를 배치





게임을 실행하고 자동차를 이동하면 벽에 막혀 더 이상 전진하지 못함

- 게임 화면에서 Scale을 조정하여 화면을 확대



- 스크립트를 수정



```
public class CarMove : MonoBehaviour
{
    .....
    Rigidbody rgBody;

    // Start is called before the first frame update
    void Start() {
        .....
        rbBody = GetComponent<Rigidbody>();
    }

    // Update is called once per frame
    void Update()
    {
        // 이동 및 회전, 키입력 관련 부분 삭제
    }

    void FixedUpdate()
    {
        // 키입력
        float vert = Input.GetAxis("Vertical");
        float horz = Input.GetAxis("Horizontal");

        // 현재 프레임에서 이동할 거리와 각도
        float amount = moveSpeed * Time.deltaTime * vert;
        float amountRot = rotSpeed * Time.deltaTime * horz;

        // 이동과 회전
        rgBody.MovePosition(rgBody.position + transform.forward * amount);
        rgBody.MoveRotation(rgBody.rotation * Quaternion.Euler(Vector3.up * amountRot));
    }
}
```

FixedUpdate() 함수는 일정한 주기로 호출되며, 이 함수 호출 직후 모든 물리 계산과 갱신이 이뤄진다

Rigidbody의 영향을 받는 오브젝트는 이 함수에서 이동하는 것이 좋음



게임을 다시 실행

- 벽에 접촉한 자동차를 계속 진전시키면, 벽을 타고 넘어간다



UPDATE() VS. LATEUPDATE() VS. FIXEDUPDATE()

- Update()

- 스크립트에서 가장 빈번하게 사용되는 함수
- 스크립트가 켜져 있을 때(enabled 상태일 때) 매 프레임마다 한번씩 호출됨
- 프레임 단위로 호출되기 때문에 이전 프레임과의 시간 차이가 일정하지 않으므로, Time.deltaTime을 사용하여 이전 프레임과의 시간 차이를 확인하여 사용해야 함
- 물리효과가 적용되지 않는 오브젝트의 움직임이나 단순한 타이머, 키입력을 받을 때 사용
- 불규칙한 호출이라서, 물리엔진 충돌 검사 등이 제대로 안될 수도 있음

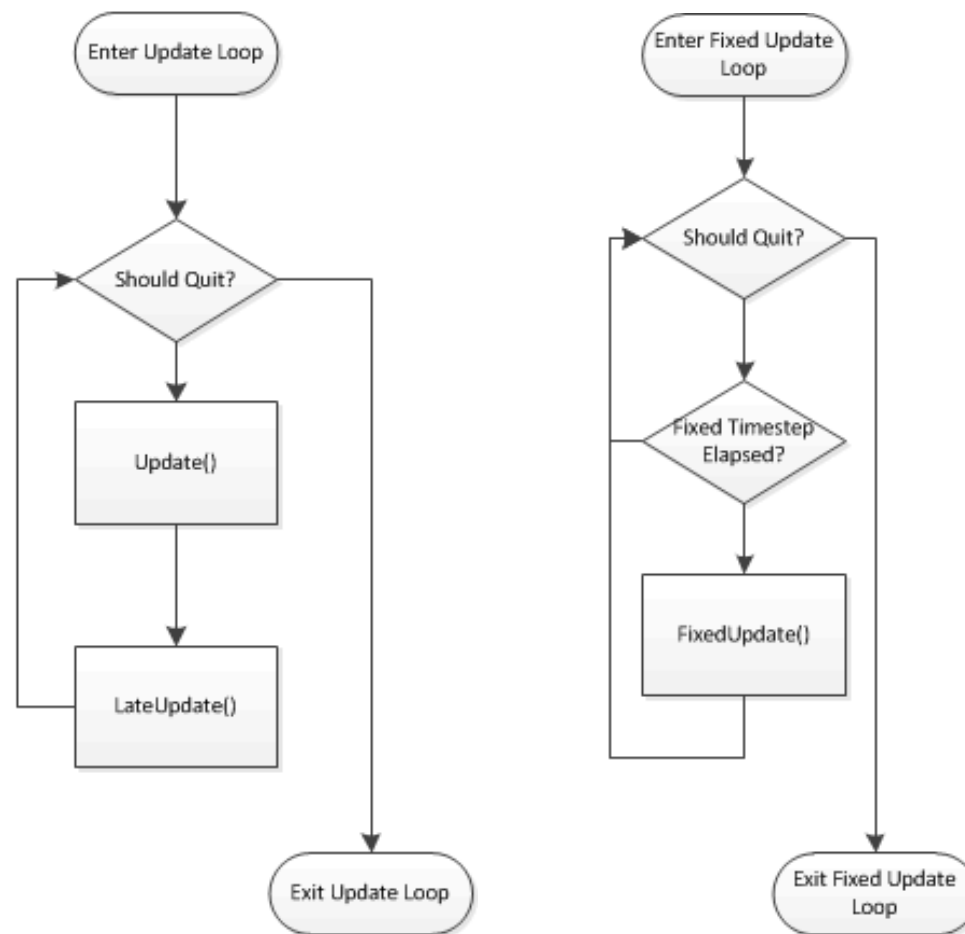
- FixedUpdate()

- 매 프레임마다 호출되지만, 시간 차이가 부정확한 Update()와 달리, 일정한 시간 간격 차를 두고 한번씩 호출됨
(Fixed Timestamp에 설정된 값에 따라 일정한 간격으로 호출)
- Frame rate가 낮으면 한 프레임에서 여러 번 호출될 수도 있으며, 높으면 한 프레임에 호출이 없을 수도 있음
- Frame rate와 독립적인 타이머에 의한 호출이므로, Time.deltaTime이 많이 필요한 모든 물리적 연산과 업데이트는 FixedUpdate()가 호출이 완료된 다음 이뤄진다
- Frame rate와 독립적이기 때문에, FixedUpdate() 메소드 안에서는 Time.deltaTime을 사용할 필요가 없음
- 물리 효과가 적용된(Rigidbody) 오브젝트를 조정할 때 사용됨

UPDATE() VS. LATEUPDATE() VS. FIXEDUPDATE()

- LateUpdate()
 - Update()와 마찬가지로 매 프레임마다 한번씩 호출되는 함수
 - Update() 호출이 완료된 이후에 호출됨 (모든 Update() 함수가 호출된 후, **마지막으로 호출**)
 - **주로 오브젝트를 따라가게 설정하는 카메라는 LateUpdate() 함수를 사용**
 - 주로 Update()에서 캐릭터를 움직인 다음, LateUpdate()에서 카메라를 이동시키도록 처리

– Update() vs. FixedUpdate()

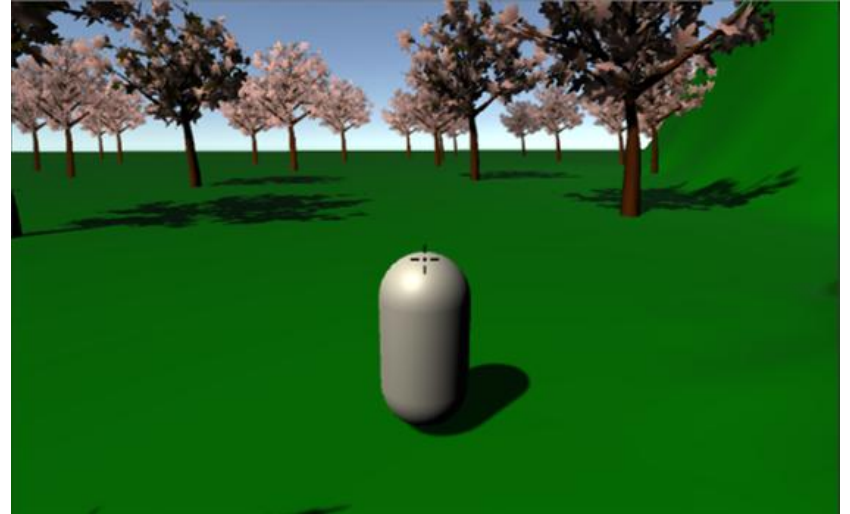
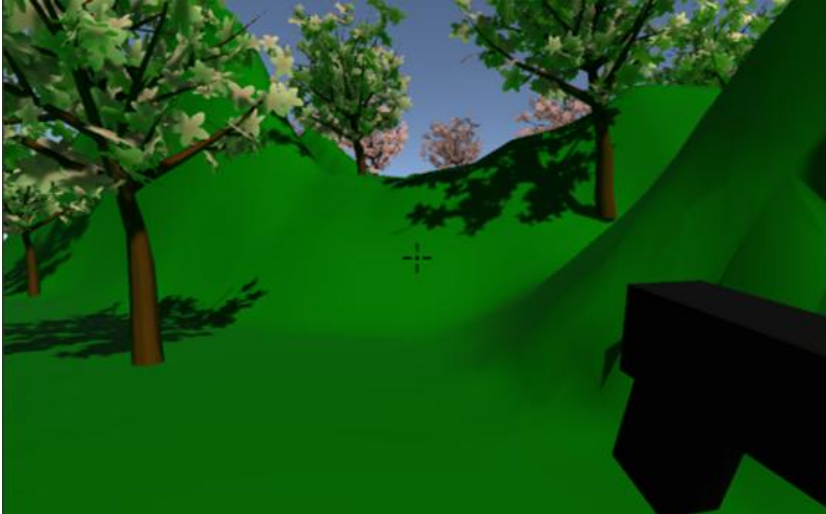


- Update() 함수에서 매 프레임마다 출력하지 않고 바뀌는 값만 출력을 하려면, 이전 값을 저장하고(예, previousValue)

```
double previousValue, Value;  
if (previousValue != Value) {  
    Debug.Log("값 출력 " + Value);  
    previousValue = Value;  
}
```


4.6 CAMERA TRACKING

- 1인칭 카메라와 3인칭 카메라 예시



4.6 CAMERA TRACKING

- 1인칭이나 3인칭 게임에서는 카메라가 플레이어의 뒤를 따라감
 - 카메라가 플레이어와 동일한 속도로 이동하면, 현장감이 떨어짐
 - 카메라는 플레이어보다 한 스텝 늦춰서 조금 루즈하게 이동 (스크립트로 처리)
 - 선형보간(Linear Interpolation)을 이용

4.6.1 Inspector에 변수 노출하기

- 스크립트를 생성하고 이름을 FollowTarget로 수정
- 스크립트에서 `public` 변수를 선언하면, 변수가 인스펙터에 노출
 - 인스펙터에서 값을 직접 변경할 수 있으므로, 편리
- `private` 변수를 인스펙터에 노출시키는 형식을 적용

- 스크립트를 수정하고 저장

```
public class FollowTarget : MonoBehaviour
{
    [Header("카메라 위치, 각도, FOV-----")]
    [SerializeField] Vector3 position = new Vector3(0, 3.6f, -7.8f); // 카메라 초기 위치
    [SerializeField] Vector3 rotation = new Vector3(14, 0, 0); // 카메라의 초기 회전각
    [SerializeField] [Range(10, 100)] float fov = 30f; // 카메라의 Field of View

    [Header("카메라 이동 및 회전 속도 -----")]
    [SerializeField] float moveSpeed = 10f; // 카메라 이동 속도
    [SerializeField] float turnSpeed = 10f; // 카메라 회전 속도

    Transform target; // 추적대상
    Transform cam; // 카메라
    Transform pivot; // 카메라 이동 및 회전 축

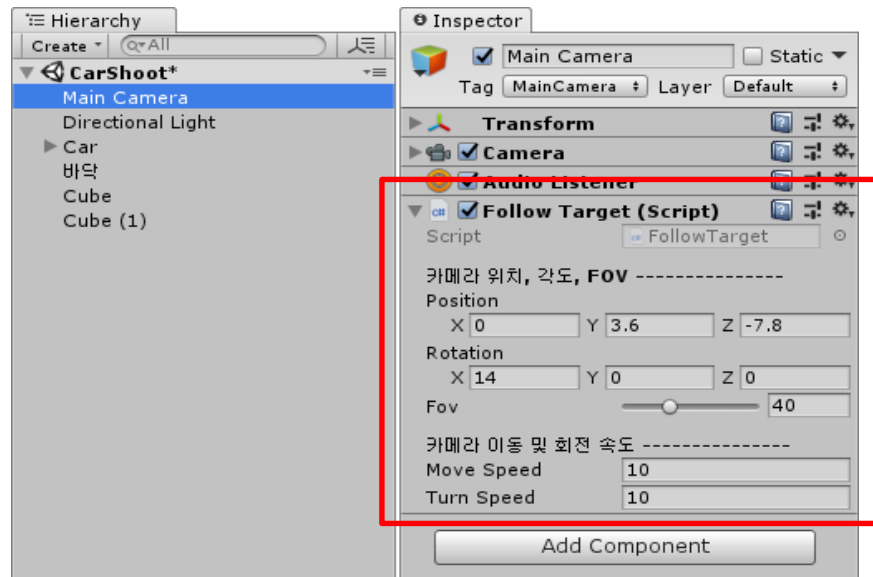
    // Start is called before the first frame update
    void Start() {}

    // Update is called once per frame
    void Update() {}
}
```



[Header()]는 인스펙터에 타이틀 표시
[SerializeField]는 인스펙터에 변수를 표시하며, 변수마다 개별적으로 설정해야 함

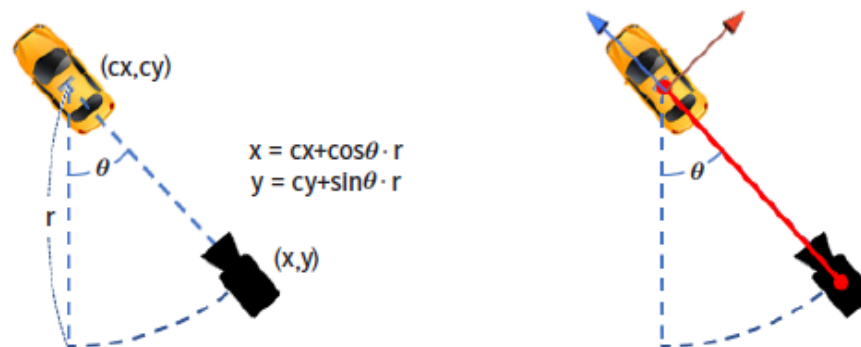
- Main Camera에 스크립트를 연결하면 인스펙터에 표시됨



- 사용자가 인스펙터에 노출된 변수의 값을 바꾸면, 그 값은 프로젝트와 함께 저장되며, 스크립트에 설정한 값은 무시된다.
- 스크립트에서 변수의 값을 다시 설정해도 인스펙터에는 반영되지 않는다.

4.6.2 Pivot 만들기

- 카메라가 오브젝트를 따라갈 때 이동하는 것을 처리가 간단
- 오브젝트의 회전을 간단하지 않음
 - 거리와 회전각을 삼각함수로 처리해서 카메라의 위치를 계산해야 함
 - (x,y) 평면에서의 회전 공식 (3차원 회전은 더욱 복잡)
- 자동차 위치에 빈 오브젝트를 생성하여 Pivot Point로 삼고, 카메라를 피벗의 하위 오브젝트로 만든다 (회전을 쉽게 처리하는 방법)
- 자동차가 회전할 때 각도 만큼 Pivot Point를 회전하면 하위 오브젝트인 카메라는 원을 그리면서 함께 회전 → Pivot Point와 카메라를 막대기로 연결하는 구조



- 스크립트를 수정



```
public class FollowTarget : MonoBehaviour
{
    .....

    // Start is called before the first frame update
    void Start() {
        target = GameObject.Find("Car").transform; // Target 설정 (카메라가 추적할 목표를 설정)

        InitCamera(); // 카메라 초기화
    }

    void InitCamera()
    {
        // 카메라 설정
        cam = Camera.main.transform; // 메인카메라 정보를 읽음
        cam.GetComponent<Camera>().fieldOfView = fov;

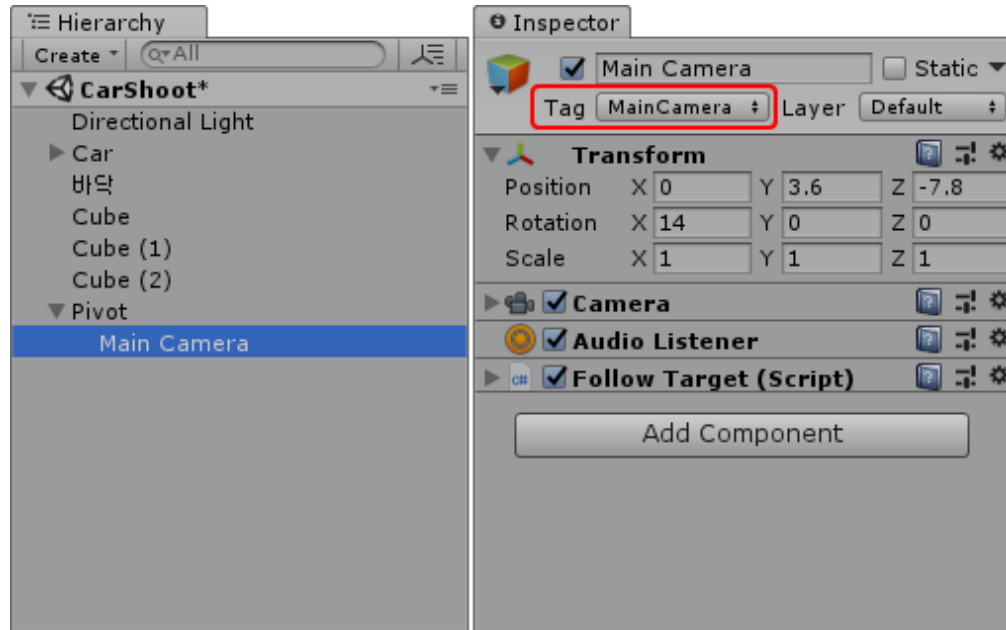
        // 피벗 만들기
        pivot = new GameObject("Pivot").transform; // pivot라는 빈 오브젝트를 생성
        pivot.position = target.position;

        // 카메라를 피벗의 차일드로 설정
        cam.parent = pivot;
        cam.localPosition = position;
        // cam.localEulerAngles = rotation;
        cam.localRotation = Quaternion.Euler(rotation);
    }
}
```



스크립트를 저장하고 게임을 실행

- 하이라키에 오브젝트 Pivot가 생성되고, Main Camera가 Pivot의 child로 설정되는 것을 확인



4.6.3 선형 보간(Interpolation)

- **보간** = 어떤 목표값이 있을 때, 시작값과 목표값 사이의 값들을 추정하는 것
(예) 값이 0 → 1 로 변할 때, 즉시 1이 되는 것이 아니라, 0.1 0.2 0.3 ... 과 같이 그 중간 과정을 단계적으로 보충해서 연속되는 그래프를 만든다.
- 이때 보간되는 값들이 직선 형태이면 “선형보간”, 직선 형태가 아니면 “비선형 보간”
- 유니티에서 제공하는 선형보간 함수 Lerp()를 이용하여 카메라가 목표물을 부드럽게 추적하는 함수를 만든다

```
public class FollowTarget : MonoBehaviour {
    .....
    void LateUpdate()
    {
        // 목적값
        Vector3 pos = target.position;
        Quaternion rot = target.rotation;

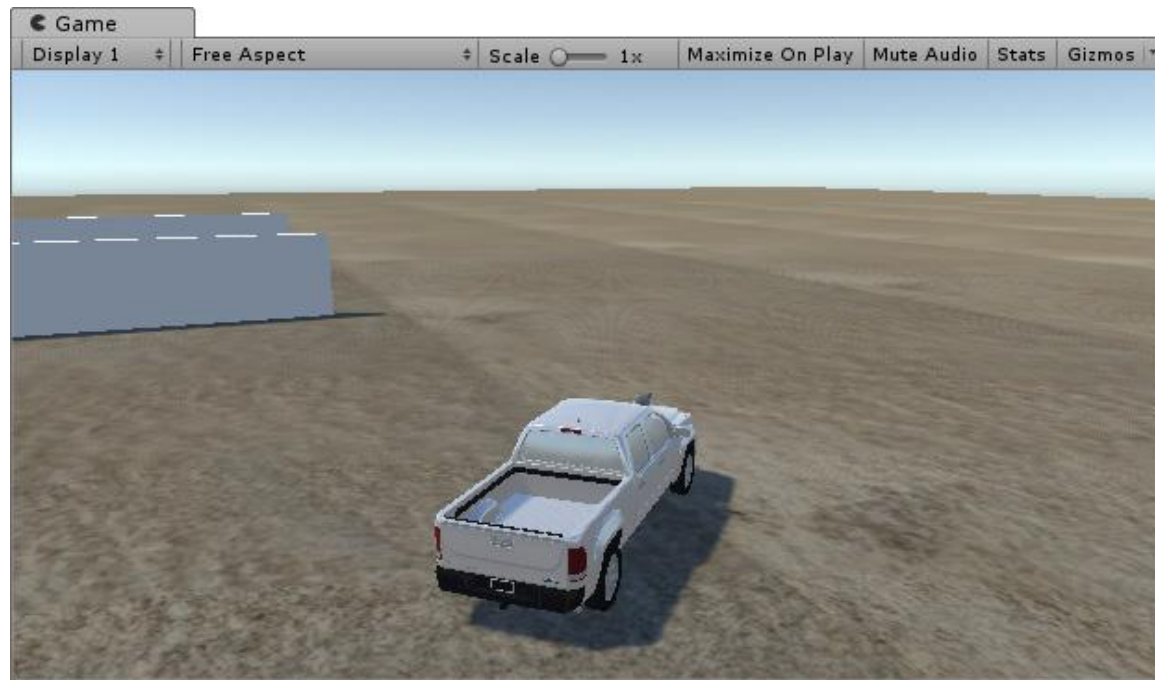
        // 이동
        pivot.position = Vector3.Lerp(pivot.position, pos, moveSpeed * Time.deltaTime);
        // 회전
        pivot.rotation = Quaternion.Lerp(pivot.rotation, rot, turnSpeed * Time.deltaTime);
    }
}
```

LateUpdate()함수는 Update()함수의 실행이 완전히 끝난 후, 호출된다



게임을 다시 실행

- 자동차가 이동하는 대로, 카메라가 부드럽게 따라감



- 선형 보간은 여러 프레임에 걸쳐 진행되며, 보간 간격이 작으면 목적값에 도달하는데 많은 프레임이 필요하므로, 카메라의 이동 속도가 늦어진다.
- 유니티에서 제공하는 Lerp()함수
 - Mathf.Lerp()
 - Vector2.Lerp()
 - Vector3.Lerp()
 - Vector4.Lerp()
 - Quaternion.Lerp()
- Slerp() 함수는 구면 보간이며, 오브젝트의 회전에서 목적값이 아주 멀 때 Lerp() 함수 보다 부드럽게 보간된다

4.6.4 Game 화면의 Zoomin/Zoomout

- 썸 뷰는 마우스 휠의 위쪽 스크롤은 Zoomin, 아래쪽 스크롤은 Zoomout
- Zoomin/Zoomout 하는 것 = 카메라의 FOV(Field of View)를 변경하는 것
- 게임에서 위쪽 스크롤을 Zoomin으로 설정

```
public class FollowTarget : MonoBehaviour {
    ....
    void Update()
    {
        // zoomin (wheel up+) zoomout (wheel down-)
        float zoom = Input.GetAxis("Mouse ScrollWheel") * 20;
        fov = Mathf.Clamp(fov - zoom, 10, 100);
        cam.GetComponent<Camera>().fieldOfView = fov;
    }
}
```

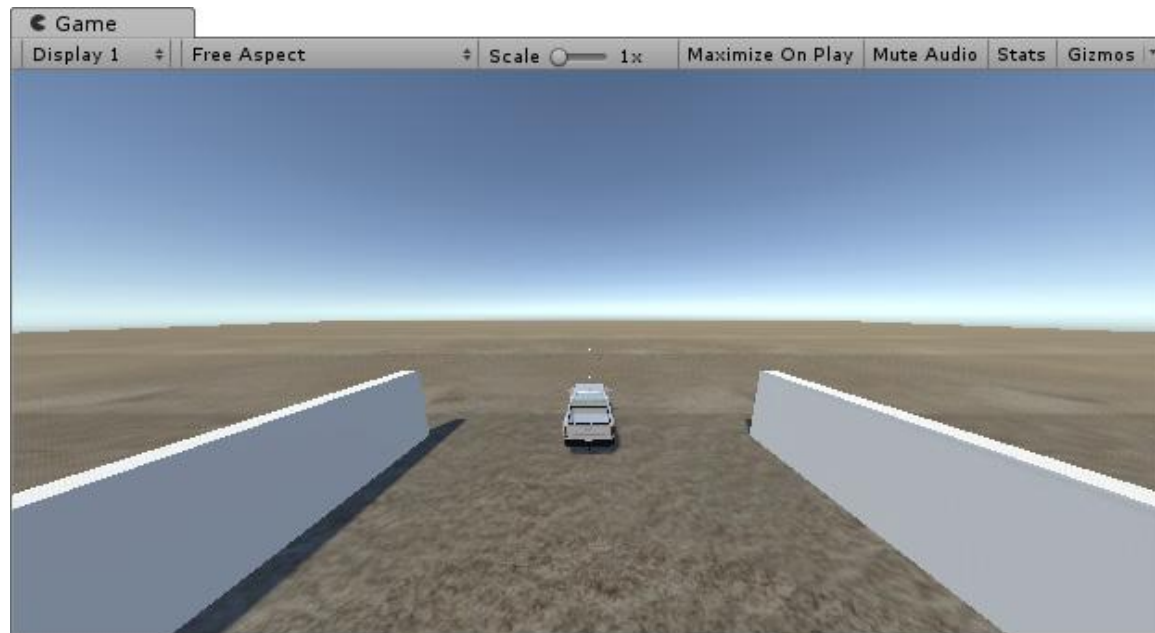
오브젝트의 이동과 관계없는 기능
이므로, Update()에서 처리

- 마우스 휠은 위로 스크롤하면 양(+)의 값을 가지며, 카메라의 FOV는 값이 작을수록 Zoom이 되므로, fov-zoom을 해준다
- 줌인/줌아웃은 적당한 범위로 제한할 필요가 있음 : Clamp() 함수의 매개변수 적용
→ (현재값, 최소, 최대)



게임을 실행

- 마우스 휠을 이용하여 게임 화면을 Zoomin/Zoomout



– Mouse로 Camera 회전

- 3인칭 FPS 게임에서는 **마우스를 드래그해서 카메라의 시점을 바꾸는 기능**을 가짐
 - 1인칭 게임에서도 마우스 드래그를 사용하지만, 이는 주인공 자체가 회전하는 것이며 카메라가 독립적으로 회전하는 것은 아님
 - 별도의 Pivot Point가 필요
- 현재의 Pivot Point는 목표물만 따라 다니므로, 마우스로 회전각을 바꿔도, 바로 원상태로 복귀해 버림
- 썬 뷰는 오른쪽 마우스 버튼을 누르고 드래그하면 카메라의 방향이 바뀌는데, 여기에서도 **오른쪽 마우스 버튼을 누르고 드래그해서 게임 화면의 화각을 바꿈**
- 현재의 오른쪽 마우스 버튼은 연발 사격의 용도로 사용되고 있으므로, CarFire 스크립트의 발사 버튼을 다른 것으로 변경할 필요가 있음

- (이전의) 연속발사 키인 Fire2를 Left-Shift키로 변경

```
public class CarFire: MonoBehaviour {
    .....
    void Update()
    {
        .....

        // 연발 사격 Coroutine 호출
        // if (Input.GetButton("Fire2") && canFire) {
        if (Input.GetKey(KeyCode.LeftShift) && canFire) {
            StartCoroutine(AutoFire2());
        }

        // 버튼을 놓으면 사운드 중지
        // if (!Input.GetButton("Fire2")) {
        if (!Input.GetKey(KeyCode.LeftShift)) {
            gunSound[1].Stop();
        }
    }
}
```



- (1) Camera 회전용 Pivot Point 추가
- 변수 영역에 새로운 Pivot Point를 추가하고 카메라 초기화 함수에 필요한 부분을 추가

```
public class FollowTarget: MonoBehaviour {
    .....

    Transform pivot;    // 카메라 이동 및 회전 포인트
    Transform pivotRot; // 마우스로 회전할 Pivot

    // 카메라 초기화
    void InitCamera() {
        // Camera 설정
        cam = Camera.main.transform;
        cam.GetComponent<Camera>().fieldOfView = fov;

        // Pivot 만들기
        pivot = new GameObject("Pivot").transform;
        pivot.position = target.position;

        // 마우스 회전용 Pivot 만들기
        pivotRot = new GameObject("PivotRot").transform;
        pivotRot.position = target.position;
        pivotRot.parent = pivot;

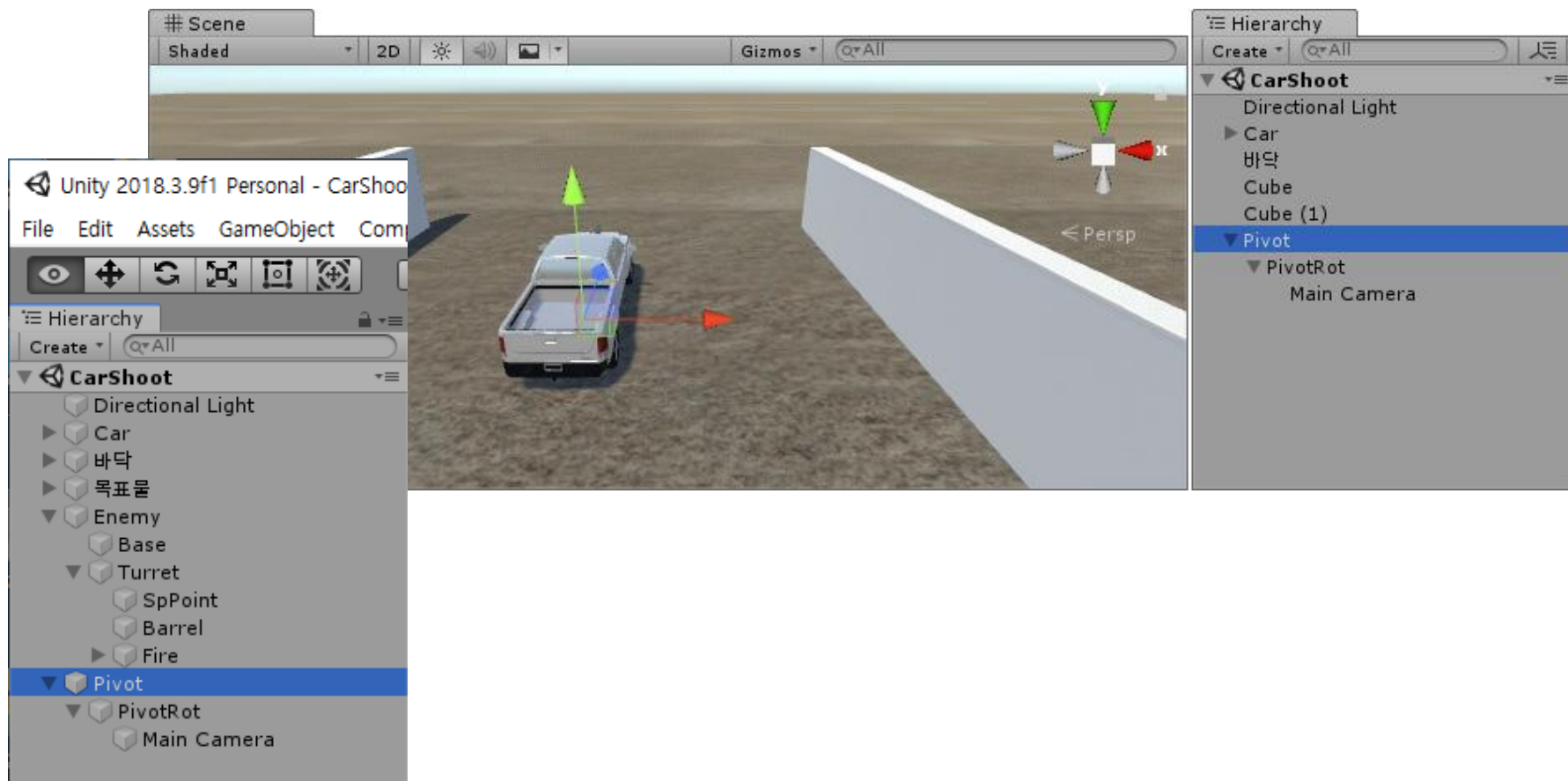
        // 카메라를 PivotRot의 Child로 설정
        // cam.parent = pivot;
        cam.parent = pivotRot;
        cam.localPosition = position;
        cam.localEulerAngles = rotation;
        cam.localRotation = Quaternion.Euler(rotation);
    }
}
```





게임을 실행

- 하이어리키의 Pivot Point가 제대로 구성되는 지 확인



- 마우스로 카메라를 회전시키는 스크립트를 Update() 끝 부분에 추가
- Pivot Point가 x축으로 회전할 때, 카메라가 지면 아래를 벗어나지 않도록 처리하는 등 복잡한 절차가 포함됨

```
public class FollowTarget: MonoBehaviour {
    .....

    void Update () {
        .....
        // 오른쪽 버튼을 누르고 있지 않으면 리턴
        if (!Input.GetMouseButton(1)) return;

        // 마우스 이동 방향
        float x = Input.GetAxis("Mouse Y") * 2;    // 상하 이동은 x축 회전
        float y = Input.GetAxis("Mouse X") * 2;    // 좌우이동은 y축 회전

        // 회전할 각도 계산
        Vector3 ang = pivotRot.localEulerAngles + new Vector3(x, y, 0);

        // x축의 회전각 변환 (0~360 --> -180~+180)
        if (ang.x > 180) {
            ang.x -= 360;
        }

        // x축 회전 범위 제한 (지면 아래와 수직을 벗어나지 않도록)
        ang.x = Mathf.Clamp(ang.x, -24, 80);

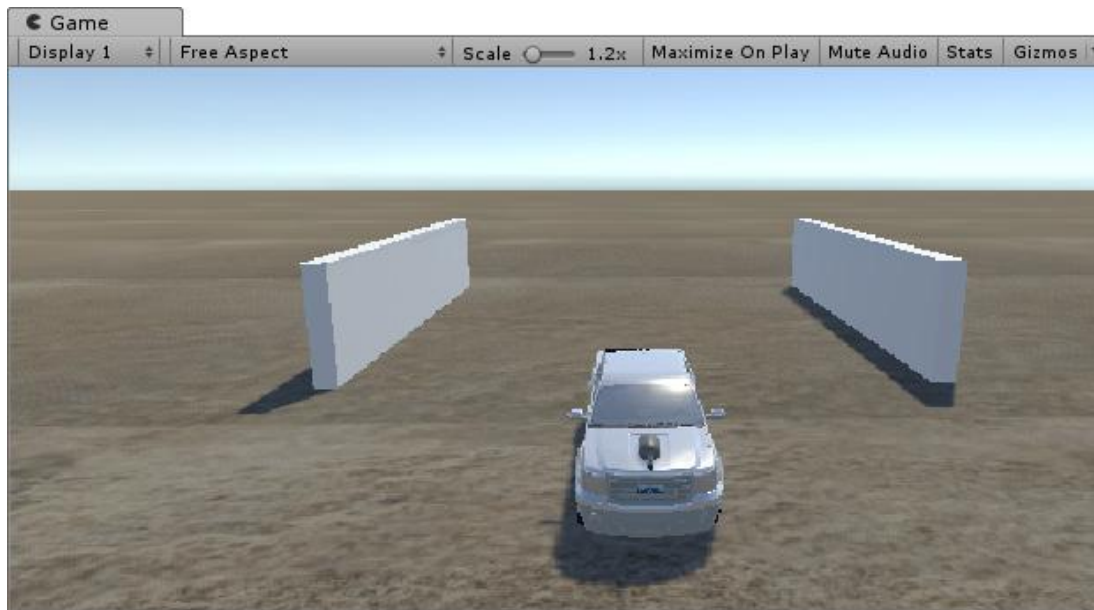
        pivotRot.localEulerAngles = ang;
        // pivotRot.localRotation = Quaternion.Euler(ang);
    }
}
```





게임을 실행

- 게임 화면이 상하 90도, 좌우 360도로 회전



4.7 발사 화염과 폭파 불꽃

- 총알을 발사할 때 총구 앞에서 화염을 표시하고, 목표물에 맞았을 때의 폭파 불꽃을 만든다

4.7.1 총구의 발사 화염

- 오브젝트가 폭파될 때 발생하는 폭파 불꽃이나 햇불처럼 지속적으로 타오르는 화염은 파티클 (Particle)로 처리한다
- 파티클은 수많은 입자를 지속적으로 변경시켜야 하므로, 하드웨어 성능에 영향을 받으며, 각 입자를 특정한 방향으로 이동시키려면 처리 절차도 복잡

(1) 총구의 화염 이미지

- 총구의 화염은 순식간에 나타났다 사라짐
- 화염의 패턴이 동일하고 지속시간이 짧기 때문에, 이미지를 사용하는 것이 효과적
- 프로젝트 브라우저에 Sprite폴더를 생성하고 필요한 이미지를 추가

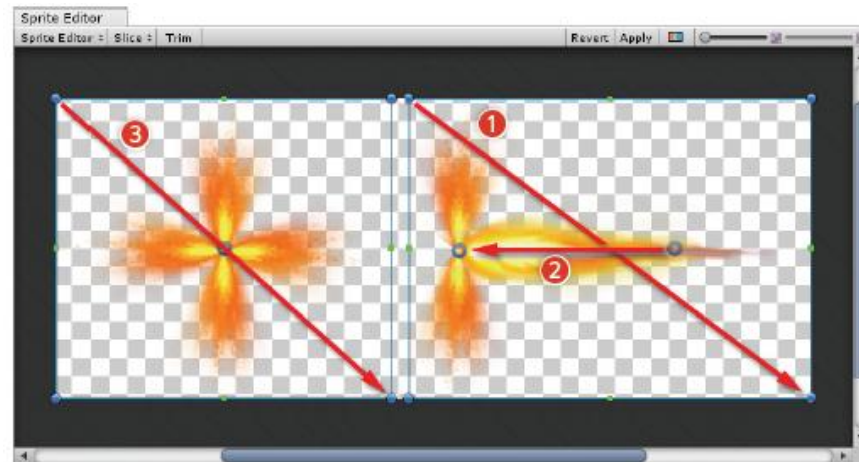


– 속성 수정

Object	Name	Texture Type	Sprite Mode	Pixel Per Unit
Sprite	FireEffect	Sprite(2D and UI)	Multiple	200

(2) Sprite 자르기

- » 인스펙터에서 [**Sprite Editor**] 버튼을 클릭
- » Sprite 편집기에서 (Slice의 Type을 조정하고) 자를 영역을 드래그
- » 스프라이트는 영역을 지정한 순서로 저장되므로, 오른쪽 부분을 먼저 지정하고 Pivot (파란색 원=그림에서 2번)을 왼쪽으로 이동시켜 둔다

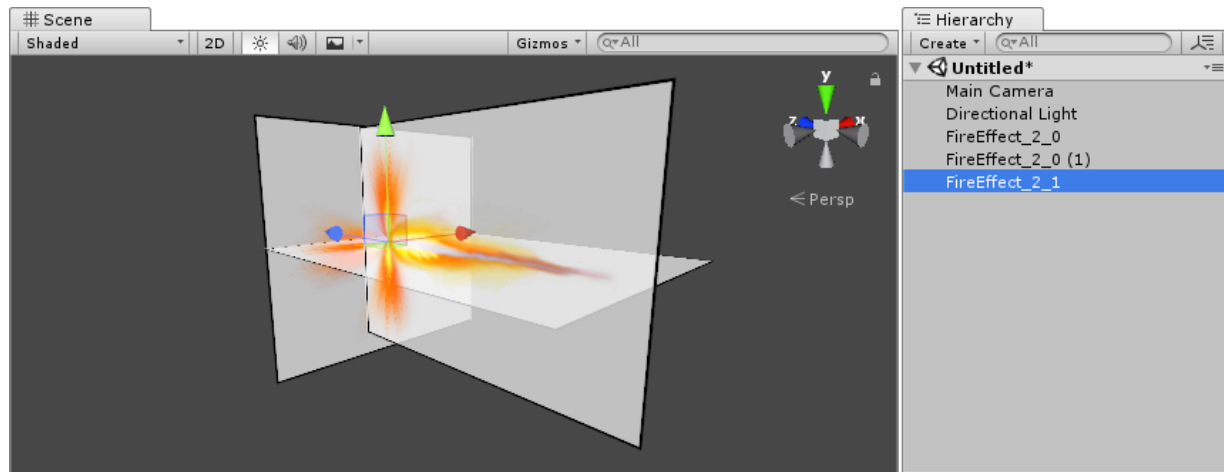


(3) 발사 불꽃 만들기

- » 2D 이미지는 각도에 따라 보이지 않는 문제가 있을 수 있음
- » (1)번 이미지 2장을 x축으로 90도 엇갈리게 배치하여 문제 해결
- » 총구 뒷면에서 불꽃이 보이도록 왼쪽 이미지를 배치
- » (1)번 스프라이트 2개, (2)번 스프라이트 1개를 설치하고 속성을 설정

Object	Name	Position	Rotation	Scale
Sprite	FireEffect_0	(0, 0, 0)	(0, 90, 0)	(1, 1, 1)
	FireEffect_0 (1)	(0, 0, 0)	(90, 90, 0)	(1, 1, 1)
	FireEffect_1	(0, 0, 0)	(0, 0, 0)	(1, 1, 1)

- » 속성을 적용하여 스프라이트를 다음과 같은 모양으로 배치 (실제 화면에는 여백이 보이지 않음)



- » (0,0,0)으로 맞추려면, Scene에서 “Tool Handle Position”을 (Center에서) Pivot으로 변경한 다음, 위치를 조정

(4) Point Light 추가

- » 발사 불꽃이 나타날 때마다 반짝이는 빛이 보일 수 있도록 Point Light를 추가
- » 메뉴 [**GameObject – Light – PointLight**]를 클릭

Object	Name	Position	Range	Intensity
Point Light	Point Light	(0, 0, 0)	1.5	2

(5) 프리팹 만들기

- » Position (0,0,0)에 빈 오브젝트를 추가하고 이름을 FireEffect로 설정
- » 씬의 스프라이트와 Point Light 모두를 드래그하여 FireEffect의 하위 오브젝트로 만들
- » 스크립트를 작성하여 FireEffect에 연결한 후, 프리팹으로 만들

```
public class FireEffect : MonoBehaviour {
    ....
    // OnEnable
    void OnEnable() { Invoke("Disable", 0.1f); } // 오브젝트가 활성화될 때 호출
                                                // Invoke()는 지정한 함수를 지시한 시간 후에 호출

    // Disable GameObject
    void Disable() { gameObject.SetActive(false); } // 오브젝트를 비활성화
}
```

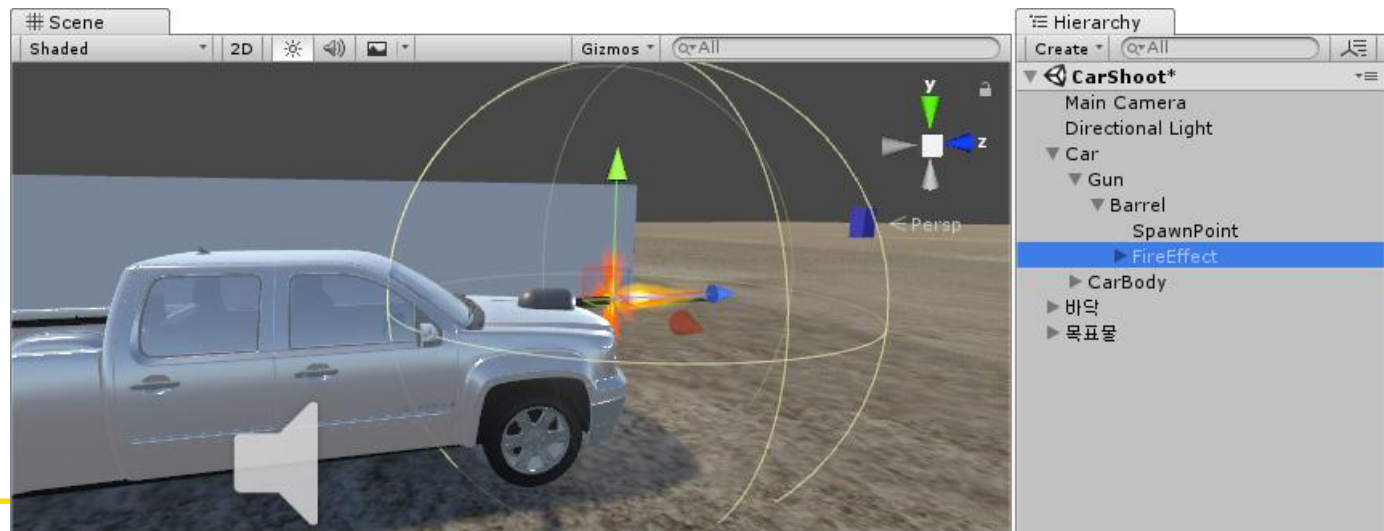
OnEnable()은 오브젝트가 활성화될 때 호출됨
Invoke()는 지정한 함수를 지시한 시간 후에 호출하며, 호출하는 함수 이름은 문자열로 지정

- » 위 스크립트는 오브젝트가 활성화될 때 호출되어, 0.1초 후에 자신을 다시 비활성화하므로, 불꽃 이미지는 0.1초 동안만 화면에 나타나게 됨
- » **Invoke()**는 함수를 지연 호출하기 위해 사용됨
- » 호출하는 함수에 매개변수를 전달하지 못하는 단점이 있음
- » 매개변수가 필요한 함수의 지연 호출은 코루틴으로 처리
- » (앞 슬라이드의) 위 스크립트를 코루틴으로 수정

```
public class FireEffect : MonoBehaviour {  
    ....  
    // OnEnable  
    void OnEnable() { StartCoroutine(Disable(0.1f)); }  
  
    // Disable GameObject  
    IEnumerator Disable(float delay) {  
        yield return new WaitForSeconds(delay);  
        gameObject.SetActive(false);  
    }  
}
```


(6) 총구 앞에 불꽃 표시하기

- » 씬에 프리팹 FireEffect를 추가하고 총구 앞에 배치
- » 불꽃 크기가 큰 경우, Scale를 조정하여 적당한 크기로 조정
- » 위치와 크기가 결정되면, FireEffect를 Car/Gun/Barrel로 드래그하여 총열의 Child로 만들
- » 처음부터 불꽃을 Barrel의 하위 오브젝트로 만들면 인스펙터에는 상대좌표가 표시되어 정밀하게 제어하기가 어려움 → 속성을 먼저 설정한 후, 하위 오브젝트로 드래그하는 것이 편리



(7) 총구의 불꽃 제어하기

- » 총구의 불꽃은 평상시에는 보이지 않고 총알이 발사되는 순간에 나타나야 함
- » 처리 부분은 CarFire에 추가

```
public class CarFire : MonoBehaviour {
    public Transform bullet;
    Transform spPoint;
    GameObject fire; // 발사 불꽃을 처리할 변수
    ....
    // 게임 초기화
    void Start() {
        ....
        spPoint = GameObject.Find("SpawnPoint").transform;
        fire = GameObject.Find("FireEffect");
        fire.SetActive(false); // 시작 시점에서는 발사 불꽃을 비활성화
    }

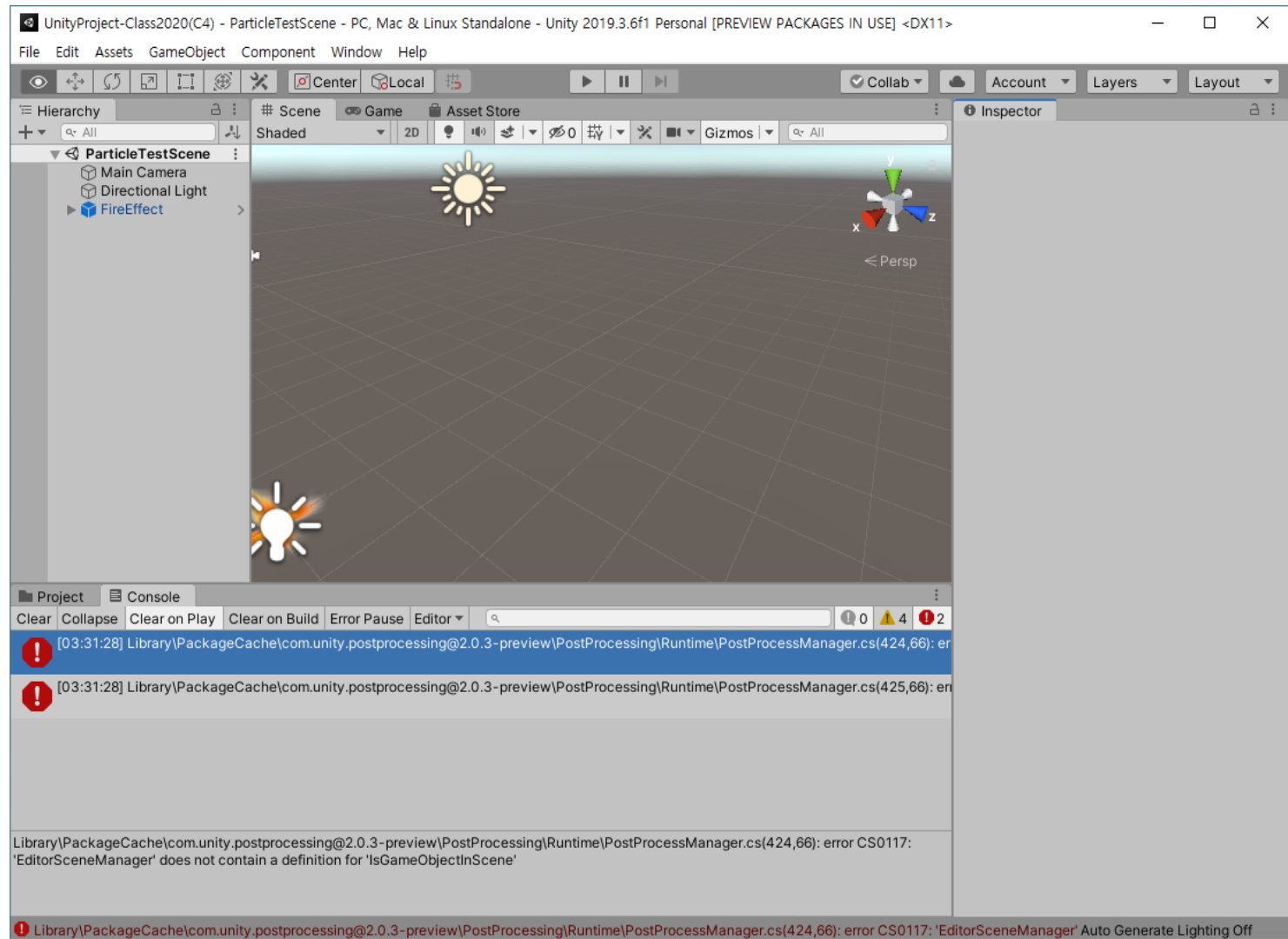
    // 단발 사격
    void SingleShut() {
        ...
        fire.SetActive(true); // 사격 시에 발사 불꽃을 활성화
    }
    // 연발사격
    IEnumerator AutoFire2() {
        ....
        fire.SetActive(true);
    }
}
```

- » (1)의 변수를 Transform 타입으로 선언해도 되지만,
fire.gameObject.SetActive()로 표기해야 함
- » 활성화/비활성화 부분이 여러 군데 추가되므로, GameObject로 선언하는 것이 간결



게임을 실행하면 총을 발사할 때마다 화염이 나타나고 총구 주변이 밝게 번쩍이는 것을 확인







Home

PUBLIC

Stack Overflow

Tags

Users

Jobs

TEAMS

What's this?

Free 30 Day Trial

Products

Customers

Use cases

Search...

Log in

Sign up

error CS0117: 'SceneManager' does not contain a definition for 'IsGameObjectInScene'

Asked 11 months ago · Active 11 months ago · Viewed 3k times

an error shows i cant make the build in unity 2018.3.8

i try to copy all assets to another folder but still error show

i am really stuck to resolve it

Library\PackageCache\com.unity.postprocessing@2.0.3-preview\PostProcessing\Runtime\PostProcessManager.cs(424,66): error CS0117: 'SceneManager' does not contain a definition for 'IsGameObjectInScene'

c# unity3d

share improve this question follow

asked Jun 16 '19 at 18:05

Moaaz Afzal 13 1 3

add a comment

1 Answer

Active Oldest Votes

Multiple people already had that issue. It seems that it points to an outdated file from PostProc version 2.0.3. E.g. in [this thread](#) they solved it by

To fix this, you can delete Library/PackageCache/ the post processing folder

or

updating the "Post Processing" package to the verified version in the package manager fixed it.

Currently it should be version 2.1.7

Packages

All packages Advanced Search by package name, verified, preview or version number...

Advertisement 2.0.8

Alembic 1.0.5

Analytics Library 3.2.2

Asset Bundle Browser 1.7.0

Post Processing Up to date 2.1.7 Remove

Version 2.1.7

View documentation · View changelog · View licenses

com.unity.postprocessing

The Overflow Blog

The Overflow #22: The power of sharing

Deno v1.0.0 released to solve Node.js design flaws

Featured on Meta

Meta escalation/response process update (March-April 2020 test results, next...

iOS Mobile App - Push notifications down from 5/25 - 6/4

Threshold experiment results: closing, editing and reopening all become more...

It's time to reward the duplicate finders

Looking for a job?

Senior .Net Developer

Inversion Limited No office location

\$54K - \$86K REMOTE

c# sql-server

High response rate

Game Programmer - Join a leading hypercasual studio and work on your own games!

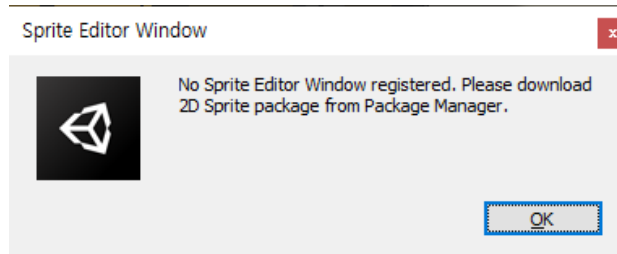
kwalee Ltd Royal Leamington Spa, UK

£25K - £35K REMOTE

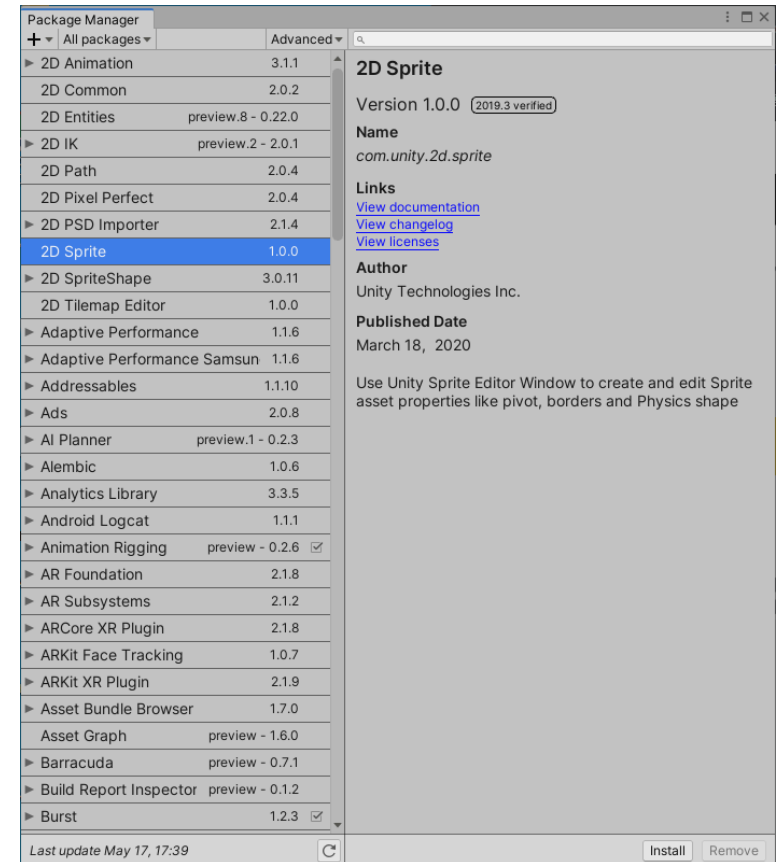
c# unity

Software Test Engineer for HANA Engine Intelligence Team

– Download 2D Sprite Package from Package Manager



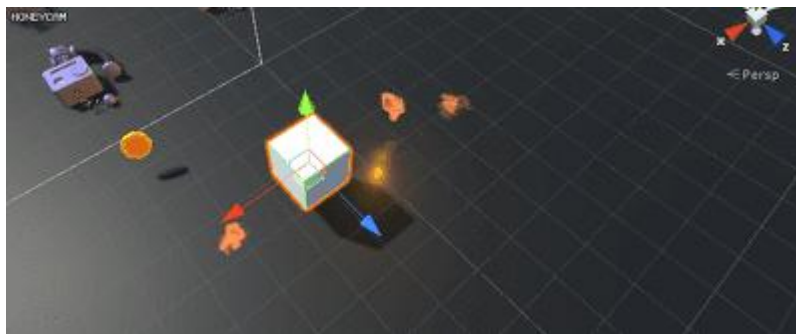
- 메뉴 [Window – Package Manager]
- 2D Sprite 를 선택하고 Install 버튼 클릭



4.7 발사 화염과 폭파 불꽃 (P.183)

- Particle System

- 파티클이라는 매우 작은 이미지나 메쉬를 시뮬레이션하고 렌더링하여 시각효과를 생성
- 불, 연기, 액체 등과 같은 동적 오브젝트를 구현할 때 유용

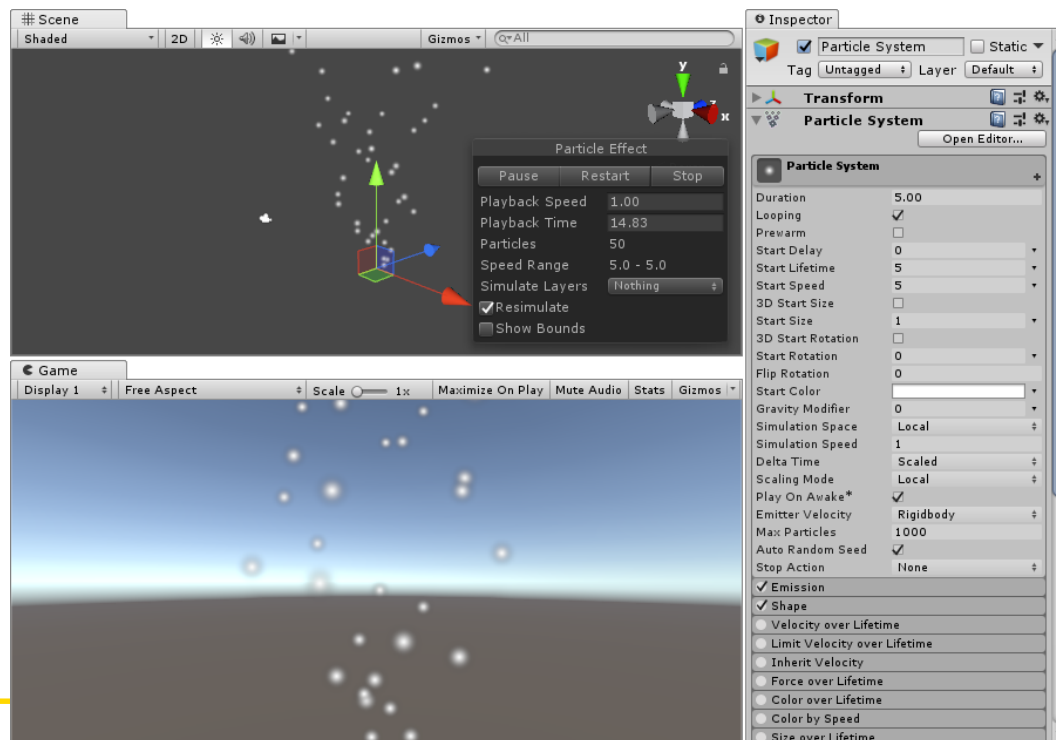


4.7.2 Target의 폭파 불꽃

- Target이 폭파될 때의 화려한 폭파 불꽃을 만들

(1) 파티클(Particle) 만들기

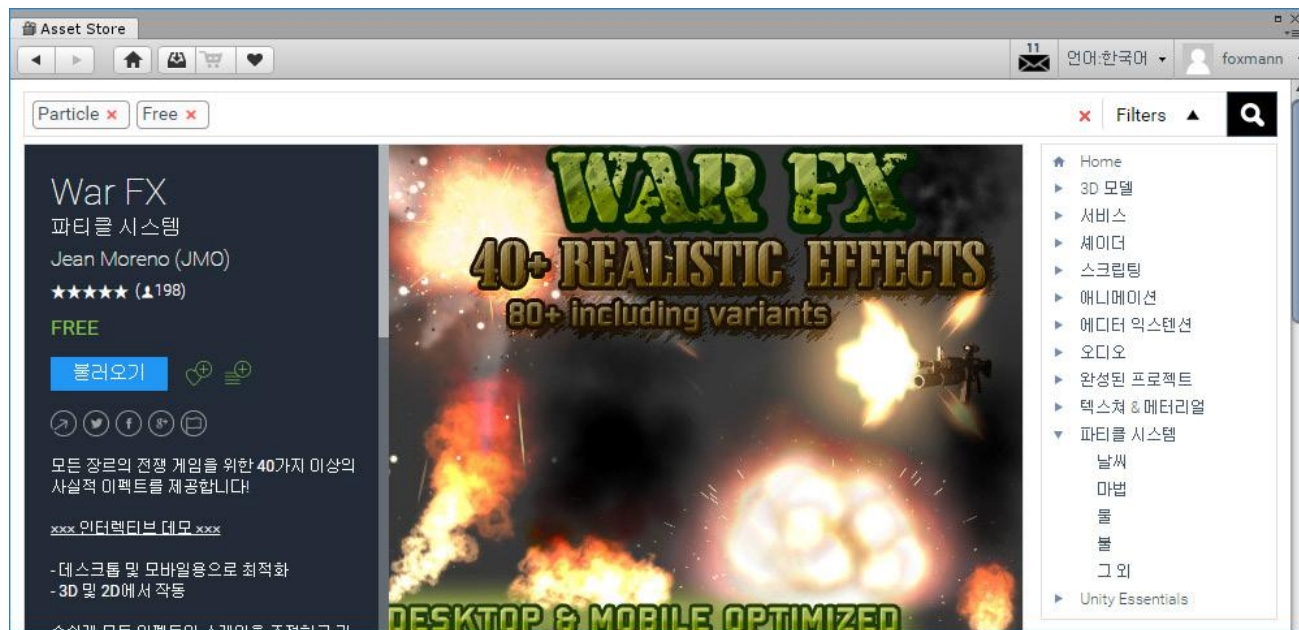
- 메뉴 [**GameObject - Effects - Particle System**]를 클릭하여 파티클을 추가
- 눈송이 같은 입자가 여기저기 날아감



- 폭파 불꽃 모양의 파티클은 불꽃과 연기 이미지가 여러 장 필요
- 입자를 제어하기 위해 많은 속성을 설정해야 함
- 불꽃은 중심부와 바깥쪽을 별도로 만들어야 하며, 연기 파티클이 필요
- 게임 개발 측면에서 보면, 파티클은 디자인 영역이므로, 개발자가 직접 만드는 것은 권장하지 않음
- 개발자가 만들 경우, 투자하는 노력에 비해 품질이 떨어질 수 있음
- 전문가들이 제작한 에셋을 사용하는 것이 효율적임
- 애셋 스토어의 파티클을 이용

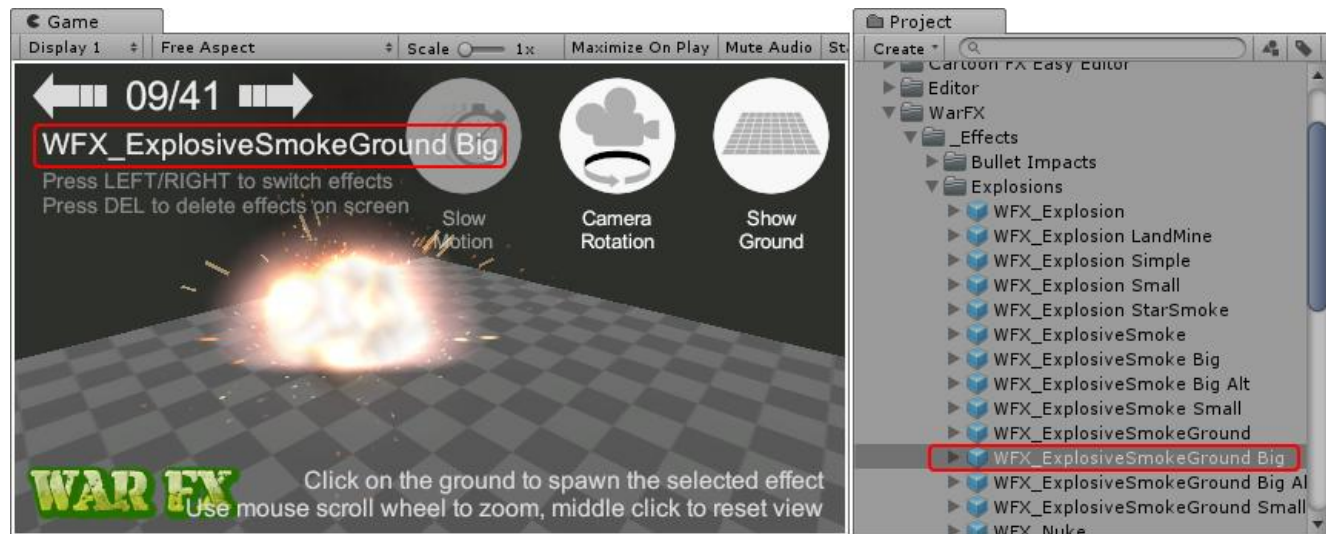
(2) Particle Asset 추가

- 에셋 스토어에서 "Particle Free"로 검색하여 "War FX"라는 무료 파티클 에셋을 다운로드



- War FX는 유니티 2018 이전 버전에서 제작된 것으로, 프로젝트에 추가하면 API Update Required 창이 나타나고 [..Go Ahead] 버튼을 클릭하여 API를 업데이트

- 추가한 에셋은 프로젝트의 JMO Assets 폴더에 저장됨
- 40개 이상의 파티클이 있으며, 자신의 프로젝트에 필요한 것을 고름
- War FX/Demo 폴더의 데모 씬을 실행하고, 바닥을 클릭하면 파티클이 표시됨
- 게임 뷰 위쪽의 좌우 화살표는 이전/이후의 파티클을 표시
- 여러 파티클을 실행하여 적당한 것을 고름



(3) Particle을 프리팹 폴더에 복사

- War FX는 파티클이 프리팹으로 되어 있음
- 게임 뷰에 프리팹 이름이 표시됨
- 사용하려는 파티클을 `_Effects` 폴더에서 찾아 `Prefabs` 폴더로 복사하고, 이름을 `Explosion`으로 설정 (프로젝트 뷰에서는 `Ctrl-D`로 복제한 후, 복제한 파티클을 `Prefabs` 폴더로 드래그하고 해당 씬에서 적용함)
- 파티클을 프리팹으로 만들어 놓고, 충돌 처리에서 게임 객체를 제거할 때 파티클을 게임 뷰에 표시한다

`Instantiate(explosion, transform.position, Quaternion.identity);`

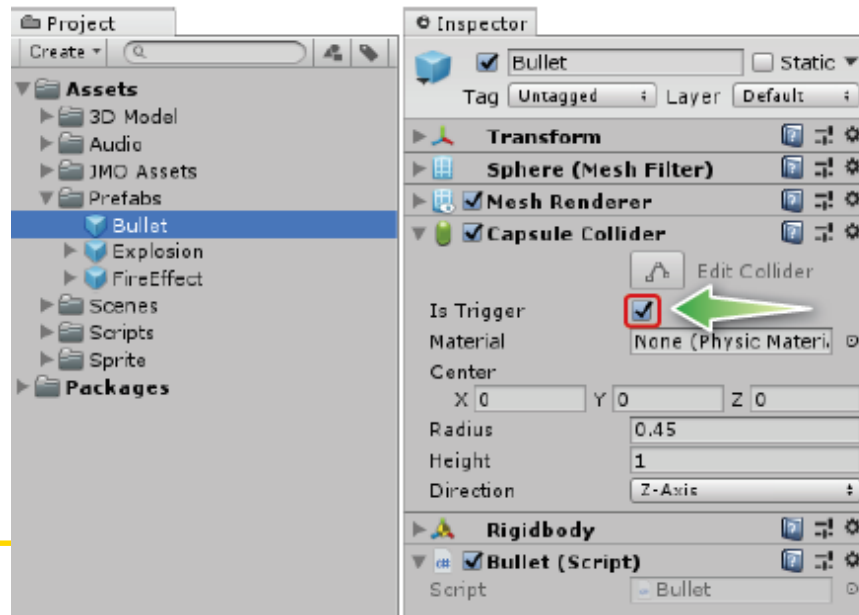
» `Quaternion.identity`는 회전각을 쿼터니언으로 표시한 것으로, `identity`는 회전하지 않은 각도

4.8 충돌의 판정과 처리

- 게임에서 충돌 판정 및 처리는 매우 중요
- 충돌 이벤트는 Rigidbody가 있는 오브젝트에서만 발생

4.8.1 충돌 이벤트의 종류

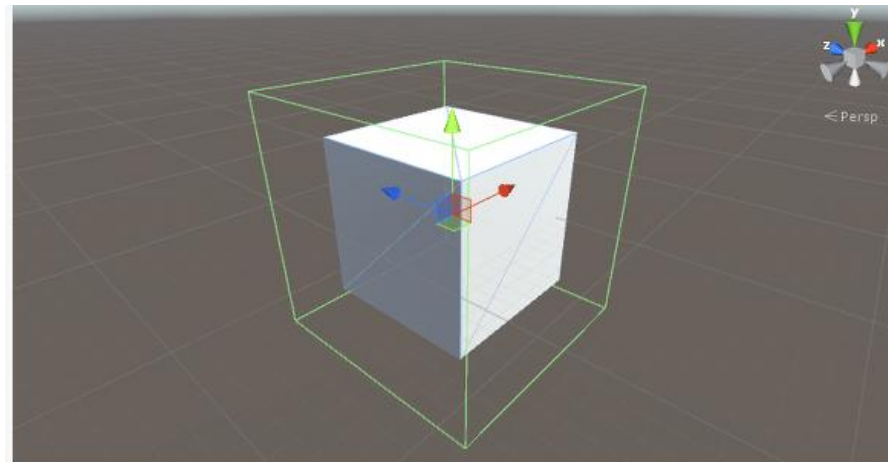
- 충돌이 발생할 때, 충격을 가하는 오브젝트가 반사되느냐 관통하느냐에 따라 발생하는 이벤트가 달라지며, 반사 및 관통 여부는 Collider의 Is Trigger 옵션으로 설정
 - Is Trigger ON : 오브젝트 관통, Trigger 이벤트 발생
 - Is Trigger OFF : 오브젝트에서 반사, Collision 이벤트 발생



- 충돌 : 2개의 오브젝트가 서로 접촉하는 상태
- 적어도 어느 하나가 Trigger On 이면 Trigger 이벤트가, 둘 다 Off면 Collision 이벤트가 발생한다
- 충돌 이벤트의 매개변수의 type
 - OnTrigger ... (Collider other) Trigger 이벤트
 - OnCollision ... (Collision other) Collision 이벤트
- Collision의 반사는 절대적인 것이 아니라서 작은 물체가 빠른 속도로 이동하는 경우에는 가끔씩 오브젝트를 관통하기도 한다. 총알이나 화살 같은 발사체는 오브젝트를 관통하기도 하고 반사되기도 하므로, 일반적으로 Trigger를 On을 설정
- 프리팹으로 만든 Bullet도 Trigger를 On으로 설정해야 한다

– 충돌 처리를 위한 조건

- 충돌이 일어나기 위해서는 두 GameObject가 모두 Collider를 가지고 있어야 하며, 둘 중 하나는 Rigidbody를 가지고 있어야 한다
- 두 GameObject 중 하나만 움직인다면, 움직이는 GameObject가 Rigidbody를 가지고 있어야 한다



- 위 그림은 Collider를 잘 보이게 하기 위해 GameObject 보다 크기를 약간 키워 보임. 위에서 보이는 초록색 부분이 Collider로 실제로 충돌을 감지하는 영역임
- 유니티에서 제공하는 Object들에는 기본적으로 Collider가 들어가 있으며, Box Collider, Sphere Collider, Capsule Collider 등이 있음. 다른 Model을 불러와 작업한다면, Model에 알맞게 Collider를 설정해 줘야 올바른 충돌 처리를 할 수 있음

- Trigger

Trigger는 GameObject간의 물리적 연산을 하지 않고 충돌을 감지할 수 있다. 즉, 두 GameObject가 접촉했을때 서로 튕겨 나가지않고 그냥 통과하게 된다.

Trigger를 쓰기 위해서는 해당 Collider의 Is Trigger 항목을 체크해야 한다.

다음으로 스크립트에서 함수로 충돌 이벤트를 처리할 수 있다.

```
1 void OnTriggerEnter(Collider col) { }  
2 void OnTriggerStay(Collider col) { }  
3 void OnTriggerExit(Collider col) { }
```

다음과 같이 Enter, Stay, Exit 3가지 버전의 OnTrigger 함수를 만들 수 있다.

3가지 함수는 다음과 같은 특징을 나타낸다.

Enter - 충돌이 시작되는 순간 호출

Stay - 충돌이 되고있을 때 매 프레임마다 호출

Exit - 충돌이 끝날 때 호출

함수의 파라미터로 Collider 객체가 들어오며 col을 이용해 충돌한 GameObject에 대한 처리를 할 수 있다.

- Collision

Collision은 Trigger와 다르게 물리적인 연산을 하며 충돌을 감지한다.

주의할 것은 Rigidbody의 Kinematic 속성이 꺼져 있어야 충돌이 발생할 수 있다.

```
1 void OnCollisionEnter(Collision col) { }  
2 void OnCollisionStay(Collision col) { }  
3 void OnCollisionExit(Collision col) { }
```

Trigger와 동일하게 3가지 함수를 만들 수 있으며 특징 또한 동일하다.

함수의 파라미터로 Collision 객체가 들어오며 col을 이용해 충돌한 GameObject에 대한 처리를 할 수 있다.

4.8.2 Target 설치

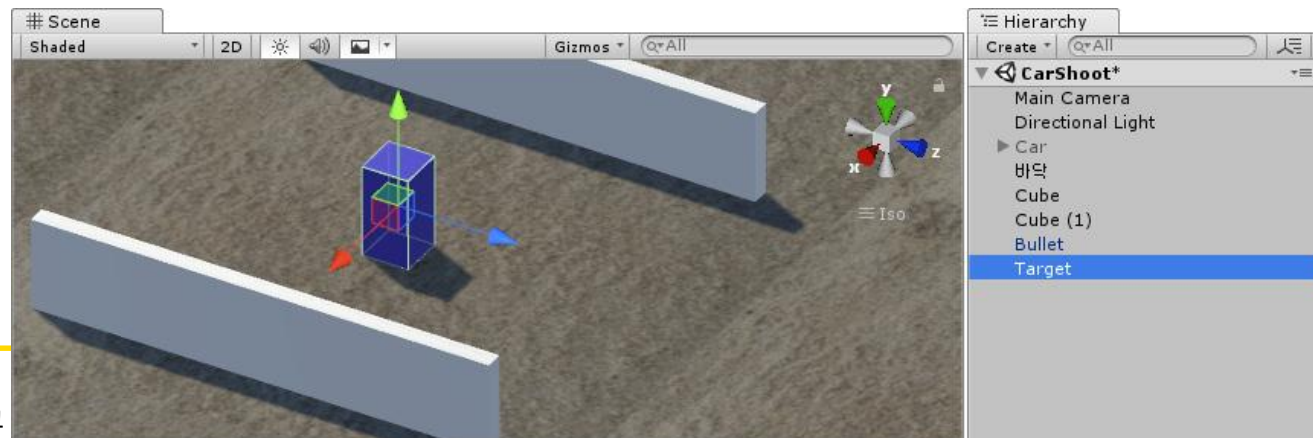
- 씬에 목표물을 몇 개 설치하고, 총알이 목표물에 명중할 때의 처리

(1) Target 만들기

- Target은 인터넷의 적당한 모델을 import 하거나 또는 간단하게 Cube로 만들
- [GameObject - 3D Object - Cube]를 클릭하여 Target을 추가

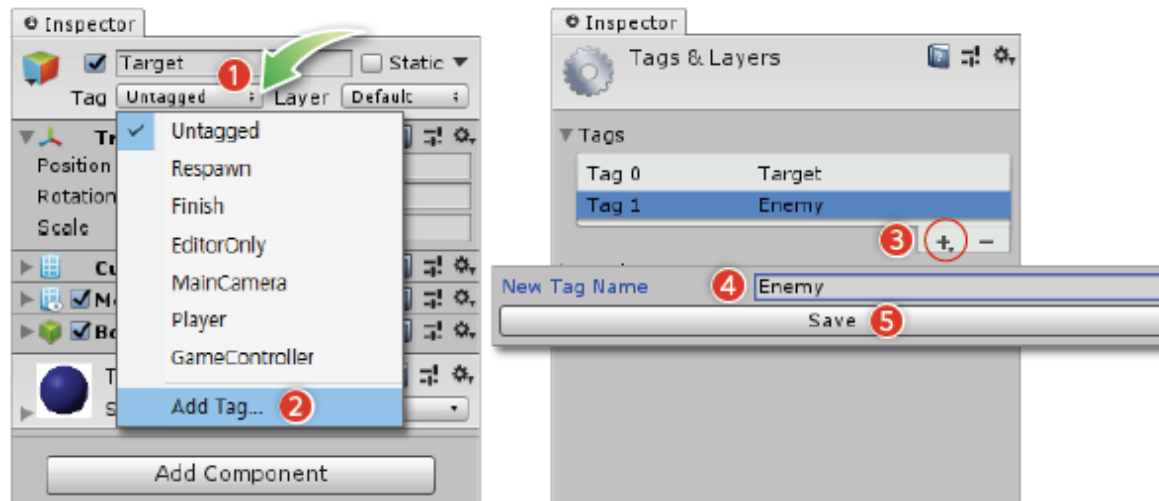
Object	Name	Scale	Position	Scale	Material
Cube	Target	1, 2, 1	0, 1, 0	1, 1, 1	적당한 색

- Target을 프로젝트 브라우저의 Prefabs 폴더로 드래그하여 프리팹으로 만들
- 프리팹용 오브젝트는 동적으로 호출할 때 좌표 계산을 용이하게 하기 위해, 씬의 기준점에 만드는 것이 편리

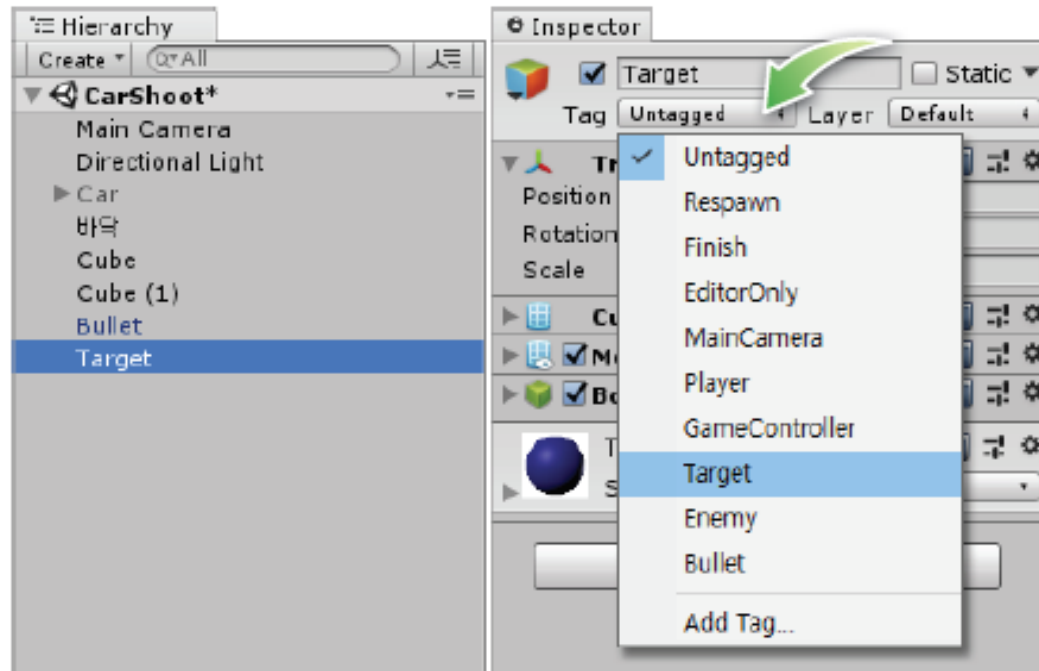


(2) Target 에 Tag 달기

- Tag는 오브젝트의 종류를 구분하기 위해 사용하는 별칭
- 인스펙터의 [Tag] 버튼을 클릭하면 Tag 입력란에서 생성
- Target, Enemy, Bullet를 추가함

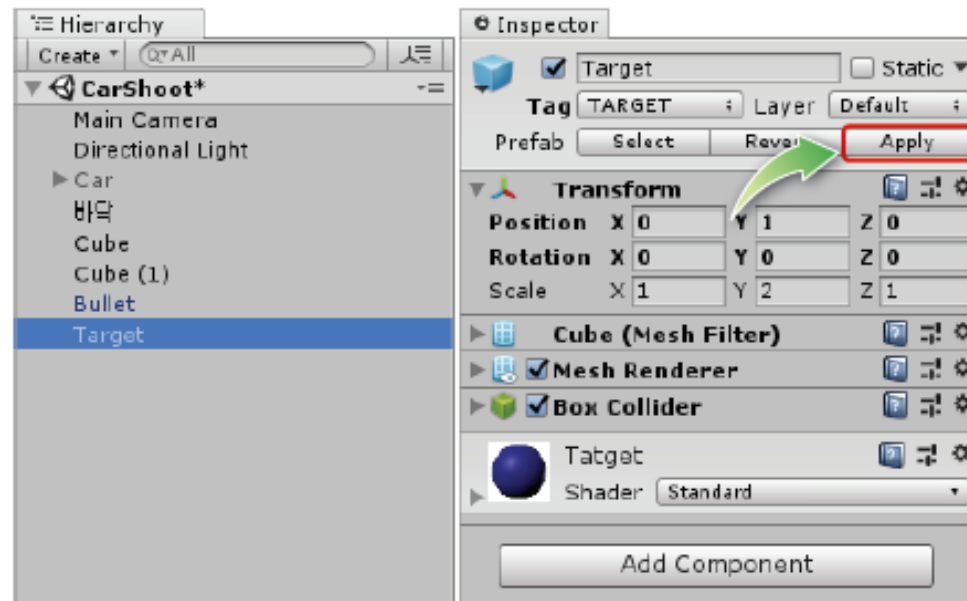


- 씬에 있는 Target을 선택하고 인스펙터의 [Tag] 버튼의 부메뉴에서 지정



(3) Prefab 복사본의 Apply

- 오브젝트가 프리팹으로 등록되면, **프로젝트 뷰의 프리팹이 원본이고, 씬의 오브젝트는 프리팹의 복사본**이 됨.
- 프리팹은 원본을 변경하면, 씬의 복사본도 함께 변경됨. **복사본은 속성을 변경하더라도 원본에는 영향을 주지 않음.**
- Target의 Tag는 복사본에 설정한 것이고 원본은 아직 Tag가 설정되지 않은 상태이므로, 복사본의 속성을 모두 원본에 적용하기 위해서는 인스펙터의 [Apply] 버튼을 클릭함



4.8.3 충돌 판정

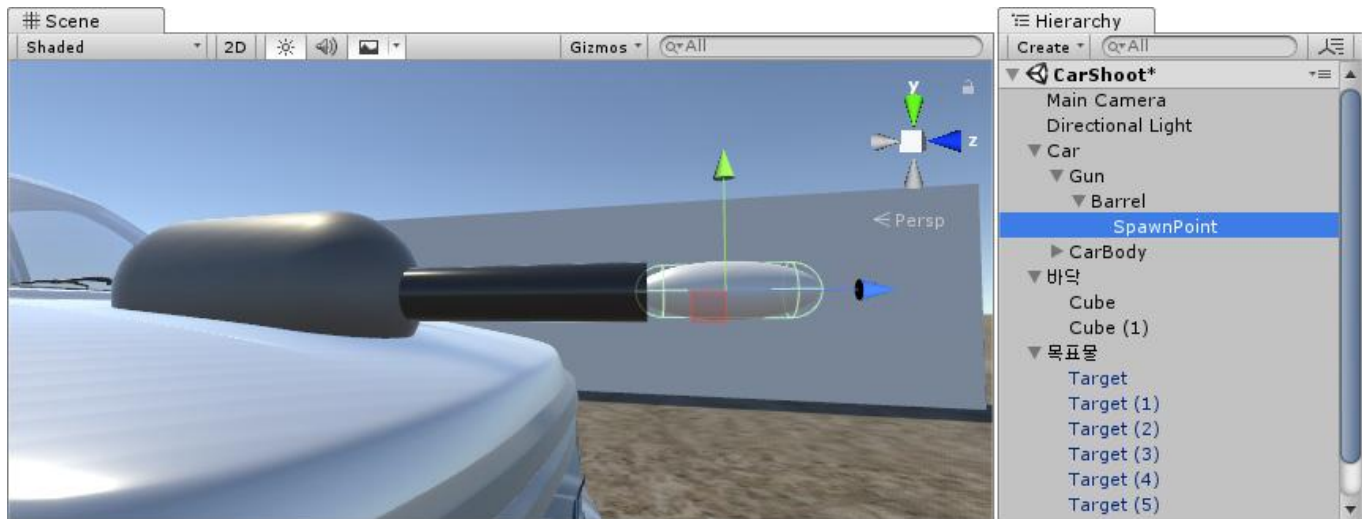
- Bullet 스크립트에 다음 부분을 추가

```
public class Bullet: MonoBehaviour {
    ....
    // 충돌 판정과 처리
    void OnTriggerEnter(Collider other)
    {
        Debug.Log(other.name + " " + other.tag);
        if (other.tag == "Target") {
            Destroy(other.gameObject);
        }
        Destroy(gameObject);
    }
}
```



스크립트에 `void OnCollisionEnter(Collider other) { }` 를 추가하고 Is Trigger 옵션을 ON으로 할 때와 OFF로 할 때를 비교

- 총알이 Target에 명중해도 Target이 제거되지 않을 수 있음. 이 경우 상태표시줄에 "Barrel Untagged"가 출력되는데, 총알이 발사될 때 총구와 충돌이 발생하는 현상임
- 총알의 중심부가 SpawnPoint에 위치하므로, 이를 고려하여 총구에서 조금 떨어진 곳에 스파운포인트를 배치해야 함



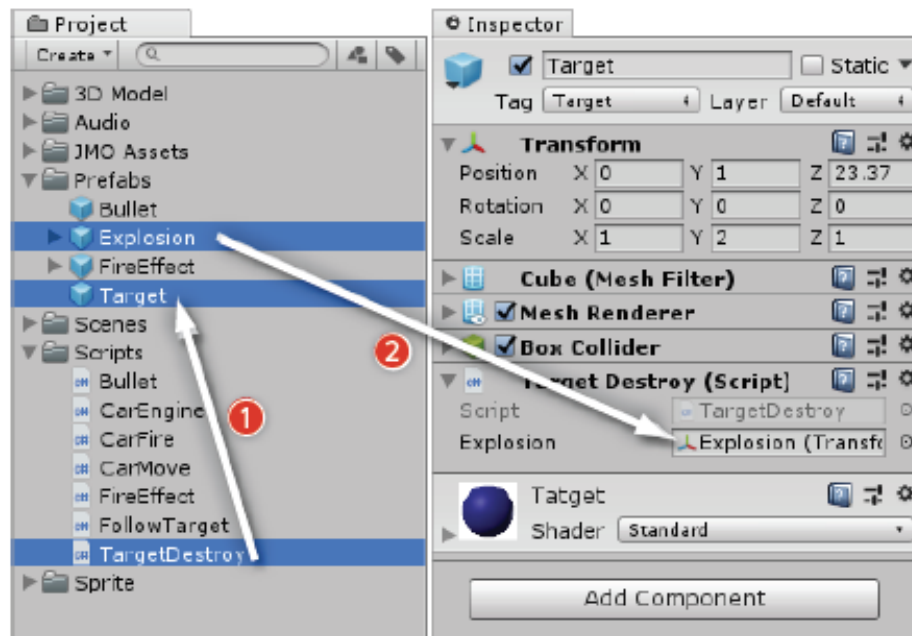
4.8.4 충돌 처리

- 충돌 이벤트는 Rigidbody가 있는 오브젝트에서만 발생하므로, 벽이나 Target 은 충돌 이벤트가 발생하지 않음
- 모든 충돌이 처리를 총알에서만 하게 되면, 장애물의 종류가 많아질수록 처리과정이 복잡해질 수 있음 → 총알은 충돌 판정만 하고, 세부적인 처리는 피격당하는 오브젝트에서 하는 것이 보다 효율적임
- Target의 스크립트
 - TargetDestroy 스크립트를 작성하고 프리팹으로 만든 Target로 연결 (프리팹을 변경하므로, 씬의 장애물도 함께 변경됨)

```
public class TargetDestroy : MonoBehaviour
{
    public Transform explosion; // 폭파 파티클

    .....
    void DestroySelf(Vector3 pos)
    {
        //Instantiate(explosion, pos, Quaternion.identity);
        Destroy(gameObject);
    }
}
```

- 플레이어가 게임 화면에서 보는 나뭇잎이나 잡초는 실제로 2D 이미지임.
- 2D 이미지는 방향에 따라 보이지 않는 문제가 있으므로, 게임 엔진은 이들을 카메라 방향으로 회전시켜 표시함.
- 이러한 기능을 Billboard라 하며 파티클도 빌보드로 처리되므로, 파티클 프리팹을 표시할 때 회전각을 지정할 필요는 없음



- Bullet의 스크립트

- Bullet 스크립트의 충돌 판정 부분을 수정

```
public class Bullet : MonoBehaviour
{
    .....

    // 충돌의 판정과 처리
    void OnTriggerEnter(Collider other) {
        // Destroy(other.gameObject)
        other.SendMessage("DestroySelf", transform.position)
    }
    Destroy(gameObject);
}
```

- SendMessage()는 다른 스크립트의 함수를 호출
 - 호출하는 함수명은 문자열로 지정하고, 호출되는 함수는 public 으로 선언되지 않아도 됨.
 - 함수 호출에 특별한 절차가 필요하지 않으므로, 간단하게 사용할 수 있지만 함수명을 잘못 기술해도 컴파일 오류가 발생하지 않으므로 주의해서 사용

- SendMessage()는 매개변수를 하나만 전달할 수 있으므로, 2개 이상의 매개변수가 필요한 함수는 호출하지 못함
- 이럴 경우 호출되는 함수를 public 으로 만들고 다음과 같이 호출
`Other.GetComponent<스크립트명>().함수명(매개변수들);`



게임을 실행하여 Target가 피격될 때 파티클이 나타나면서 파괴되는 것을 확인



- 투명하게 사라지기
 - Target에 비해 폭발 불꽃이 너무 크고 불꽃이 나오는 순간 오브젝트가 사라짐
 - Target이 점점 투명해지면서 사라지는 기능을 추가
 - » 프리팹 Explosion의 scale을 (0.5, 0.5, 0.5)로 설정해서 크기를 줄임
 - » Target에서 사용한 매터리얼의 Rendering Mode를 Transparent로 설정해서 투명색이 나타나도록 수정
 - » TargetDestroy 스크립트 수정

```

public class TargetDestroy : MonoBehaviour
{
    public Transform explosion; // 폭발 파티클

    .....
    void DestroySelf(Vector3 pos)
    {
        //Instantiate(explosion, pos, Quaternion.identity);
        // Destroy(gameObject);
        StartCoroutine(DestoryLazy());
    }

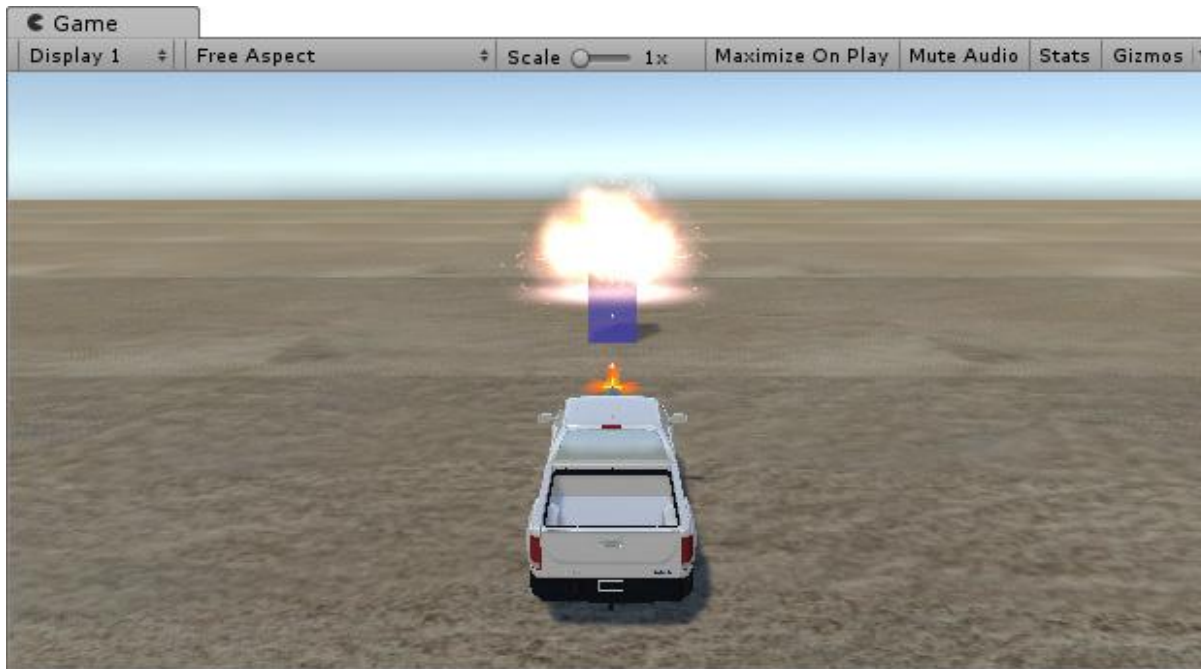
    // 투명하게 사라지기
    IEnumerator DestroyLazy() {
        // 오브젝트 매터리얼 읽기
        Material mat = GetComponent<Renderer>().material;
        Color color = mat.color;

        // 투명도 설정
        for (float alpha = 1; alpha >= 0; alpha -= 0.02f) {
            color.a = alpha;
            mat.color = color;
            yield return null;
        }
        Destroy(gameObject);
    }
}

```



게임을 실행하여, 총알을 맞은 후 점점 투명해지면서 사라지는 객체를 확인



- Target의 폭발 사운드는 Instantiate() 직후에 처리함

4.9 적군 다루기

- 씬에 적군을 만들고 자동차(탱크)가 접근하면 사격하는 부분을 추가

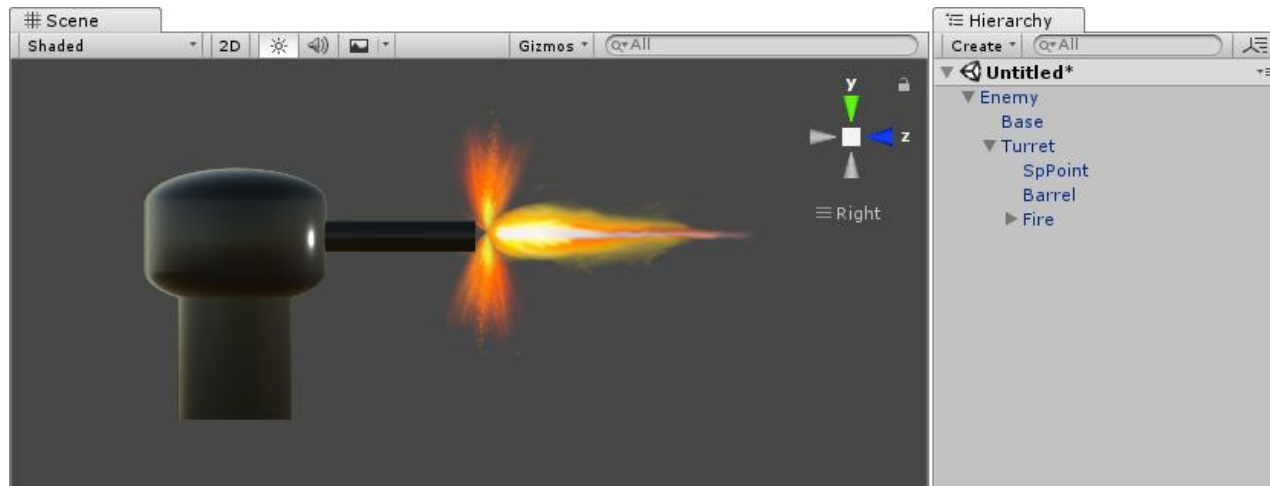
4.9.1 적군 뱅커 만들기

- 적군 뱅커는 기본 오브젝트를 이용해서 단순하게 만들

Object	Name	Scale	Position	Rotation	Tag
Cylinder	Base	(0.5, 0.3, 0.5)	(0, 0, 0)	(0, 0, 0)	Enemy
Capsule	Turret	(0.8, 0.3, 0.8)	(0, 0.5 0)	(0, 0, 0)	Enemy
Cylinder	Barrel	(0.14, 0.35, 0.14)	(0, 0.5 0.7)	(90, 0, 0)	Enemy
Empty	SpPoint	(1, 1, 1)	(0, 0.5 1.4)	(0, 0, 0)	
Empty	Enemy	(1, 1, 1)	(0, 0, 0)	(0, 0, 0)	

- 새로운 씬에서 만드는 것이 편리 (나중에 필요에 따라 복사해 옴)

- 오브젝트를 만든 후 다음의 과정이 필요
 - 각 오브젝트를 적당한 색으로 매핑
 - Barrel과 SpPoint를 Turret의 Child로 만들고, Base와 Turret은 Enemy의 Child로 설정
 - Enemy의 Position을 (0,0,0)으로 해서 Pivot를 (0,0,0)으로 설정
 - 포신 앞에 프리팹 FireEffect를 추가하고 Turret의 Child로 만든 다음 이를 Fire로 설정
 - Enemy에 AudioSource를 추가하고 연발 사격 사운드 할당(Play On Awake 옵션 제거)
 - Enemy를 프리팹으로 만들기



4.9.2 적군 제어용 스크립트

- Enemy 스크립트를 만들고 Enemy 오브젝트에 할당
- (1) 변수 추가
 - 적군 제어에 필요한 변수를 추가
 - 프리팹을 처리하기 위해 public 변수가 2개 있으므로, 인스펙터에서 값을 할당

```
public class Enemy : MonoBehaviour
{
    public Transform bullet;    // 총알
    public Transform explosion; // 폭발 불꽃

    Transform car;             // 자동차
    Transform turret;          // 포탑
    Transform spPoint;         // 스파인포인트
    Transform fire;            // 발사 불꽃
    AudioSource gunSound;      // 발사 사운드

    const float RADAR_DIST = 12f; // 자동차 탐지 거리
    const float FIRE_DIST = 10f;  // 사정거리

    bool canFire = true;

    // Start is called before the first frame update
    void Start() { }

    // Update is called once per frame
    void Update() { }
}
```


(2) 게임 초기화

– 변수에 오브젝트를 할당

```
public class Enemy : MonoBehaviour
{
    // Start is called before the first frame update
    void Start() { InitGame(); }

    // Update is called once per frame
    void Update() { }

    void InitGame()
    {
        // 자동차와 포탑
        car = GameObject.Find("Car").transform;
        turret = transform.Find("Turret");
        spPoint = transform.Find("Turret/SpPoint");

        // 발사 불꽃
        fire = transform.Find("Turret/Fire");
        fire.gameObject.SetActive(false);

        // Sound 처리
        gunSound = GetComponent();
    }
}
```

4.9.3 적군의 공격

- 자동차가 병커의 탐지 범위에 들어오면 포탑을 자동차 방향으로 회전시키고 사격 범위로 들어오면 연발 사격을 처리

```
public class Enemy : MonoBehaviour
{
    // Start is called before the first frame update
    void Start() { InitGame(); }

    // Update is called once per frame
    void Update() {
        // 자동차와의 거리 구하기
        float dist = Vector3.Distance(car.position, transform.position); // 두 지점의 직선거리를 구하기
        // 레이더 탐지 거리 이내인지 검사
        if (dist <= RADAR_DIST) { turret.LookAt(car); } // 포탑을 자동차 거리로 회전
        // 사정거리 이내인지 검사
        if (dist <= FIRE_DIST && canFire) { StartCoroutine(AutoFire()); }
        if (dist > FIRE_DIST) { gunSound.Stop(); }
    }
    // 연발사격
    IEnumerator AutoFire() {
        Instantiate(bullet, spPoint.position, spPoint.rotation);
        fire.gameObject.SetActive(true);
        gunSound.Play();
        canFire = false;
        yield return new WaitForSeconds(0.2f);
        canFire = true;
    }
    void InitGame() { ... }
}
```



씬에 포탑을 여러 개 설치한 후, 게임을 실행

- 자동차가 포탑의 탐지 거리 이내로 접근하면 포탑이 자동차 방향으로 회전하고 사정 거리로 접근하면 연발 사격하는 것을 확인



- LookAt() 함수의 회전이 순식간에 이뤄지므로, 포탑 회전이 보여지지 않을 수 있음
- 포탑을 부드럽게(천천히) 회전시키려면, 선형보간이나 구면보간을 이용해서 회전하도록 수정

```
public class Enemy : MonoBehaviour
{
    // Start is called before the first frame update
    void Start() { InitGame(); }

    // Update is called once per frame
    void Update() {

        .....

        // 레이더 탐지 거리 이내인지 검사
        if (dist <= RADAR_DIST) {
            // turret.LookAt(car);
            Vector3 delta = car.position - transform.position;
            Quaternion rot = Quaternion.LookRotation(delta);
            turret.rotation = Quaternion.Slerp(turret.rotation, rot, 5*Time.deltaTime);
        }

    }
}
```



게임을 실행하여, 포탑이 부드럽게 회전하는지를 확인

- Update() 함수를 보면, car.position, transform.position이 연달아 나옴. 중복되는 부분이 나타나지 않도록 수정

```
public class Enemy : MonoBehaviour
{
    // Start is called before the first frame update
    void Start() { InitGame(); }

    // Update is called once per frame
    void Update() {

        .....

        // 자동차와의 거리 계산
        Vector3 delta = car.position - transform.position; // 두 지점의 거리를 Vector로 구함
        float dist = delta.magnitude; // Vector의 길이(직선거리)를 구함

        if (dist <= RADAR_DIST) {
            Quaternion rot = Quaternion.LookRotation(delta);
            turret.rotation = Quaternion.Slerp(turret.rotation, rot, 5*Time.deltaTime);
        }

    }
}
```

4.9.4 적군 파괴

- Enemy에 자신을 파괴하는 함수를 작성
- TargetDestroy() 함수와 유사하지만 벙커가 3부분으로 되어 있으므로 각 부분을 투명하게 만드는 처리를 추가

```
public class TargetDestroy : MonoBehaviour
{
    .....
    // 오브젝트 제거 - 외부호출
    void DestroySelf(Vector3 pos)
    {
        Instantiate(explosion, transform.position, Quaternion.identity);
        StartCoroutine(DestoryLazy());
    }

    // 투명하게 사라지기
    IEnumerator DestoryLazy() {
        // 오브젝트 매터리얼 읽기
        Material mat1 = turret.GetComponent<Renderer>().material;
        Material mat2 = transform.Find("Base").GetComponent<Renderer>().material;
        Material mat3 = transform.Find("Turret/Barrel").GetComponent<Renderer>().material;
        Color color = mat1.color;

        // 투명도 설정
        for (float alpha = 1; alpha >= 0; alpha -= 0.02f) {
            color.a = alpha;
            mat1.color = color;
            mat2.color = color;
            mat3.color = color;
            yield return null;
        }
        Destroy(gameObject);
    }
}
```

- Bullet 충돌 처리 부분을 수정
- 벙커는 3부분으로 되어 있으므로 총알이 어느 부분을 명중하더라도 최상위 개체 (Root)인 Enemy의 스크립트를 실행해야 함

```
public class Bullet : MonoBehaviour
{
    void OnTriggerEnter(Collider other) {
        ....
        // Enemy
        if (other.tag == "Enemy") {
            other.transform.root.SendMessage("DestroySelf", transform.position);
        }
        Destroy(gameObject);
    }
}
```

4.10 주인공 파괴와 **SCENE RELOAD**

- 자동차가 여러 방의 총알을 맞아서 파괴되도록 수정
- HP(체력)의 초기값을 설정해두고, 총알을 맞을 때마다 HP를 감소시킴
- HP가 0이 되면 Game Over
 - 자동차는 Rigidbody가 있어 충돌 판정을 할 수 있으므로, CarFire 스크립트에 충돌을 판정하는 함수를 추가
 - 해당 함수에서 Bullet의 Tag를 조사하고 있으므로, Bullet에 Tag를 설정하도록 수정


```

Using UnityEngine.SceneManagement; // SceneManager을 사용

public class CarFire : MonoBehaviour
{
    ....
    int hp = 10;

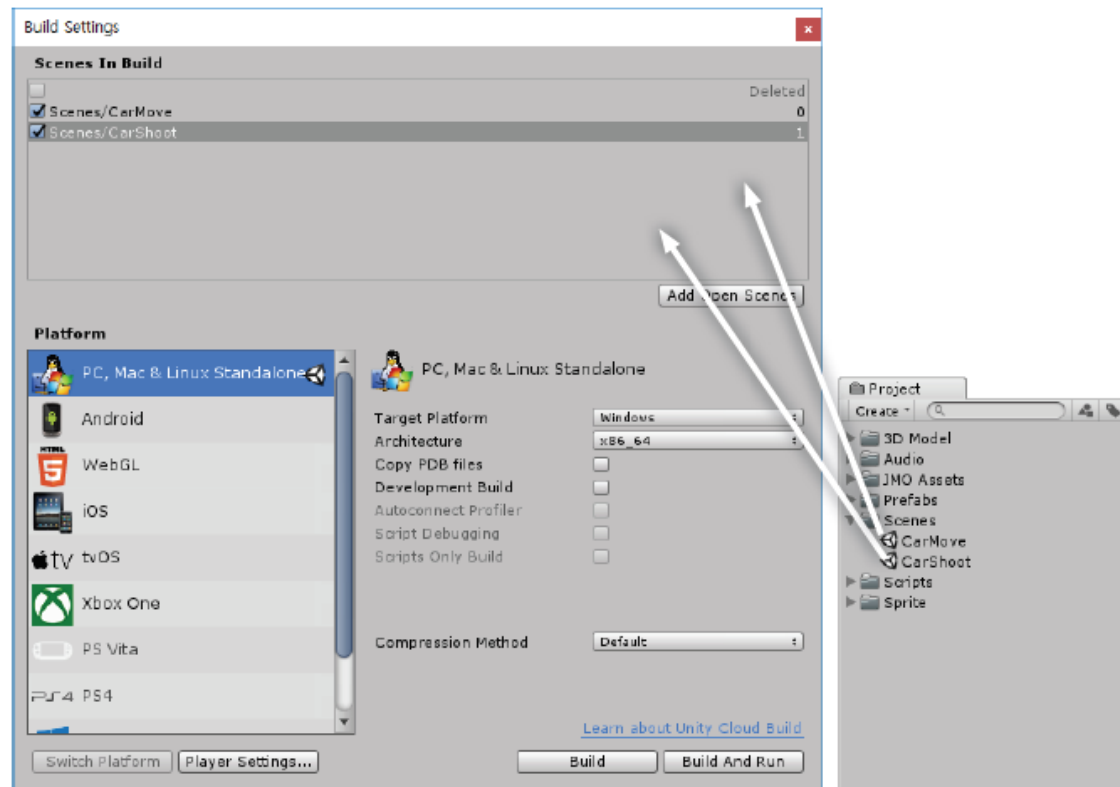
    // 충돌 판정 및 처리
    void OnTriggerEnter(Collider other) {
        if (other.tag == "Bullet") {
            hp--;
            if (hp < 0) { StartCoroutine(DestroySelf());
        }
    }

    // Reset game
    IEnumerator DestroySelf() {
        Instantiate(explosion, transform.position, Quaternion.identity);
        yield return new WaitForSeconds(2f);

        // 현재 실행중인 씬을 다시 불러온다. 씬의 오브젝트가 모두 초기화됨
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }
}

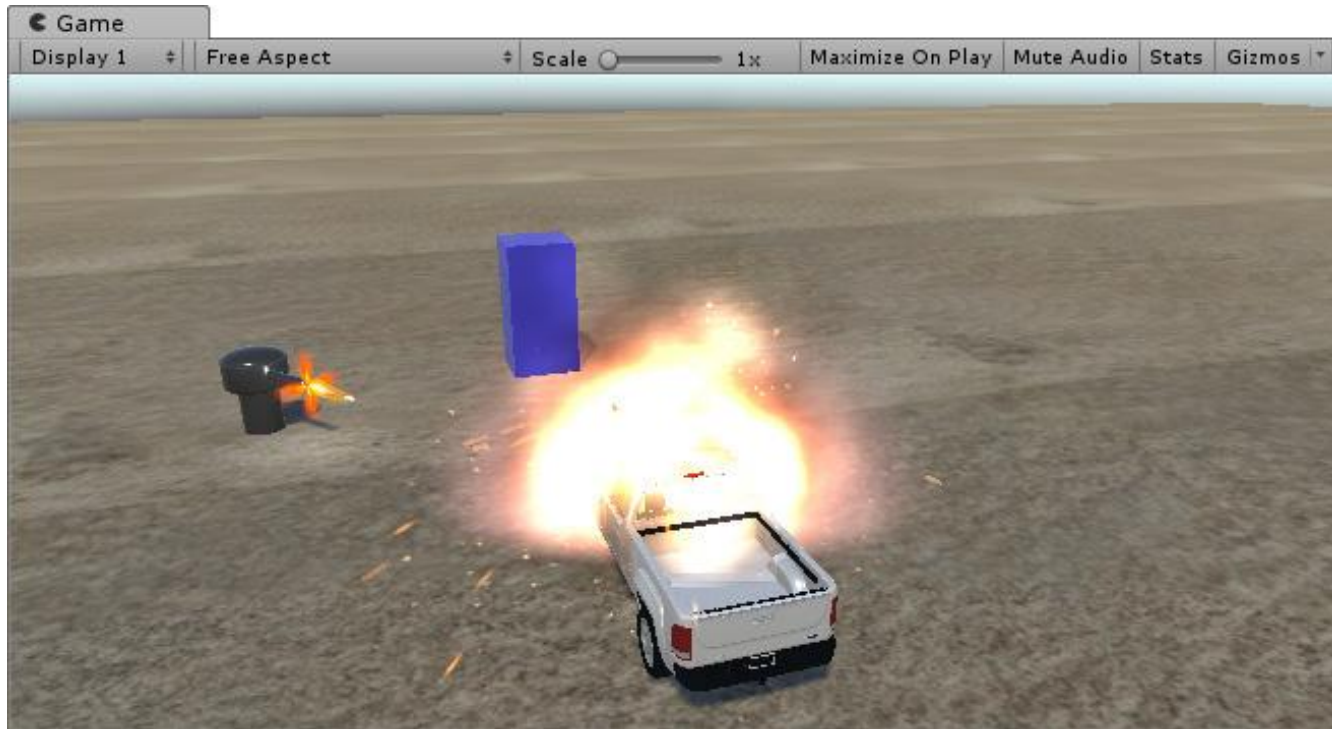
```

- SceneManager는 LoadScene("씬이름") 또는 LoadScene(씬번호)와 같이 사용
- 사용하려면, [File - Build Settings] 메뉴를 실행해서 씬을 미리 등록





게임을 실행하여, 자동차가 10발 이상 피격될 경우, 폭파 불꽃이 나타나면서 씬이 초기화되는 것을 확인





GAME 기초

Part 4. Supplement



현재까지의 탱크 작업 내용

필요한 경우, Github에서 다운로드

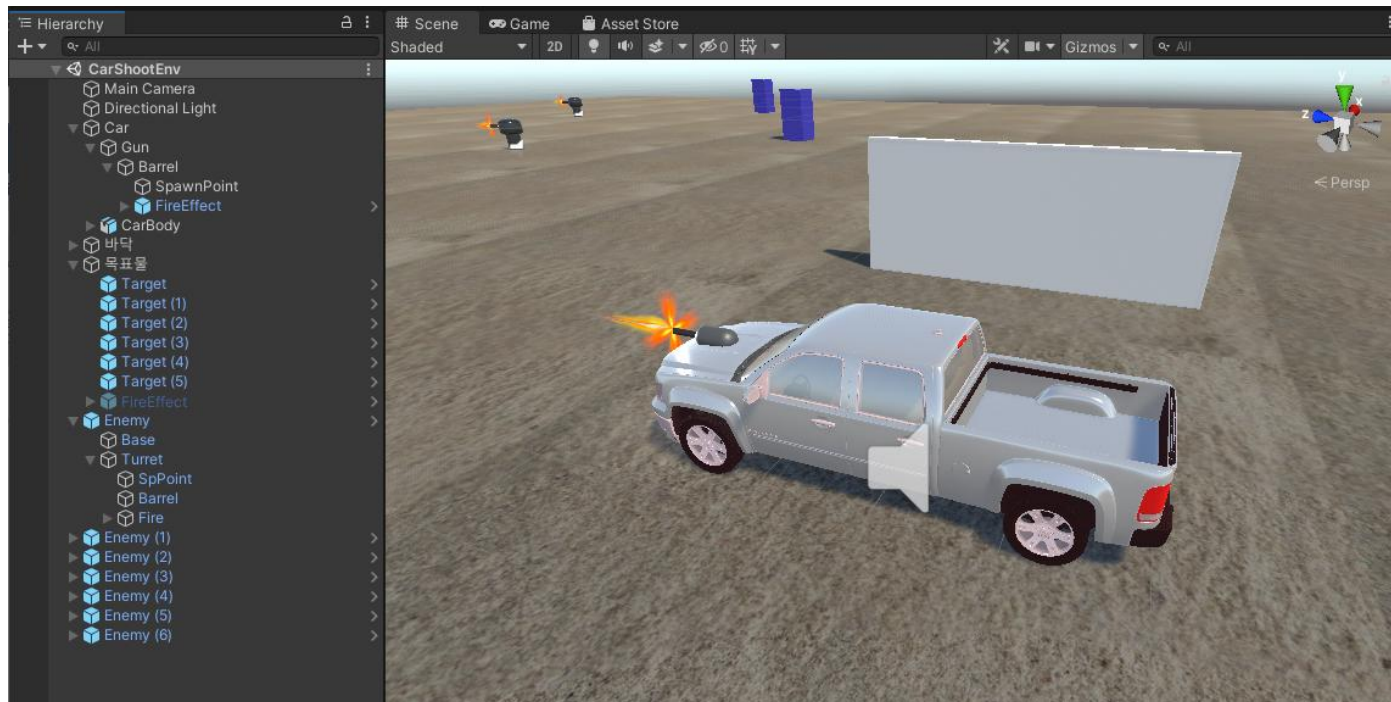


<https://github.com/hwany1458/Unity-SampleTankGame>

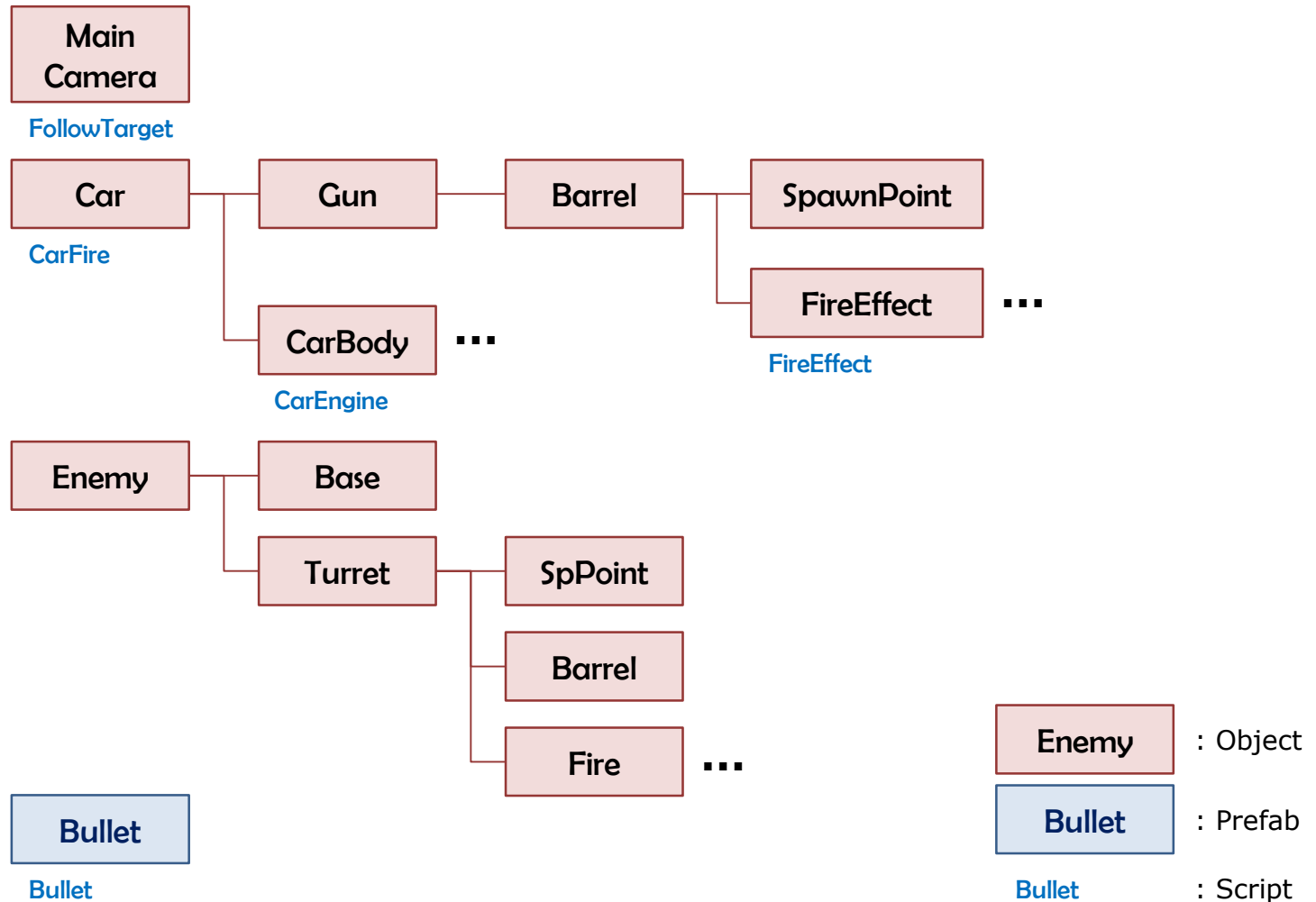
처리 기능	교재 목차	교재 페이지
3D 모델	[4.1] 3D Model 다루기	P.125~128
매터리얼	[4.1.2] Model의 Material	P.128~132
FPS와 Delta Time	[4.2.1]	P.133~134
오브젝트 이동	[4.2.3] 오브젝트 이동	P.135~140
오브젝트 회전	[4.2.6] 오브젝트 회전	P.140~141
슈팅	[4.3] Shooting (단발사격, 연발사격)	P.142~155
사운드 다루기	[4.4] Sound 다루기	P.156~161
물리 처리	[4.5] 오브젝트의 물리 처리	P.161~166
충돌 판정	[4.8] 충돌의 판정과 처리	P.185~189
충돌 처리	[4.8.4] 충돌의 처리	P.189~193
적군 생성 및 공격	[4.9] 적군 다루기	P.193~198
적군 파괴	[4.9.4] 적군 파괴	P.198~199
아군 파괴	[4.10] 주인공 파괴와 Scene Reload	P.199~201
카메라 트래킹	[4.6] Camera Tracking	P.167~172
화면 Zoom In/Out	[4.6.4] Game 화면의 Zoomin/Zoomout	P.173
마우스로 카메라 회전	[4.6.5] Mouse로 Camera 회전	P.174~177
발사 화염	[4.7] 발사 화염과 폭파 불꽃	P.177~182
폭파 불꽃	[4.7.2] Target의 폭파 불꽃	P.183~185



현재의 탱크 게임에 나타난 객체들 간의 구조



연결된 COMPONENTS를 채우세요



추가 작업

- 3D 객체 (자동차, 적군 벙커, 장애물 등) 변경
- 지형 변경 (터레인 추가)
- 총알 발사 스크립트 수정 - `AddForce()` 이용
 - 현재, 발사된 총알의 Z축 값만을 변경하기 때문에, 높낮이가 있는 적군 또는 장애물을 격파할 수가 없음 (총알 위치의 Y축 값보다 작거나 큰 경우 맞추지를 못함)
- HP 값을 화면에 표시 (Canvas의 Text 이용)
- 적군 탱크를 추가하여 자동차가 이동할 경우, 이동하는 자동차 위치를 탐색하고 이동 경로에 따라 적군 탱크가 함께 이동할 수 있도록
 - `Raycast()` 이용
 - Machine Learning 이용
- 다수 개의 씬을 제작하여 스테이지별로 게임을 할 수 있도록
- 게임 시작과 종료 버튼을 추가하여 동작하도록

FINAL RESULT PREVIEW



1. Unity Standard Assets Import
2. Terrain(터레인) 만들기 및 가져오기
3. 총알 발사
4. Canvas를 통한 점수 표시
5. Scene 전환 (값을 넘겨주기)



Unity Asset Store에서 제공하는 Standard Assets을 Import

Standard Assets (for Unity 2017.3) | unity cs0619 guitext is obsolete | GUITexture adn GUIText are ob | +

assetstore.unity.com/packages/essentials/asset-packs/standard-assets-for-unity-2017-3-...

New to Unity? Checkout these top picks for all experience levels

unity Asset Store

Search for assets

Assets Tools Services By Unity Industries

Sell Assets Feedback FAQ

Standard Assets (for Unity 2017.3)

Unity Technologies

★★★★☆ 4 | 1072 Reviews

FREE

유니티에서 열기

♡ 목록에 추가하기 공유하기

License Extension Asset

File size 182.7 MB

Latest version 1.1.5

Latest release date 2018년 3월 6일

Support Unity versions 2017.3.0 or higher

Support Visit site

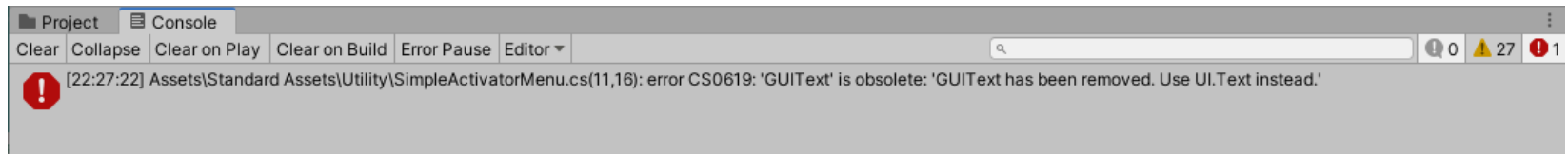
Standard Assets

1/9



Standard Assets에서 오류 수정

- 아래 스크립트에서 파란색 부분을 수정 (SimpleActivatorMenu.cs)



```
using UnityEngine;
using UnityEngine.UI;

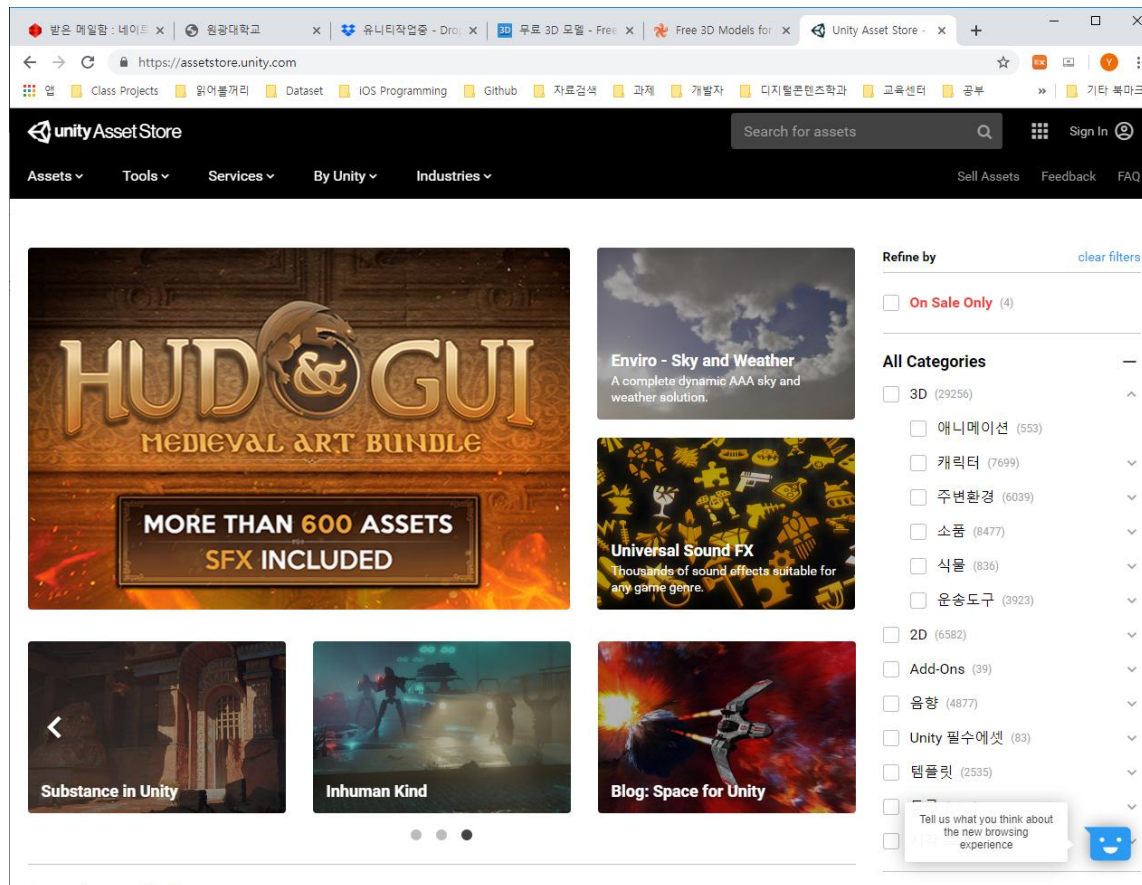
namespace UnityStandardAssets.Utility
{
    public class SimpleActivatorMenu : MonoBehaviour
    {
        ...

        // 아래 줄을 remark
        // public GUIText camSwitchButton;
        public Text camSwitchButton;

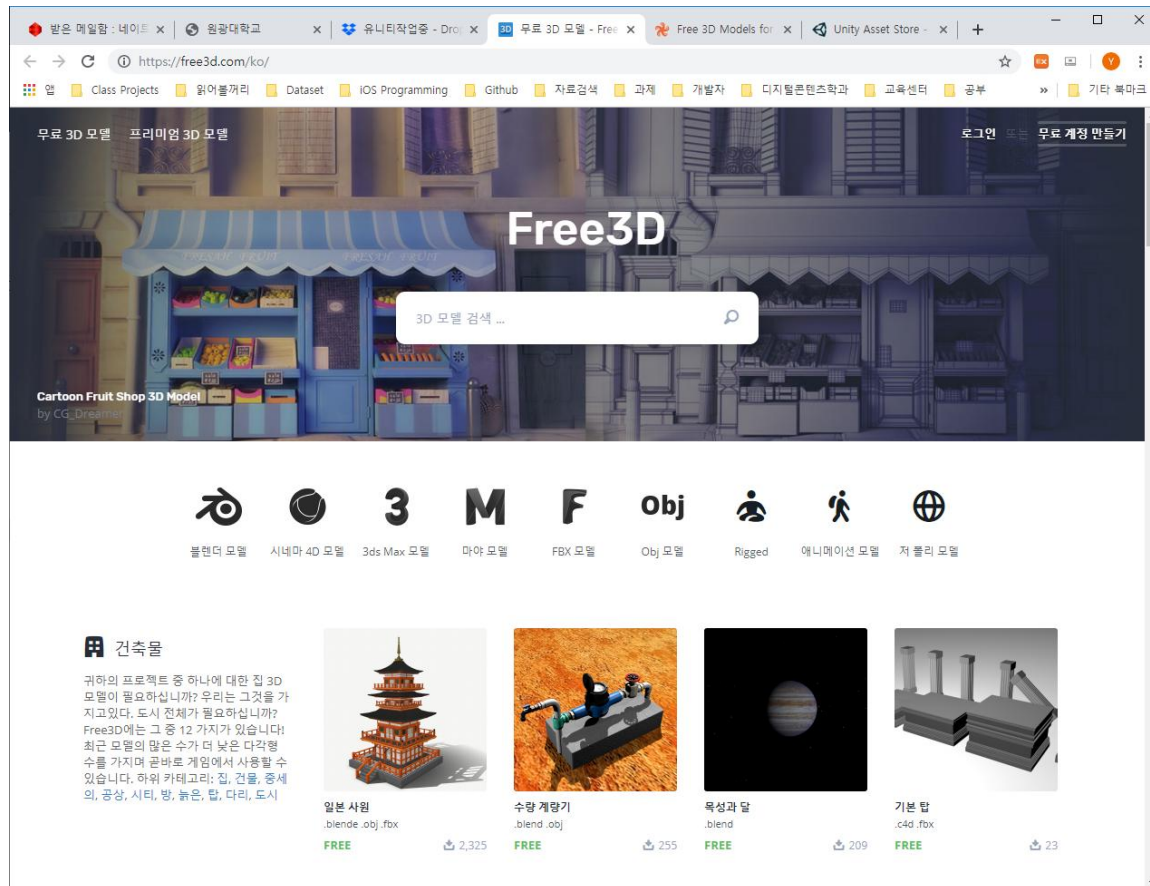
        ...
    }
}
```

3D 모델 제어

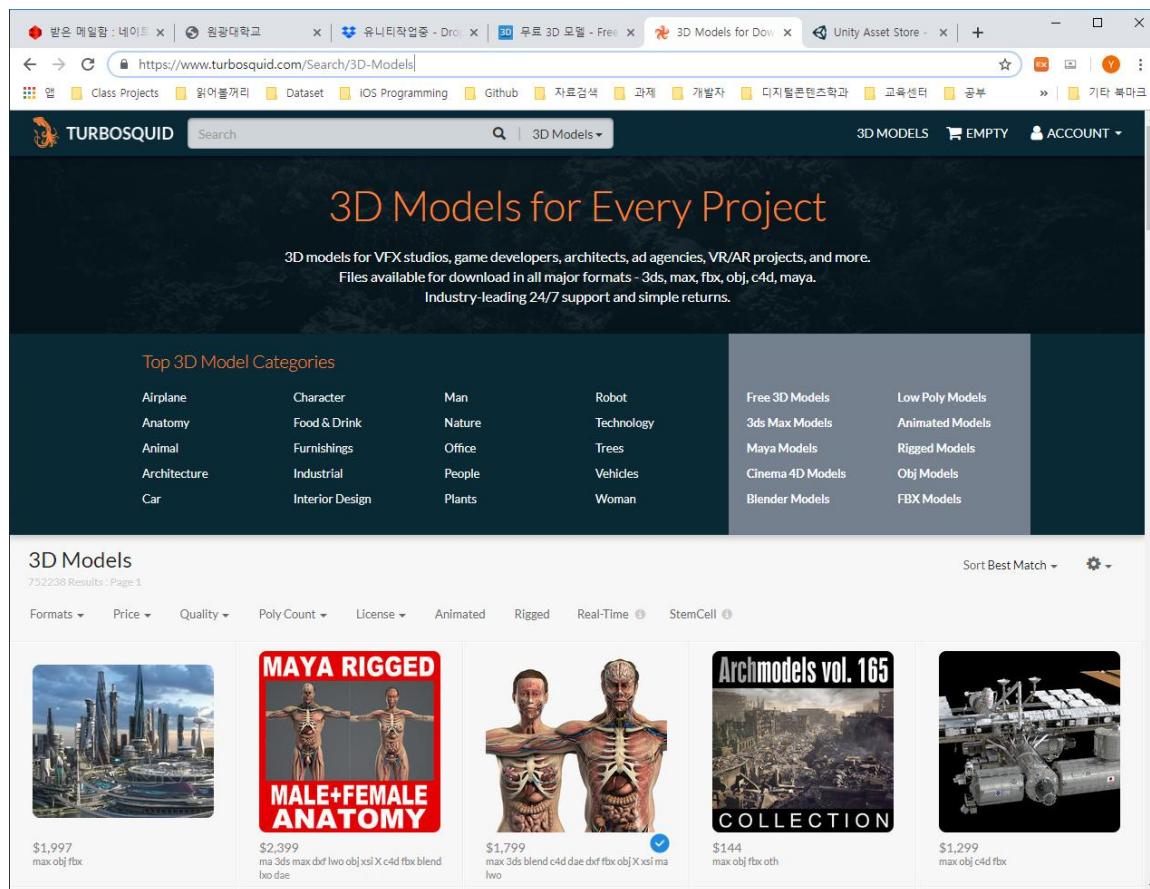
- 외부 저장소에서 제공하는 3D 객체 추가
 - 유니티 애셋스토어 (<https://assetstore.unity.com/>)



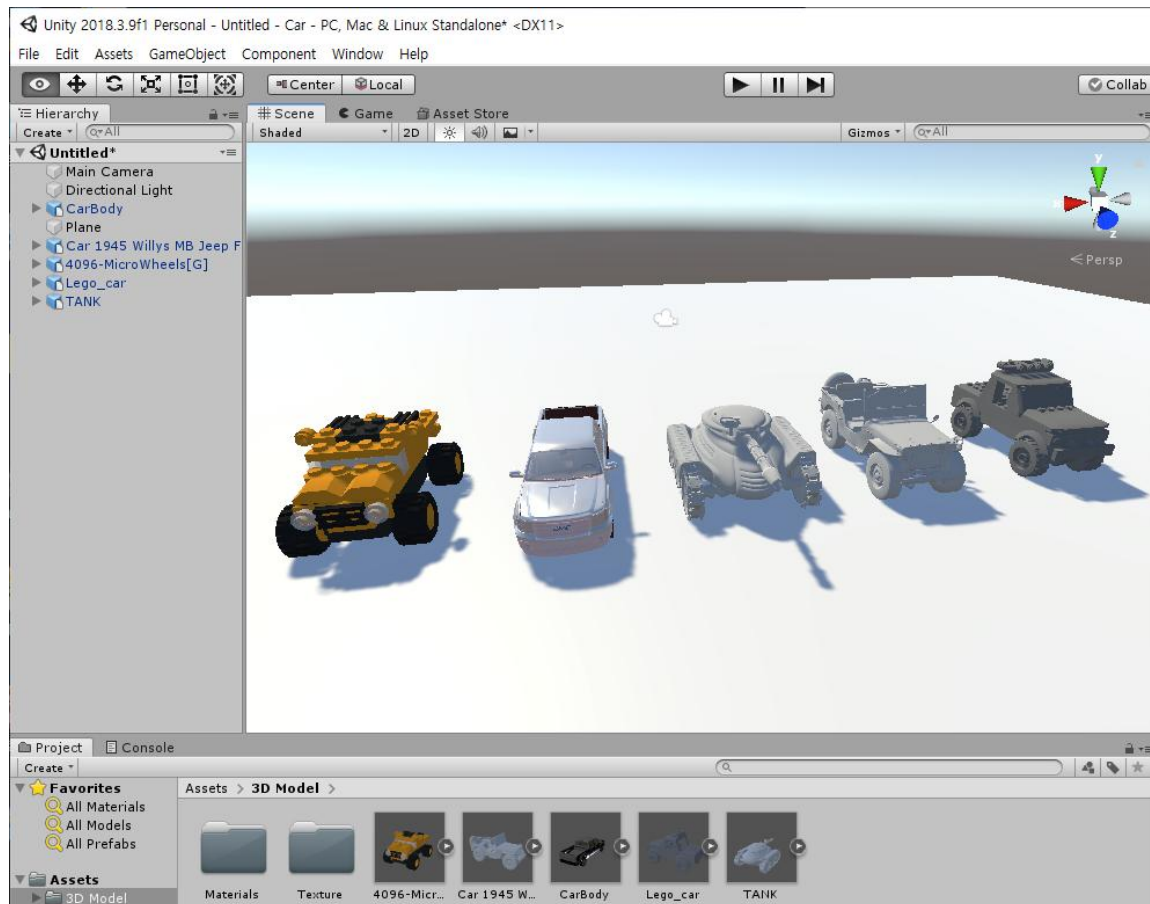
- <https://free3d.com/ko/>



- <https://www.turbosquid.com/Search/3D-Models>



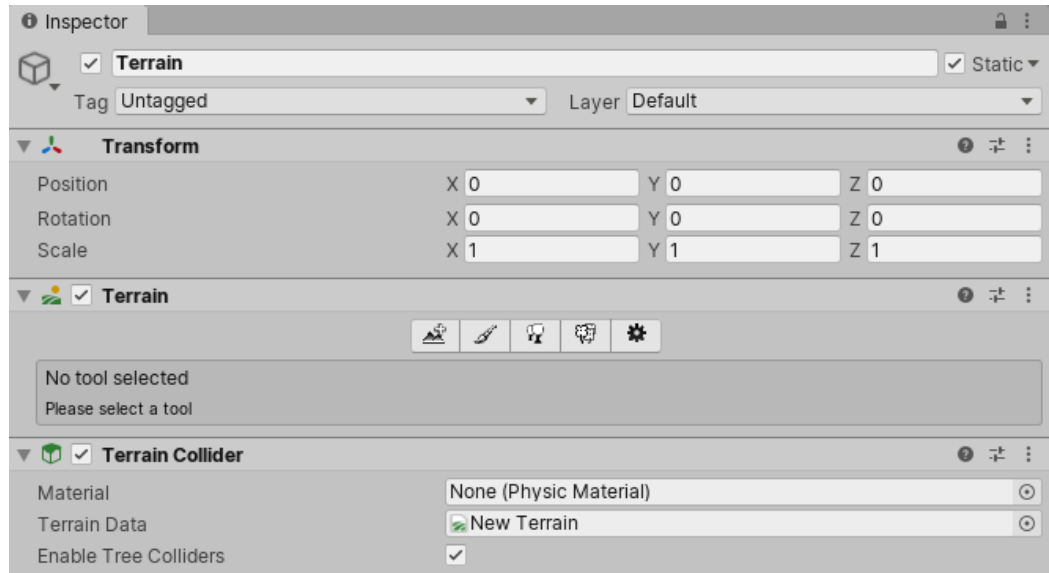
- 아군 탱크, 적군 병커, 장애물 등 다양한 객체를 다운로드 받아, 씬을 구성





Terrain (터레인, 지형) 생성 및 편집

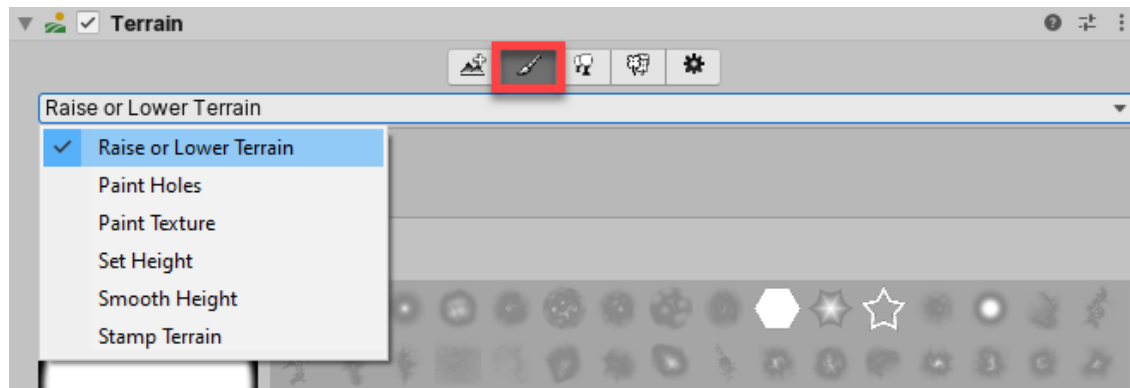
- [File – New Scene]를 클릭하여 새로운 씬을 생성
- 메뉴 [GameObject – 3D Object – Terrain]을 클릭하여 씬에 터레인 게임 오브젝트를 추가





터레인 툴

- 하이러키 창에서 터레인 오브젝트를 클릭하여 인스펙터에 나오는 [Paint Terrain] 아이콘을 클릭하여 터레인 페인팅 툴을 실행

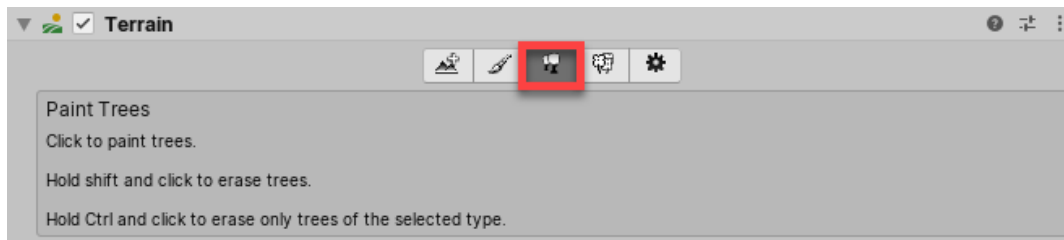


- Raise or Lower Terrain : 페인트브러시 툴로 하이트 맵을 페인팅
- Paint Holes : 터레인의 일부를 숨김
- Paint Texture : 표면 텍스처를 적용
- Set Height : 하이트맵을 특정 값으로 조정
- Smooth Height : 하이트맵을 매끄럽게 만들어 터레인 지형을 부드럽게 만듦
- Stamp Terrain : 현재 하이트맵 위에 브러시모양을 스탬핑



나무

- 하이트맵과 텍스처를 페인팅하는 것과 유사한 방식으로, 터레인에 나무를 페인팅

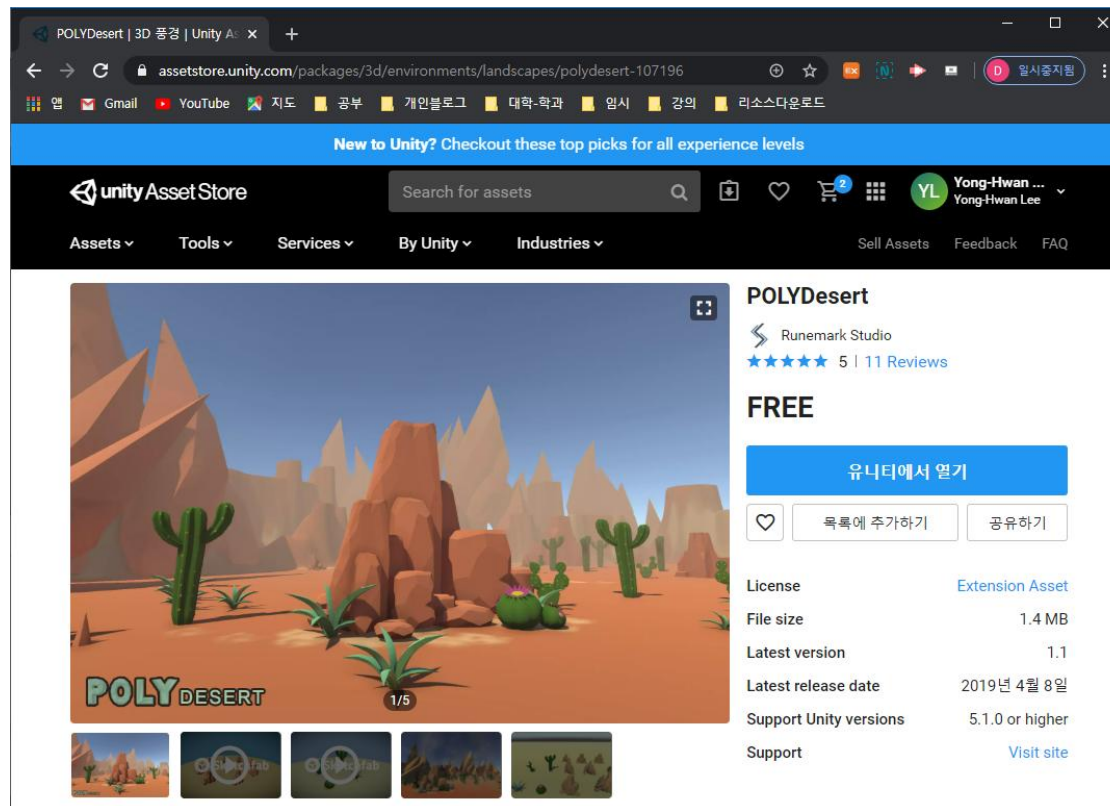


- 나무 프로토타입을 추가하고 브러시를 통해 나무를 만듦
[Edit Trees] 버튼을 클릭한 후, [Add Tree]를 선택하여 나무 에셋을 추가



지형 만들기 (1)

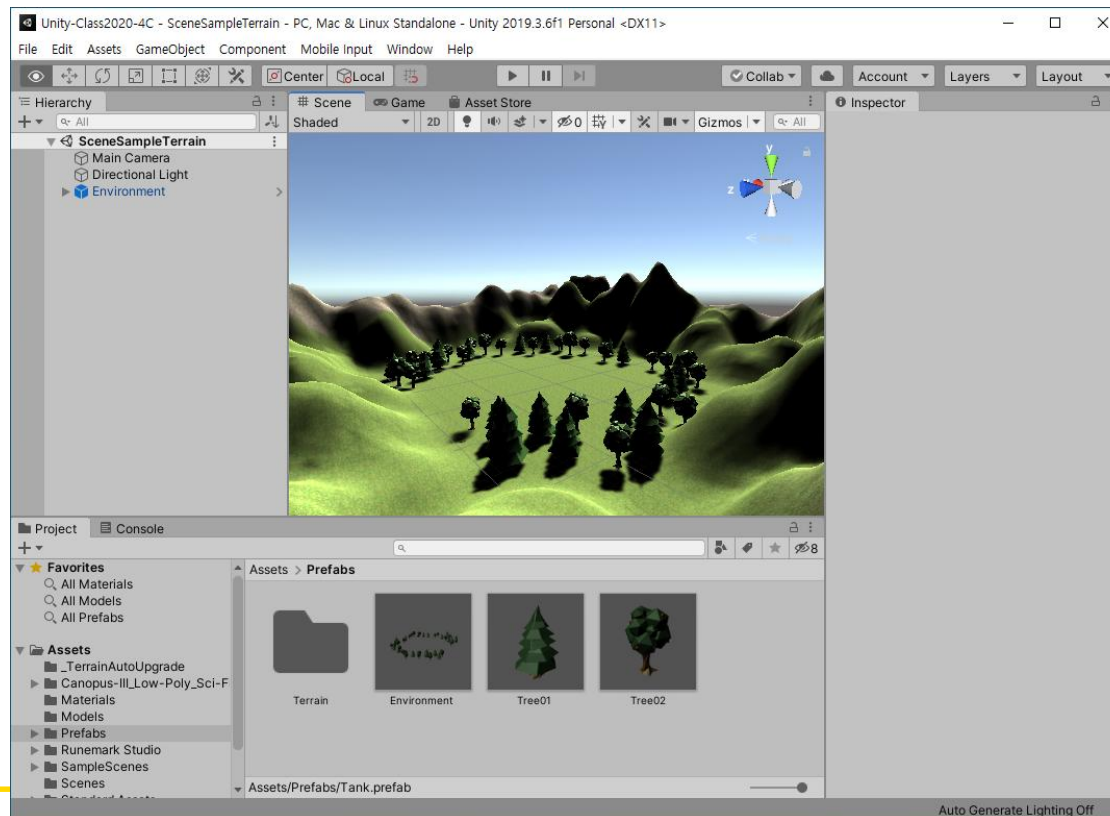
- Asset Store 에서 <poly desert free>로 검색하여 POLYDesert 를 다운로드



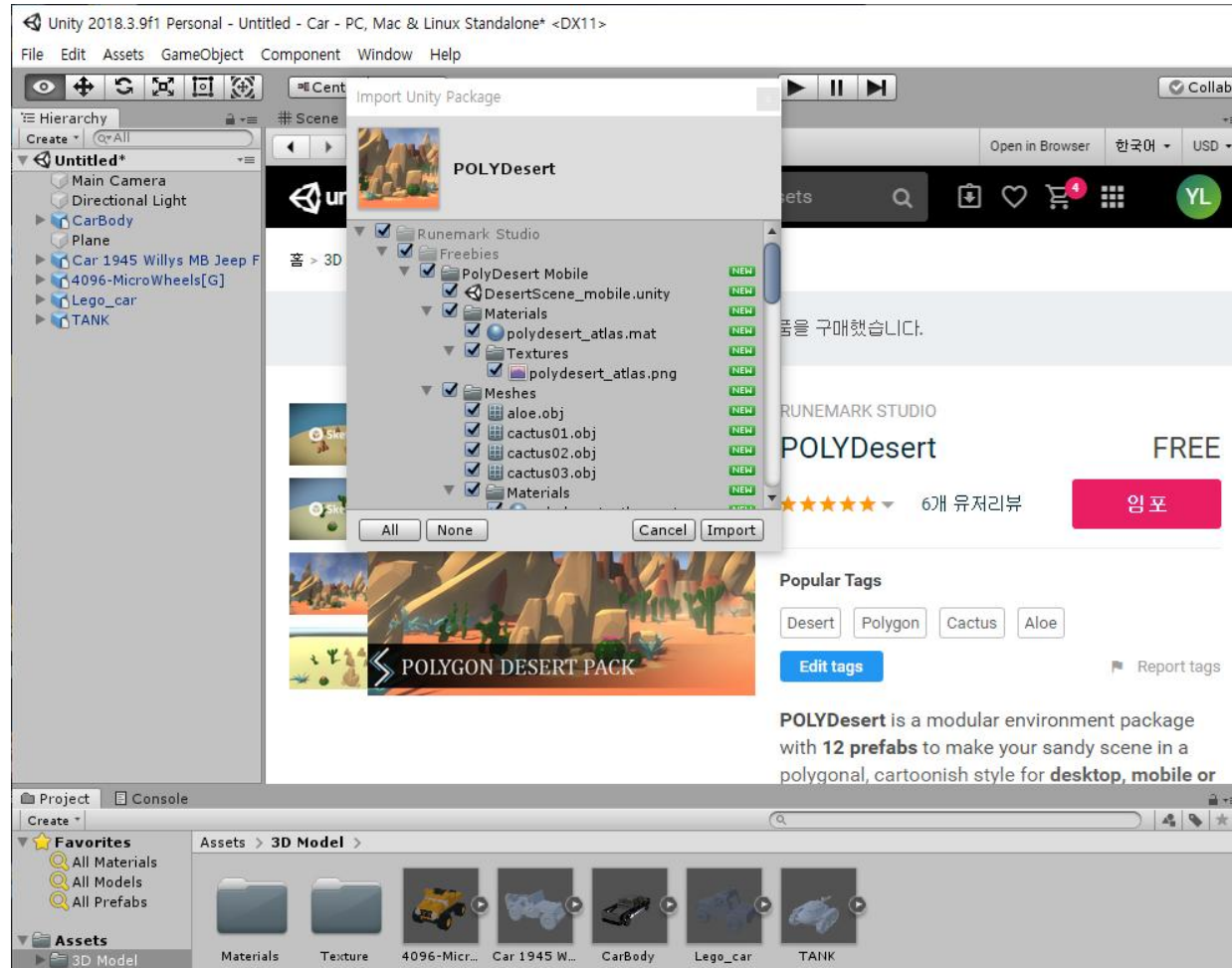


지형 만들기 (2)

- 샘플을 다운로드 받고, [Assets - Import Package]를 클릭하여 패키지를 импорт
- Prefab으로 만들어진 <Environment>를 하이라키 창으로 드래그하여 지형을 가져옴



- 애셋스토어에서 <주변환경>을 import



– Asset Store <low poly desert free>

The screenshot shows the Unity Asset Store interface. The main header includes the Unity logo, 'Asset Store', a search bar, and navigation links like 'Assets', 'Tools', 'Services', 'By Unity', and 'Industries'. The user's profile 'Yong-Hwan Lee' is visible in the top right.

The main content area displays the 'Canopus-III Low Poly Sci-fi Desert Units Set' by Planesfar Studio. The asset is marked as 'FREE'. A large preview image shows a desert environment with various sci-fi units and structures. Below the main image are several smaller thumbnail images.

On the right side, there is a section for reviews. A review from 'fajuto' dated 6 months ago is visible, stating: 'Canopus-III Sci-fi Desert Units Very easy to use. I didn't know what alreat service Canopus-III Sci-fi Desert I didn't know what alreat service. We're loving it!'. There are also buttons for 'Add to favorites', 'Add to wishlist', and 'Share'.

Below the reviews, there is a table with asset details:

License	Extension Asset
File size	41.0 MB
Latest version	1.0
Latest release date	2018년 1월 6일
Support Unity versions	5.2.0 or higher

At the bottom, there is a 'Description' tab and a 'Package Content' section. The description states: 'This low-polygon set contains sci-fi units, structures and arid / desert environment models, which include: ~ 6 Unit models ~ 23 Structure models ~ 20 Environment models'. The 'Package Content' section lists 'Canopus-III Sci-fi Desert Units' and 'Sci-fi Corridors'.

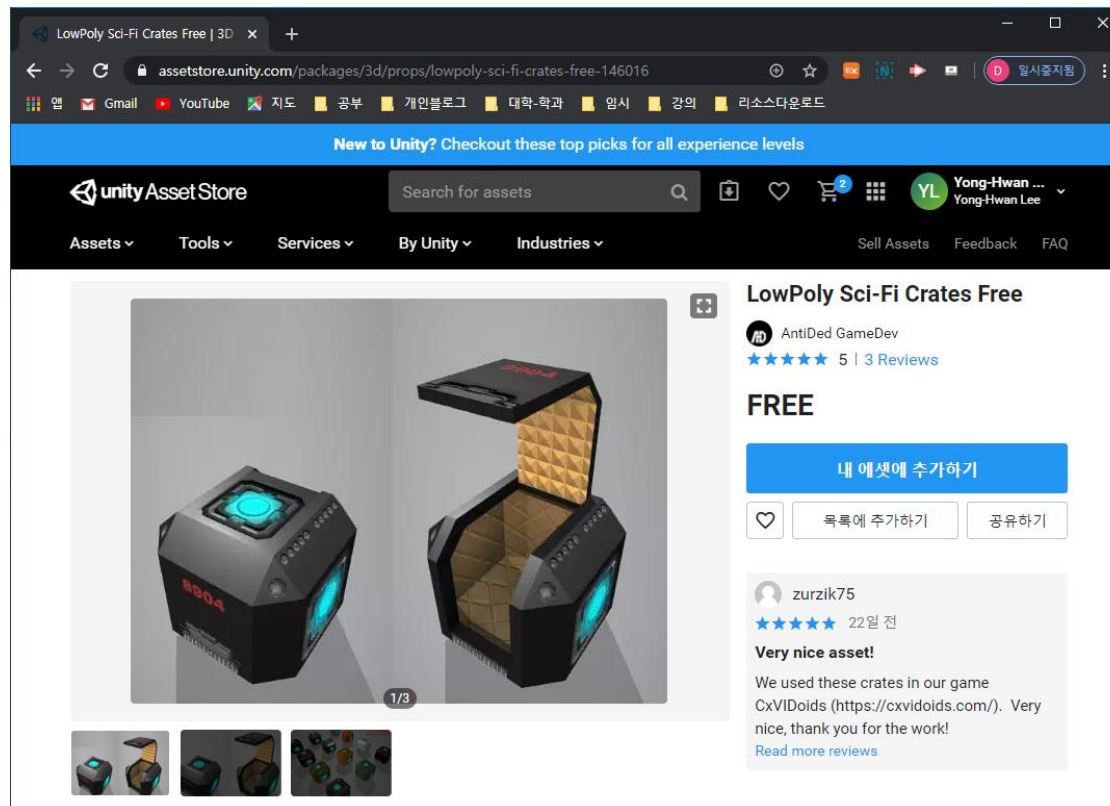
– Environments 예





장애물 변경하기

- Asset Store 에서 <box free>로 검색하여 LowPoly Sci-Fi Crates Free 다운로드



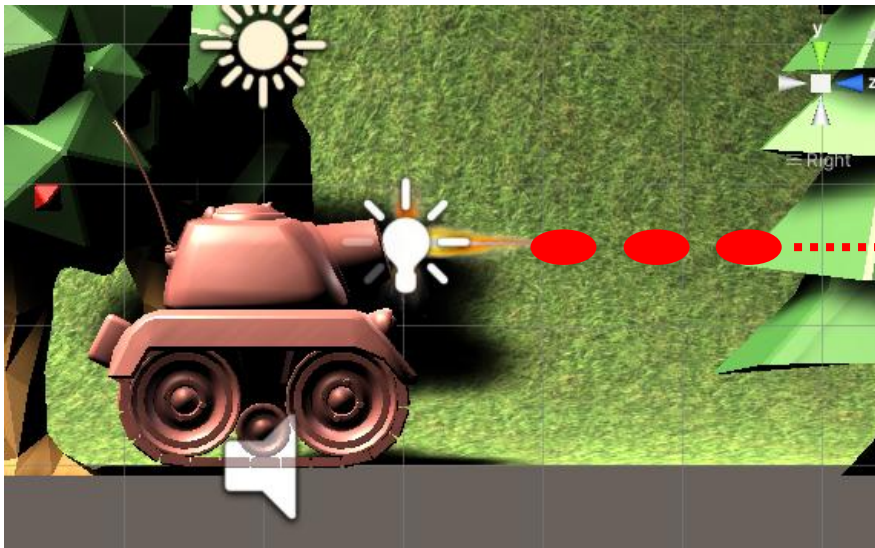
수정된 게임 화면



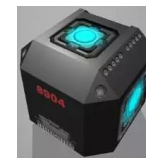


현재 총알 발사의 형태

- 키(Ctrl) 입력에 따라, 총알을 스파인포인트에 생성 (CarFire.cs)
- 총알이 생성되면 (Bullet.cs가 실행되어) 총알의 위치가 **Z축에 따라 이동**
- 현재의 상태에서는 Y축을 기준으로 높은 곳에 위치한 적군과 작은 장애물을 총알로 맞출 수가 없음



높은 곳에 위치한 적군



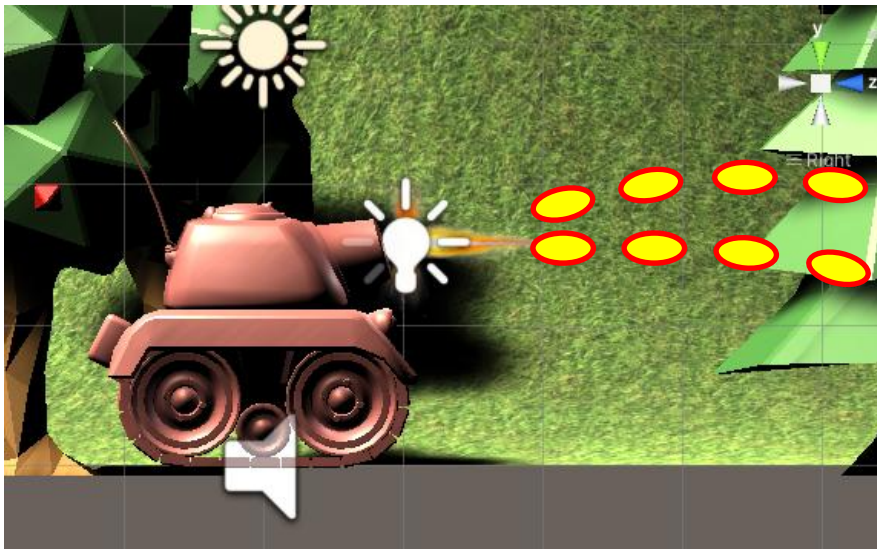
작은 장애물

(맞출 수가 없음)



AddForce()를 통해 총알을 발사

- 생성한 Bullet 의 Rigidbody 컴포넌트를 찾아, AddForce()를 통해 힘을 가함
- 총알 프리팹은 물리력을 가져야 하며, 중력을 적용하기 위해 Use Gravity를 체크
- 중력에 의해 직선으로 날아가지 않고, 포물선을 그리며 날아감
- **스판포인트의 방향을 (세밀하게) 조정할 필요가 있음**



높은 곳에 위치한 적군



작은 장애물

총알 발사 – **ADDFORCE()**

– 총알 프리팹 생성

- Sphere로 생성하고 Rigidbody 추가

오브젝트	이름	속성	값	비고
Sphere	BulletF	Scale x, y, z material Rigidbody	0.2, 0.2, 0.5 ColorRed 추가	빨간색 리지드바디 추가

- 총알(Sphere) 인스펙터에서 Is Trigger를 체크한다
- 스크립트를 생성하고 (다음 슬라이드) 총알 객체로 연결
- 사운드 파일을 다운로드 받아 (스크립트에서 선언한) AudioClip 에 연결
- 프리팹으로 추가
 - 프로젝트 브라우저에서 Prefab 폴더로 드래그 (필요하면 이름을 수정)
 - 하이러라키에서는 필요가 없으므로 삭제

- 총알 스크립트(BulletF)

```
public class BulletF : MonoBehaviour
{
    public AudioClip sndExp;

    // Start is called before the first frame update
    void Start() {}

    // Update is called once per frame
    void Update() { }

    // 충돌처리
    void OnTriggerEnter(Collider coll)
    {
        AudioSource.PlayClipAtPoint(sndExp, transform.position); // 사운드
        if (coll.tag == "Target") { // 충돌체의 태그 조사
            Destroy(coll.gameObject); // 벽제거
        }
        Destroy(gameObject); // 포탄제거
    }
}
```

- Fire 스크립트를 작성하고 자동차객체(Car)에 연결, 인스펙터에서 총알 프리팹을 bullet로 연결

```
public class Fire : MonoBehaviour
{
    public Transform bullet;
    Transform spPoint;
    int power = 2000;

    // Start is called before the first frame update
    void Start() {
        spPoint = GameObject.Find("SpawnPoint").transform;
    }

    // Update is called once per frame
    void Update() {
        if(Input.GetButtonDown("Jump")) { FireBullet(); }
    }

    void FireBullet() {
        Transform obj = Instantiate(bullet, spPoint.position, spPoint.rotation) as Transform;
        obj.GetComponent<Rigidbody>().AddForce(spPoint.forward * power);
    }
}
```

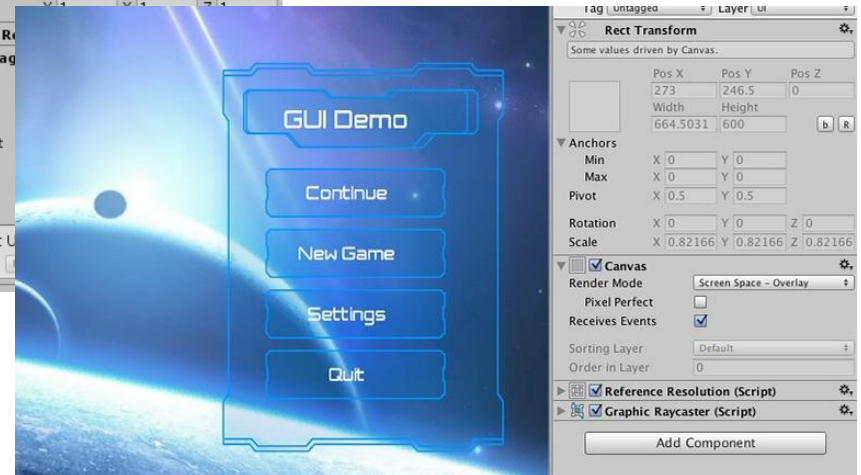
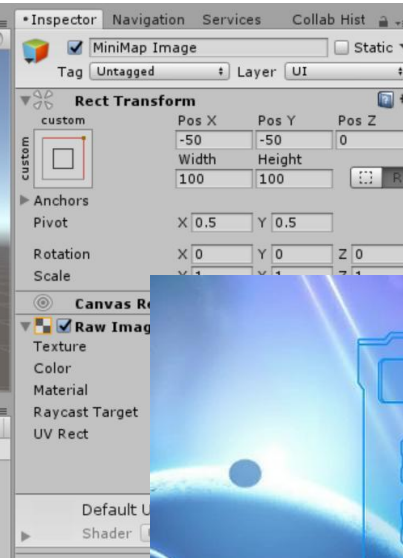
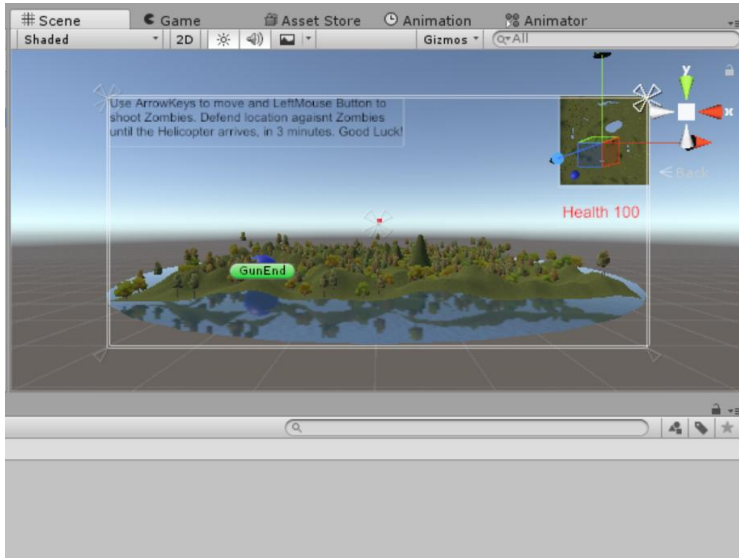
- 스파인포인트(SpawnPoint)를 미리 만들어 놓거나 혹은 기존에 있는 스파인포인트를 사용하고, 스파인포인트 각도를 조정하여 발사 여부를 확인

점수 획득 표시

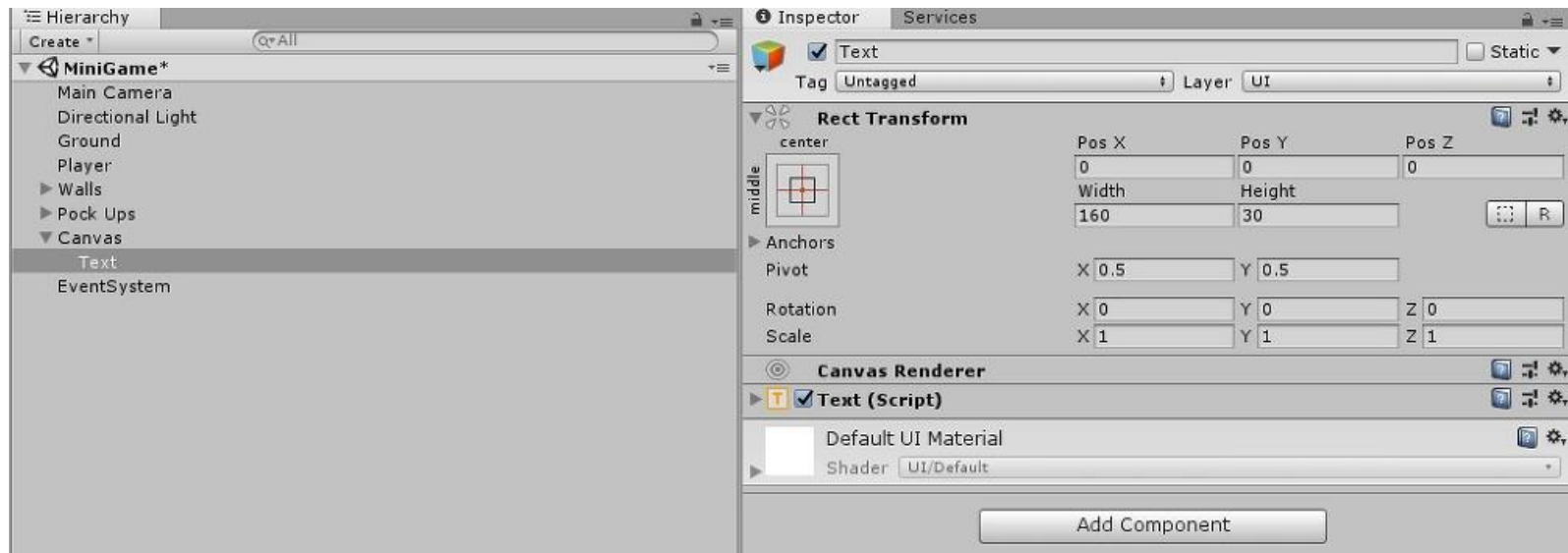


Canvas 다루기

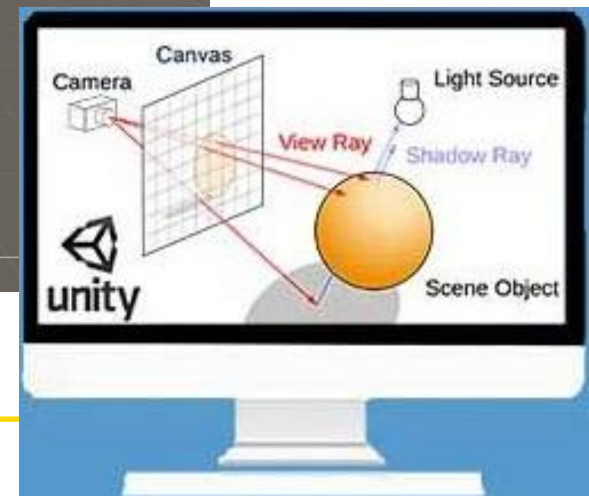
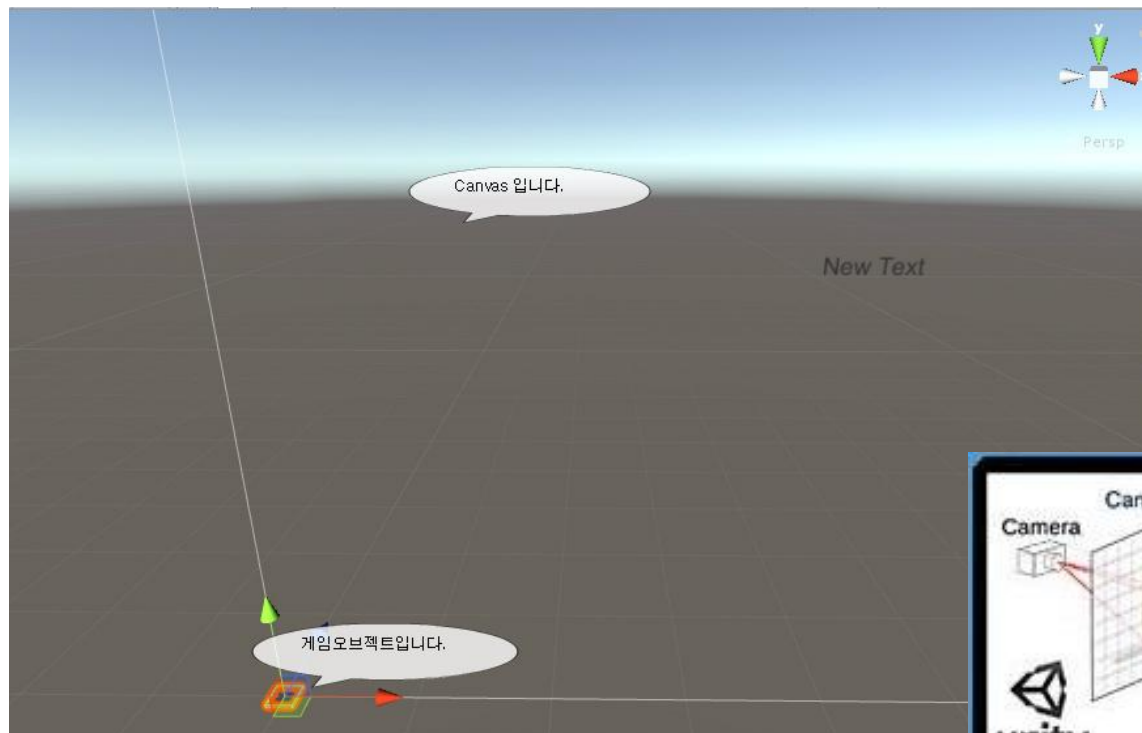
- 하이라리키 창에서 [Create - UI - Canvas]를 클릭하여 캔버스를 추가
(Canvas와 함께 Event System이 추가됨 - 여기서는 Canvas만 사용)
- 캔버스를 선택한 상태에서 메뉴 [GameObject - UI - Text]를 클릭하여 텍스트를 추가



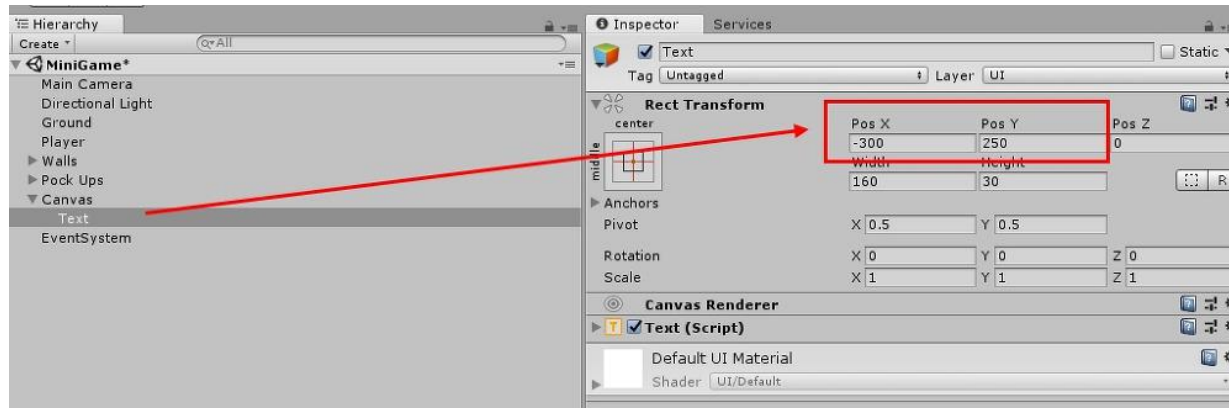
- 하이라키 뷰에 Canvas의 자식 객체로 Text 객체가 생김
- UI가 화면에 표시되려면, 모든 UI 오브젝트는 Canvas의 자식으로 되어 있어야 함



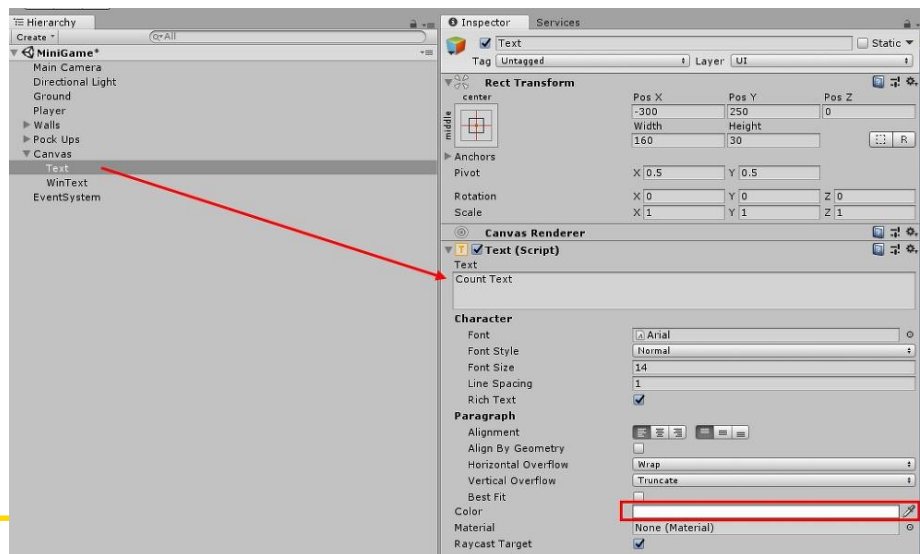
- Canvas를 씬 뷰에서 확인하면, 게임오브젝트와 거리상으로 떨어져 있는 곳에 배치됨
- 게임 뷰를 촬영하는 카메라 화면과 UI를 촬영하는 카메라 화면을 하나로 합쳐서 게임 화면에 출력됨



- Position의 X 값과 Y 값을 변경하여 좌측 상단에 표시되도록



- Text 값을 입력한 다음, 속성을 수정



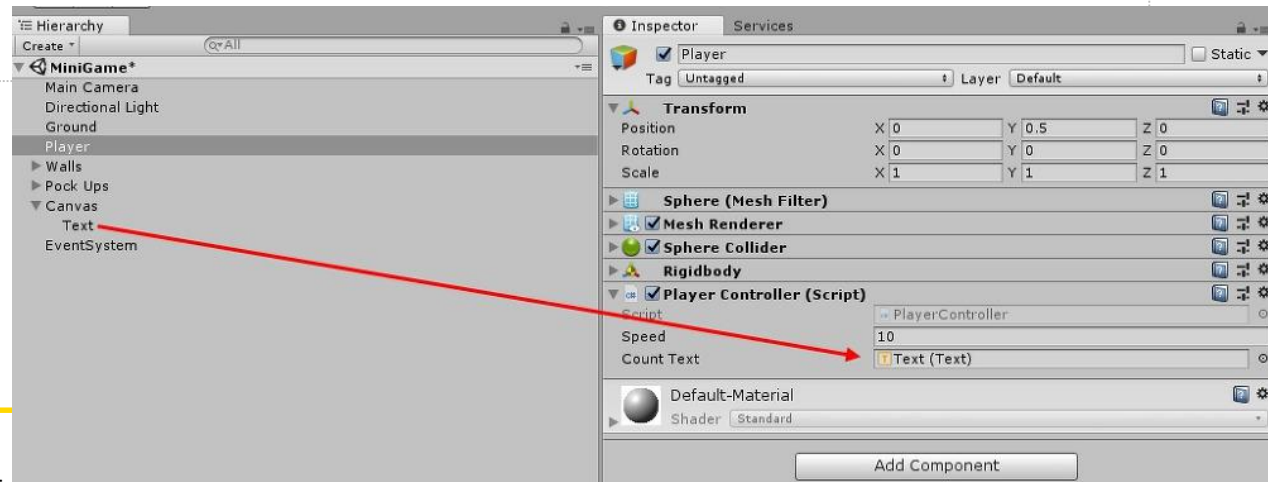
- 스크립트를 수정하고 public으로 선언한 변수를 인스펙터에서 연결

```
using UnityEngine.UI;

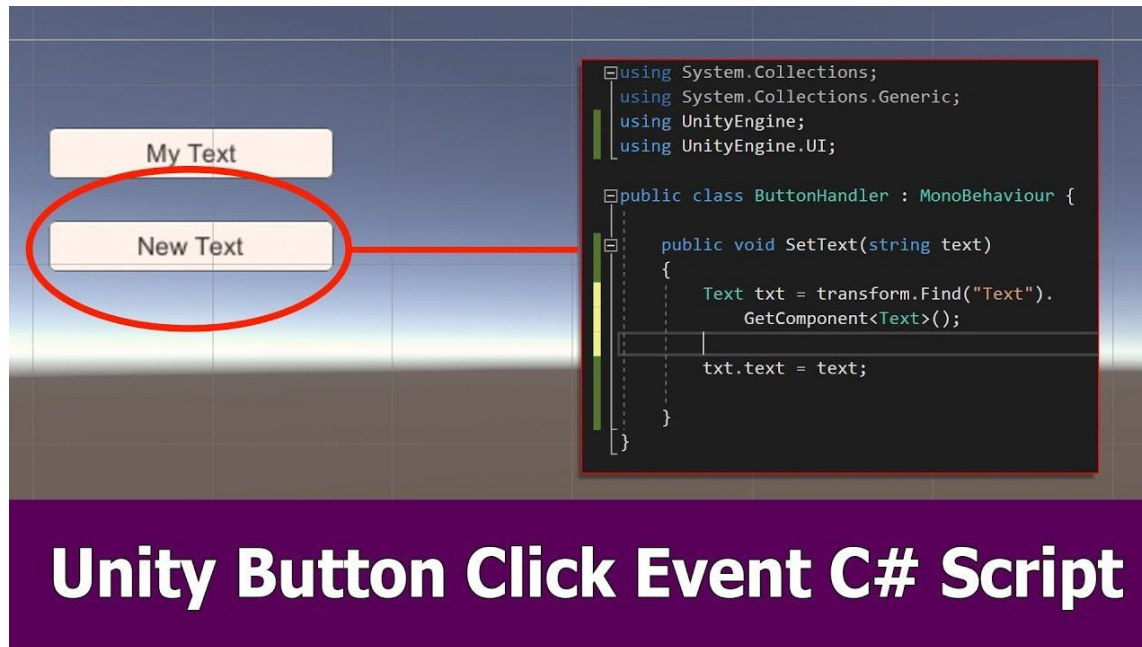
public class CarFire : MonoBehaviour
{
    .....
    public Text countText;

    void Start() {
        ....
        countText.text = "HP : " + hp.ToString();
    }

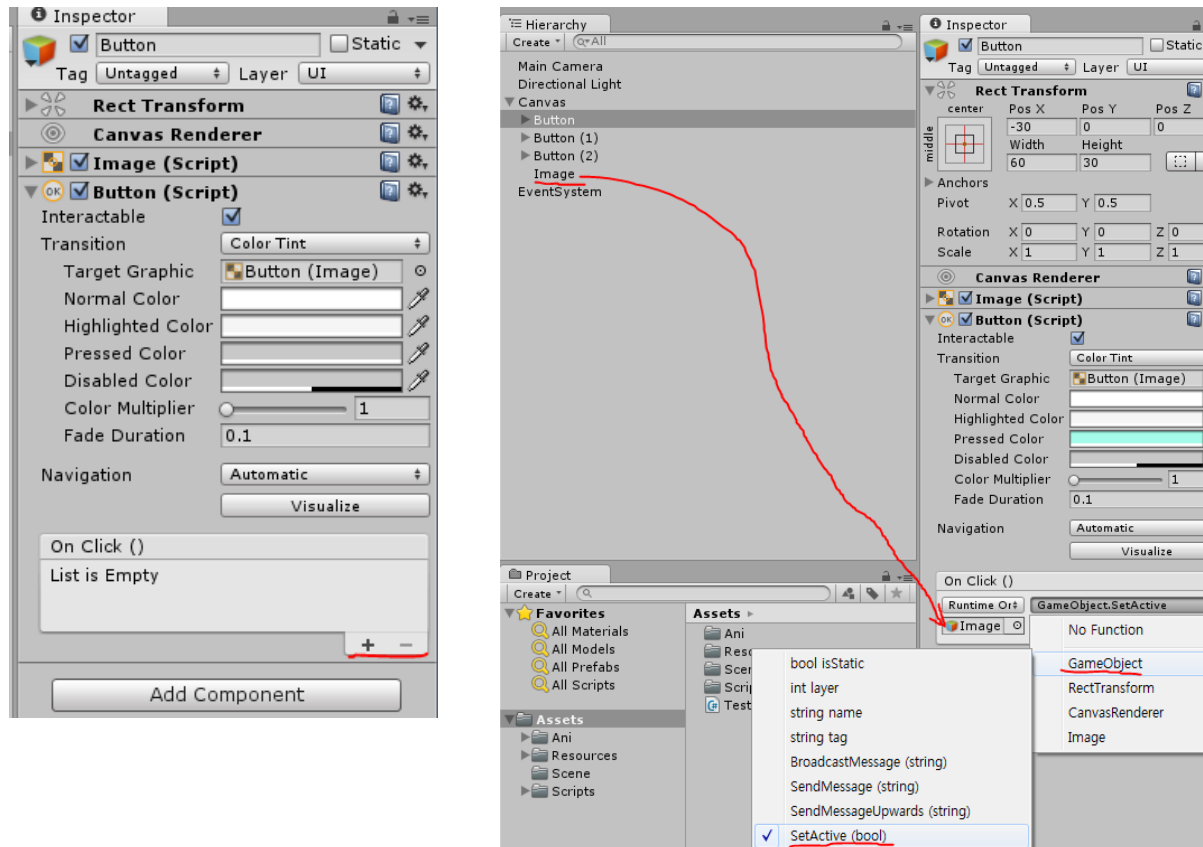
    void OnTriggerEnter(Collider other) {
        if (other.tag == "Bullet") {
            hp--;
            countText.text = "HP : " + hp.ToString();
        }
    }
}
```



- how to add an UI button to canvas
- C# script on button OnClick() function



- 버튼 인스펙터에서 OnClick() 설정
 - (+)를 클릭하여 OnClick()에서 실행하는 오브젝트와 스크립트의 함수를 선택
(함수는 public 유형으로 선언되어야 목록에 나타남)





장애물과 적군을 피격하였을 때, 서로 다른 점수를 부여하도록 코드를 수정하고 이를 반영하여 캔버스에 점수를 표시

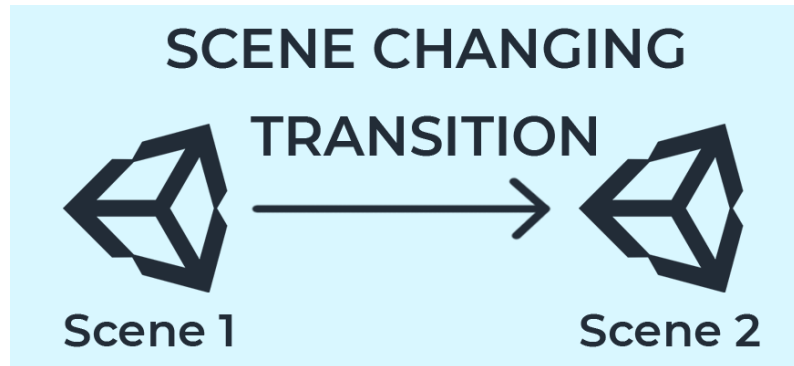
- 이를 처리하기 위한 단계와 방법을 한글 파일에 적어봅니다
- (뒤에 나오는) 과제 제출 시, 한글 파일을 함께 압축하여 제출

SCENE TRANSITION



Scene 전환

- How to create Scene Transition Effects in Unity
- SceneManager를 위해, 스크립트에 using UnityEngine.SceneManagement 가 필요



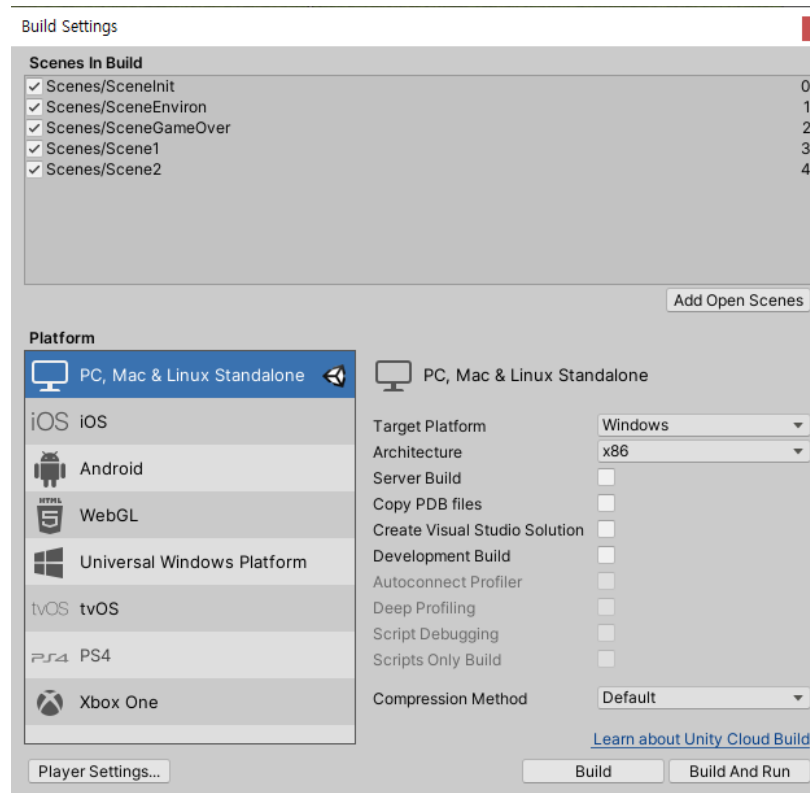
- 스크립트를 수정하고 public으로 선언한 변수를 인스펙터에서 연결

```
using UnityEngine.SceneManagement;

public class CarFire : MonoBehaviour
{
    .....

    public void GoToNextScene() {
        SceneManager.LoadScene("NextScene");
    }
}
```

- LoadScene(“전환하는씬이름”)를 통해 씬을 전환
- 전환하는 씬은 Building Setting에 씬이 등록되어 있어야 함

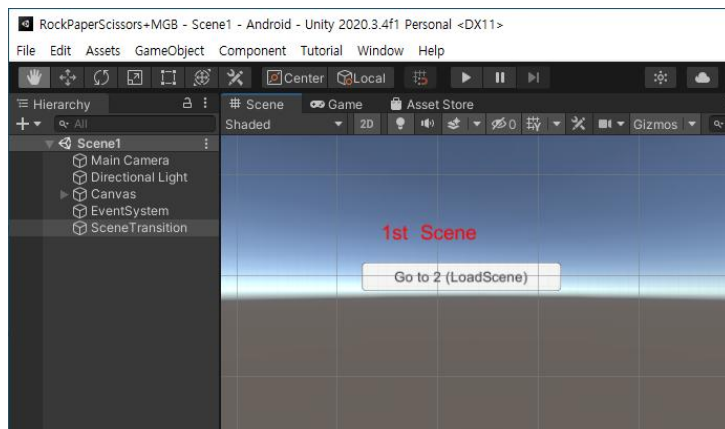


EXAMPLE OF SCENE TRANSITION

1. Scene Transition using LoadScene()
2. Scene Transition using LoadSceneAsync()
3. Scene Transition with Parameter
4. DontDestroyOnLoad() 로 설정된 게임 오브젝트 파괴

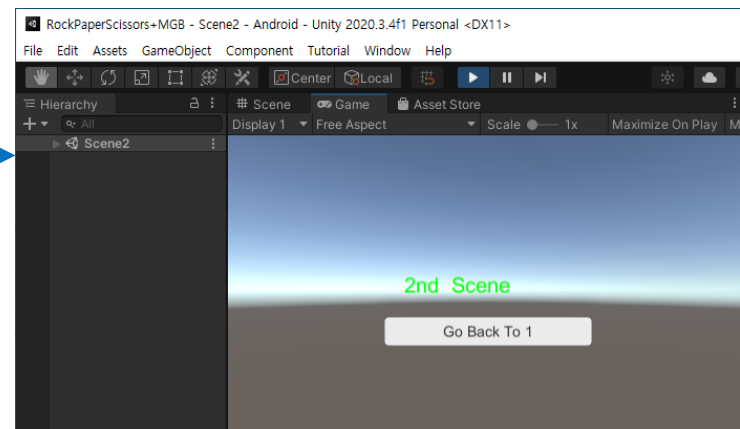
1 Scene Transition by LoadScene()

화면1

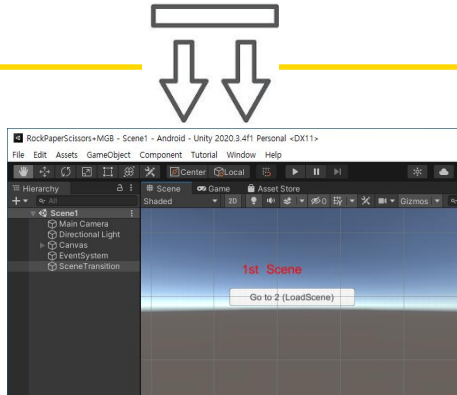


버튼을 클릭하여
씬 전환

화면2

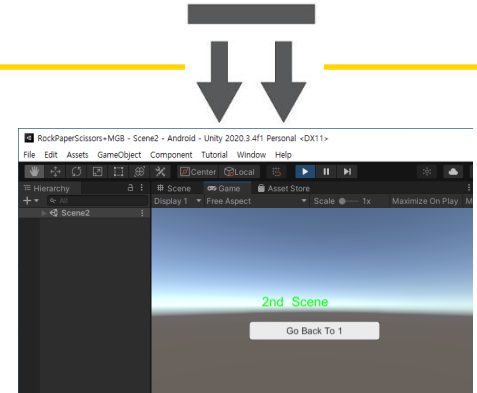


- [Go to 2] 버튼을 클릭하여 (현재의 Scene1에서 다음의) Scene2로 씬 전환



• Scene1 에서

- 빈 오브젝트 (named SceneTransition) 생성
- 스크립트 (named SceneTransitionContoller) 생성
- SceneTransitionController 스크립트를 SceneTransition 오브젝트에 연결
- 스크립트 수정 : 다음 슬라이드 참조
- Canvas에 Text와 Button을 생성
- Button의 OnClick() 함수에 SceneTransition 오브젝트를 연결하고, LoadScene()함수를 호출하는 함수를 연결



- Scene2 에서
 - 빈 오브젝트 생성 (SceneTransition)
 - 스크립트 SceneTransitionController를 오브젝트 SceneTransition에 연결
 - Canvas에 Text와 Button을 생성
 - Button의 OnClick() 함수에 SceneTransition 오브젝트를 연결하고 LoadScene()함수를 호출하는 함수를 연결



```
using UnityEngine.SceneManagement;

public class SceneTransitionController : MonoBehaviour
{
    .....

    public void CallScene1() {
        SceneManager.LoadScene("Scene1");
    }

    public void CallScene2() {
        SceneManager.LoadScene("Scene2");
    }
}
```

2

Scene Transition by LoadSceneAsync()

- 비동기적으로 Scene을 호출
 - 유니티는 씬을 불러올 때, 동기적으로 호출하는 LoadScene() 대신 LoadSceneAsync()를 사용할 것을 권장
 - LoadScene()은 중지 또는 약간의 지연 등이 발생할 수 있기 때문
 - LoadSceneAsync()은 AsyncOperation 클래스를 리턴 값으로 전달
 - AsyncOperation 프로퍼티로 현재 씬의 호출 상태를 알 수도 있음
- Unity Documents 에서 발췌

"In most cases, to avoid pauses or performance hiccups while loading, you should use the asynchronous version of this command which is:

[LoadSceneAsync\(\)](#)."



```
using UnityEngine.SceneManagement;

public class SceneTransitionController : MonoBehaviour
{
    .....
    void Update() {
        // Space bar 키 입력
        if(Input.GetKeyDown(KeyCode.Space)) {
            LoadNextScene();
        }
    }

    public void CallScene1() {
        SceneManager.LoadScene("Scene1");
    }

    public void CallScene2() {
        SceneManager.LoadScene("Scene2");
    }

    public void LoadNextScene()
    {
        // 비동기적으로 Scene를 불러오기 위해 Coroutine()을 사용
        StartCoroutine(LoadMyAsyncScene());
    }

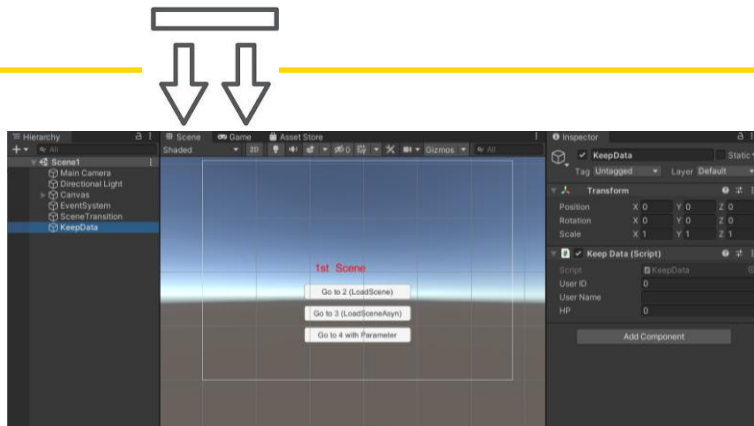
    IEnumerator LoadMyAsyncScene()
    {
        //Debug.Log("LoadMyAsyncScene() is called");
        // AsyncOperation을 통해 SceneLoad() 정도를 알 수 있음
        AsyncOperation asyncLoad = SceneManager.LoadSceneAsync("Scene3");

        // Scene를 불러오는 것이 완료되면, AsyncOperation은 isDone상태가 됨
        while (!asyncLoad.isDone)
        {
            yield return null;
        }
    }
}
```

3

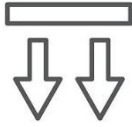
Scene Transition with Parameter

- 여러 개의 Scene으로 구성된 게임은 Scene들 간의 이동 시에 데이터 공유 등이 필요할 수 있다. 예를 들어, 캐릭터가 A 씬에서 B씬으로 이동할 때, 캐릭터의 HP, MP 등의 데이터를 넘겨줘야 한다
- 실제 게임 오브젝트를 넘기는 것이 아닌 데이터를 파일로 저장하고, 새로운 씬이 호출되면 저장된 값을 불러와 사용하도록 설정



- Scene1 에서

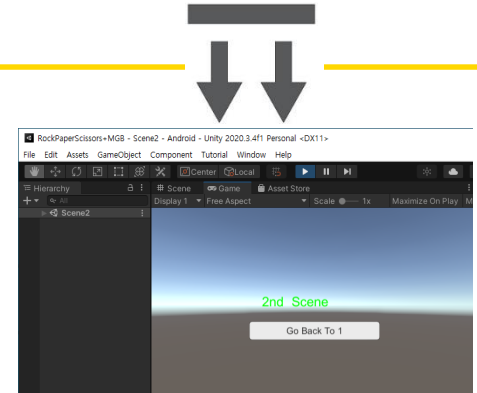
- 씬 전환 시에 넘겨주는 데이터를 게임 오브젝트로 보존
- Awake() 에서 DontDestroyOnLoad() 함수를 통해, 전환 시점에서 게임 오브젝트의 폐기를 방지
 - (씬이 바뀌어도 삭제되지 않은 객체의 변수에 접근이 가능함)
- 데이터 타입은 public 로 선언



```
public class KeepData : MonoBehaviour
{
    public string username;
    public int HP;

    void Start() {
        username = "YongHwan";
        HP = 100;
    }
    .....

    private void Awake() {
        DontDestroyOnLoad(this.gameObject);
    }
}
```



- Scene4 에서

- 삭제되지 않고 넘어온 게임 오브젝트를 탐색
- 탐색한 게임 오브젝트에 있는 (연결된) 스크립트를 불러옴 (GetComponent)
- 스크립트에 있는 변수 사용
(해당 변수에 있는 값을 자유로이 사용이 가능함)



```
using UnityEngine.UI;

public class GetData : MonoBehaviour
{
    // 매개변수로 넘어오는 값을 받기 위한 임시 오브젝트 생성
    GameObject paramObj = null;
    Text UItextName;
    Text UItextHP;

    void Start() {
        paramObj = GameObject.Find("KeepData");
        if (paramObj != null) {
            string name = paramObj.GetComponent<KeepData>().username;
            int HP = paramObj.GetComponent<KeepData>().HP;

            UItextName.text += name;
            UItextHP.text += HP.ToString();
        } else {
            Debug.LogWarning("[경고] 매개변수가 넘어오지 않았습니다");
        }
    }

    ...
}
```

4

DontDestoryOnLoad() 게임 오브젝트 파괴하기

- DontDestoryOnLoad()로 설정된 게임 오브젝트도 필요에 따라 파괴
 - Can't 가 아니라 Don't 임 (파괴할 수 없는 것이 아니라 파괴하지 않는 것임)
- Destory(gameObject); 로 파괴할 수 있음

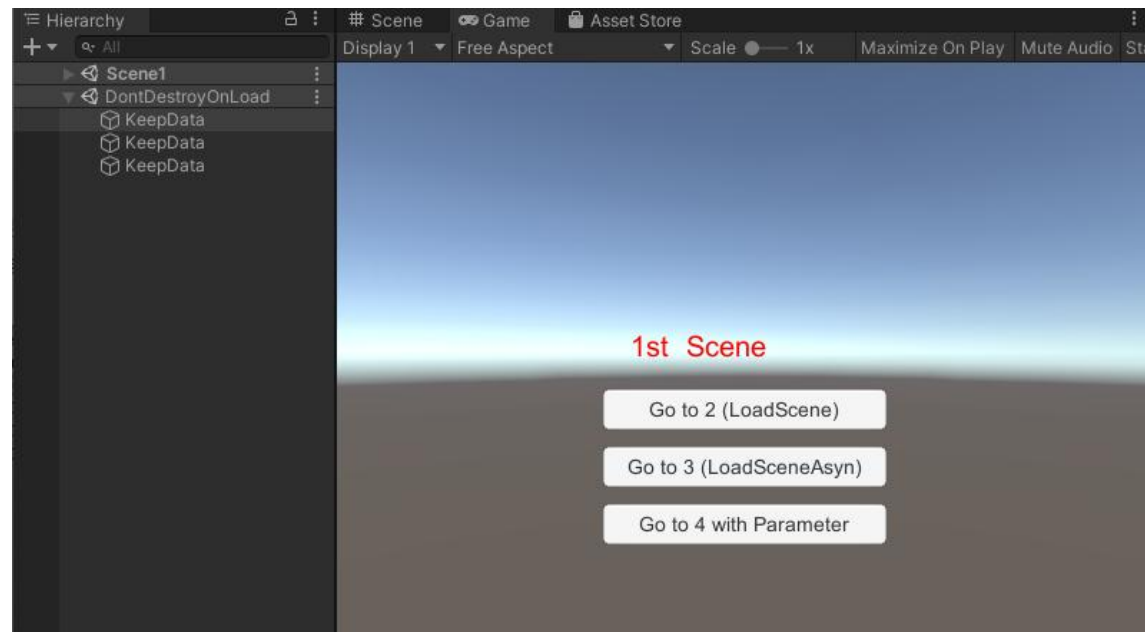
– 설계할 때 유의 사항

- DontDestoryOnLoad() 가 적용된 게임 오브젝트가 있는 씬과 다른 씬을 여러 차례 왔다 갔다 하는 경우

(예) TestScene에 진입하면, 파괴하지 않은 게임 오브젝트가 DontDestoryOnLoad 영역으로 이동되어 씬을 이동해도 파괴되지 않음. 그 다음 OtherScene으로 이동했다가 다시 TestScene으로 돌아오면 어떻게 되는지 확인한다



- 씬이 새로 불러올 때는 해당 씬의 초기 상태로 불러오기 때문에, 기존에 파괴되지 않은 게임 오브젝트와 새로이 생성된 게임 오브젝트가 중복적으로 나타남
- 즉, 두 씬을 왕복하면 같은 게임 오브젝트가 계속적으로 누적되는 문제가 발생
불필요한 메모리 사용을 증가시키며, (중복된 게임 오브젝트로 인해) 예측할 수 없는 오동작이 발생할 수도 있음





Solutions

- (1) 중복 검사 후 파괴
- (2) 초기화 실행 구현
- (3) 라이프사이클 관리



Solutions

(1) 중복 검사 후 파괴

- 파괴하지 않는 게임 오브젝트를 설정하는 이유가 게임 내에서 단 하나만 존재하는 오브젝트를 만들고자 하는 경우라면, 쉬운 해결 방법으로 게임 오브젝트가 생성된 직후에 실행되는 **Awake()** 함수에서 현재 씬에 있는 같은 오브젝트가 몇 개인지를 검사한 다음, 2개 이상이라면 생성된 오브젝트를 파괴



```
public class DontDestoryObject : MonoBehaviour
{
    ...

    void Awake() {
        GameObject obj = FindObjectsOfType<DontDestoryOnLoad>();
        if (obj.Length == 1) {
            DontDestroyOnLoad(gameObject);
        } else {
            Destroy(gameObject);
        }
    }
}
```



- You better use Singleton pattern which ensures that object will exist in a single copy
- You don't need to use heavy `FindObjectsOfType()` method



```
public class KeepData : MonoBehaviour
{
    ...
    public static KeepData instance = null;

    void Awake() {
        if (instance == null) {
            instance = this;
        } else if (instance != this) {
            Destroy(gameObject);
        }
        DontDestroyOnLoad(gameObject);
    }
}
```



Run 시켜서 `DontDestroyOnLoad` 영역에 게임 오브젝트가 중복적으로 생성되는지를 확인

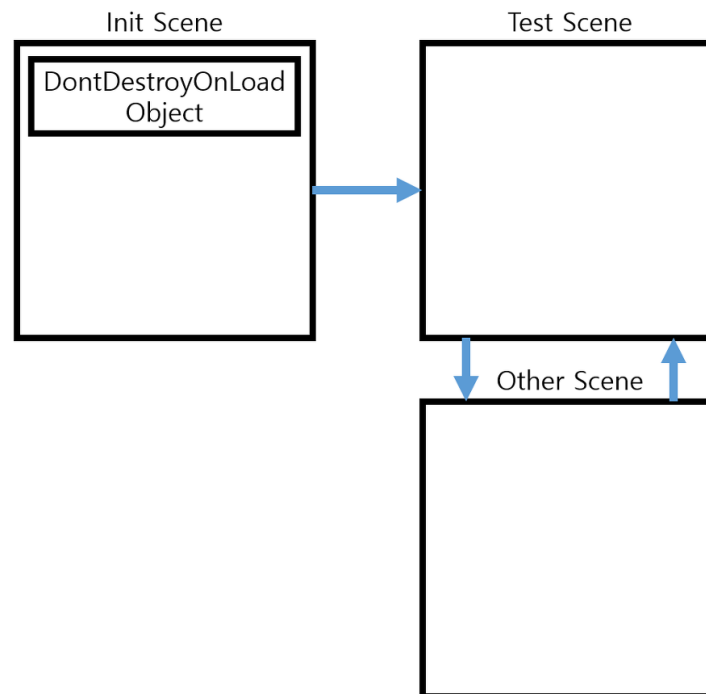




Solutions

(2) 초기화 씬 구현

- 파괴하지 않는 게임 오브젝트를 생성하는 전용 씬(InitScene)을 만들고, 해당 씬에서 파괴하지 않는 게임 오브젝트를 생성한 후, 다른 씬들에서만 씬 전환을 진행하는 방식





Solutions

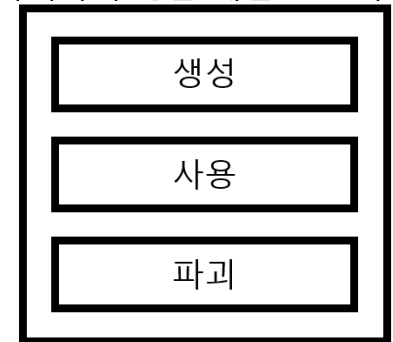
(3) 라이프사이클 관리

- 앞의 2가지 방법은 게임 내에서 해당 오브젝트를 단 하나만 만들고자 할 때 유용한 방법
- 해당 오브젝트가 파괴되지 않아야 하면서 여러 개의 오브젝트를 만드는 것을 허용하는 구조하면 적절하지 않음

(파괴하지 않을 게임 오브젝트를 정적으로 씬에 배치해서 씬이 불러올 때마다 생성되는 구조임)

- 같은 종류의 파괴되지 않을 게임 오브젝트를 여러 개 생성하는 것을 허용하려면, 정적인 씬 배치 방식보다는 프리팹으로 관리하며 원하는 시점에 생성하는 것이 좋음
 - » 파괴하지 않는 게임 오브젝트가 동적으로 여러 개 생성되어 동작하므로, 이후에 계속적으로 생성될 수도 있음
 - » 파괴하지 않는 게임 오브젝트에 대한 추적, 관리가 필요하며, 해당 오브젝트의 사용이 끝나면 파괴하는 방식의 라이프사이클 관리가 필요

파괴하지 않는 게임 오브젝트



4

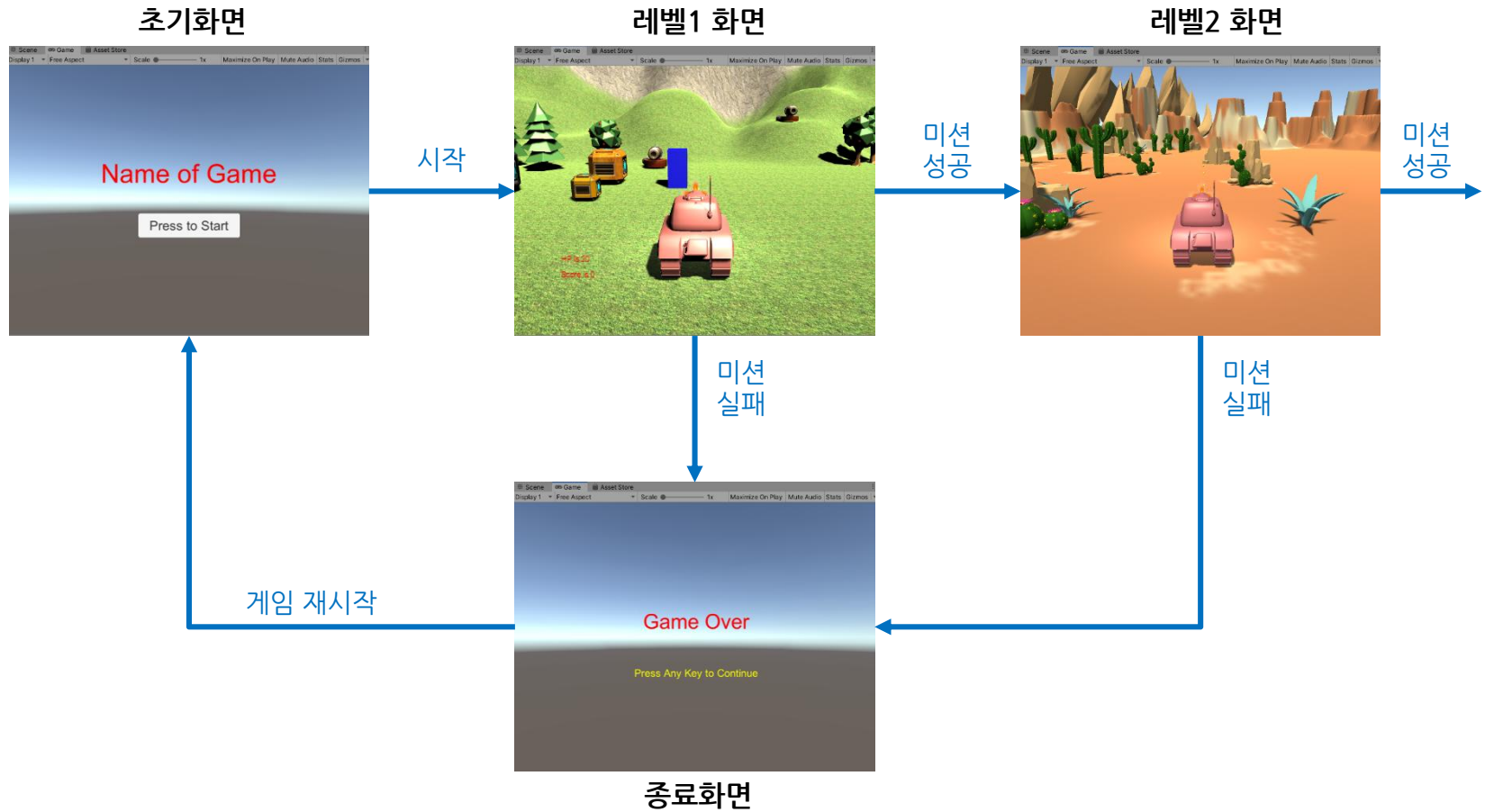
Scene Transition with Parameter (using Prefab)

- 씬을 전환하면서 데이터를 넘겨줘야 하는 경우,

TANK GAME



화면 흐름 예





화면 흐름 예 - (점수 넘어가기)

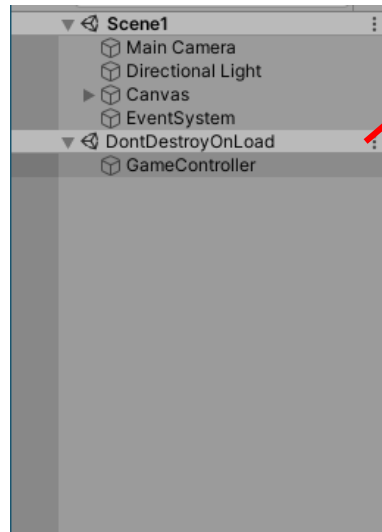


- 레벨1에서 미션을 성공하여 레벨2로 넘어갈 때, 현재의 score를 유지해야 함
(씬 간의 전환에서 현재의 변수값을 이동 씬에서도 사용해야 하는 경우)
- DontDestroyOnLoad() 함수를 통해 화면 전환 시, 게임 오브젝트의 파괴를 막고 이를 계속적으로 유지하여, 원하는 변수에 접근이 가능

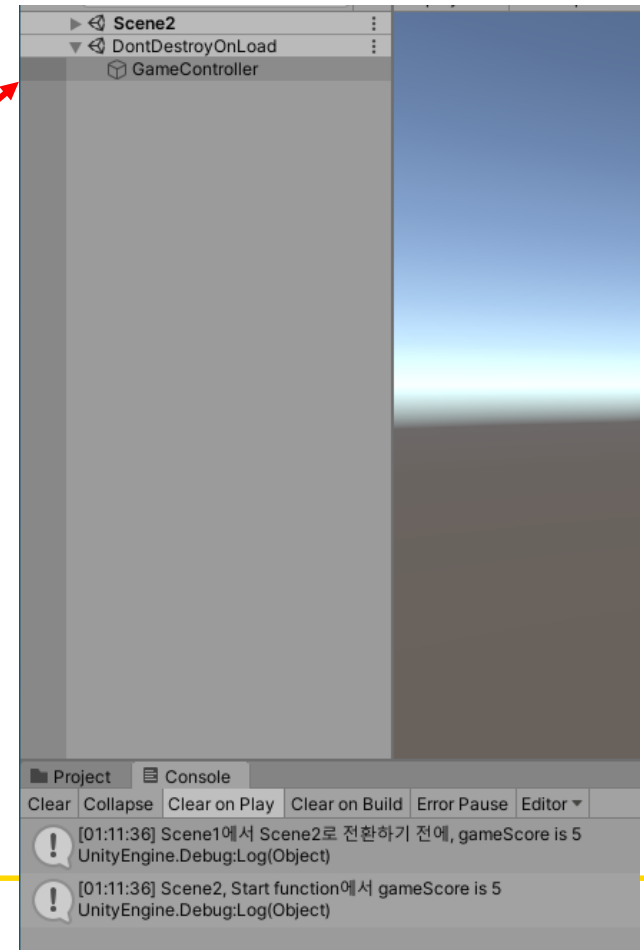


Scene 전환 (값 넘기기)

- 전환 이전의 씬에서 DontDestroyOnLoad() 함수를 통해, 전환 시점에서 게임 오브젝트의 폐기를 방지
- 일반적으로, Awake()에 게임 오브젝트를 등록



게임
오브젝트
유지





DontDestroyOnLoad()를 활용할 때 주의 사항

- 대상 오브젝트는 항상 루트 오브젝트이어야 한다
- 루트 오브젝트가 아닌, 어떤 게임 오브젝트의 하위 오브젝트일 경우, 어떤 메시지가 나오는지 확인

과제



현재까지의 작업을 5개 이상 화면 캡처하고
압축하여 과제 관리에 제출합니다.

기존에 실행 가능한 탱크 게임 (예, 앱스토어 또는 구글 스토어에서 탱크 게임 앱을
다운) 또는 유니티에서 제공한 탱크 튜토리얼, 이외에 인터넷에서 검색하여 탱크
게임을 실행합니다.

- 다운 받은 탱크 게임과 비교하여, (지금까지 작업한 슈팅 게임에) 추가하거나 보완
하고 싶은 기능을 3개 이상 적어봅니다. 기술한 내용을 압축 파일에 포함하여 제출
합니다.



GAME 기초

Part 5. Multi-User Games

FOR MULTI-USER

- Tank Game을 다중 사용자용 게임으로 확장
 - 2인용 게임
 - N인용 네트워크 게임 (동일한 씬으로 접속)



• 2인용 게임 제작

- 2인이 플레이할 수 있도록 수정
- 입력 관리자 확인

- 사용자1 입력 키를 1번으로 설정(Horizontal1, Vertical1, Fire1)
- 사용자2 입력 키를 2번으로 설정(Horizontal2, Vertical2, Fire2)

The image displays two screenshots of the Unity Input Manager interface. The left screenshot shows the 'Duplicate Array Element' button being clicked, and the right screenshot shows the 'Horizontal' button being clicked. Below these are detailed views of the input settings for Horizontal1, Vertical1, Fire1, Horizontal2, Vertical2, and Fire2.

Input Type	Name	Descriptive Name	Descriptive Negative Name	Negative Button	Positive Button
Horizontal1	Horizontal1			a	d
Vertical1	Vertical1			s	w
Fire1	Fire1				space
Horizontal2	Horizontal2			left	right
Vertical2	Vertical2			down	up
Fire2	Fire2				return



```
public class TankController : MonoBehaviour
{
    ...
}
```

– TankController 스크립트 수정

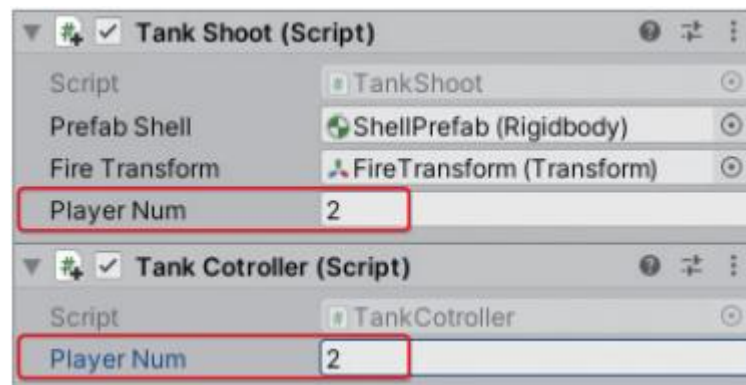
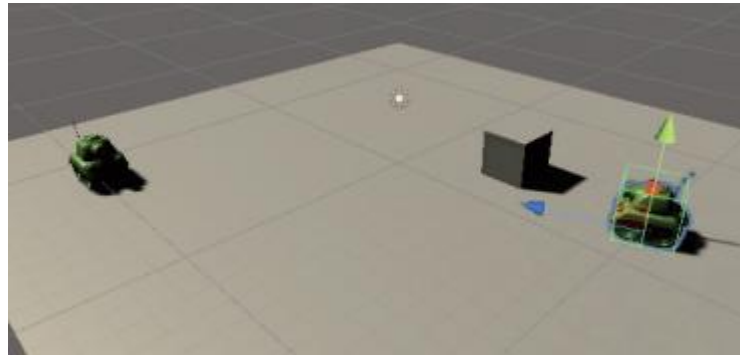
플레이어 번호를
사용하여 입력키
구분

```
01 public class TankCotroller : MonoBehaviour
02 {
03     public int playerNum = 1;    // 탱크 번호
04     string mvAxisName;           // 이동축의 이름 저장
05     string rotAxisName;          // 회전축의 이름 저장
06     float tkMvSpeed = 10.0f;     // 탱크의 이동 속도
07     float tkRotSpeed = 150.0f;   // 탱크의 회전 속도
08
09     // Start is called before the first frame update
10     void Start()
11     {
12         mvAxisName = "Vertical" + playerNum;
13         rotAxisName = "Horizontal" + playerNum;
14     }
15
16     // Update is called once per frame
17     void Update()
18     {
19         float mv = Input.GetAxis (mvAxisName) * tkMvSpeed * Time.deltaTime;
20         float rot = Input.GetAxis (rotAxisName) * tkRotSpeed * Time.deltaTime;
21
22         transform.Translate (0 , 0 , mv);
23         transform.Rotate (0 , rot , 0);
24     }
25 }
```


– TankShoot 스크립트 수정

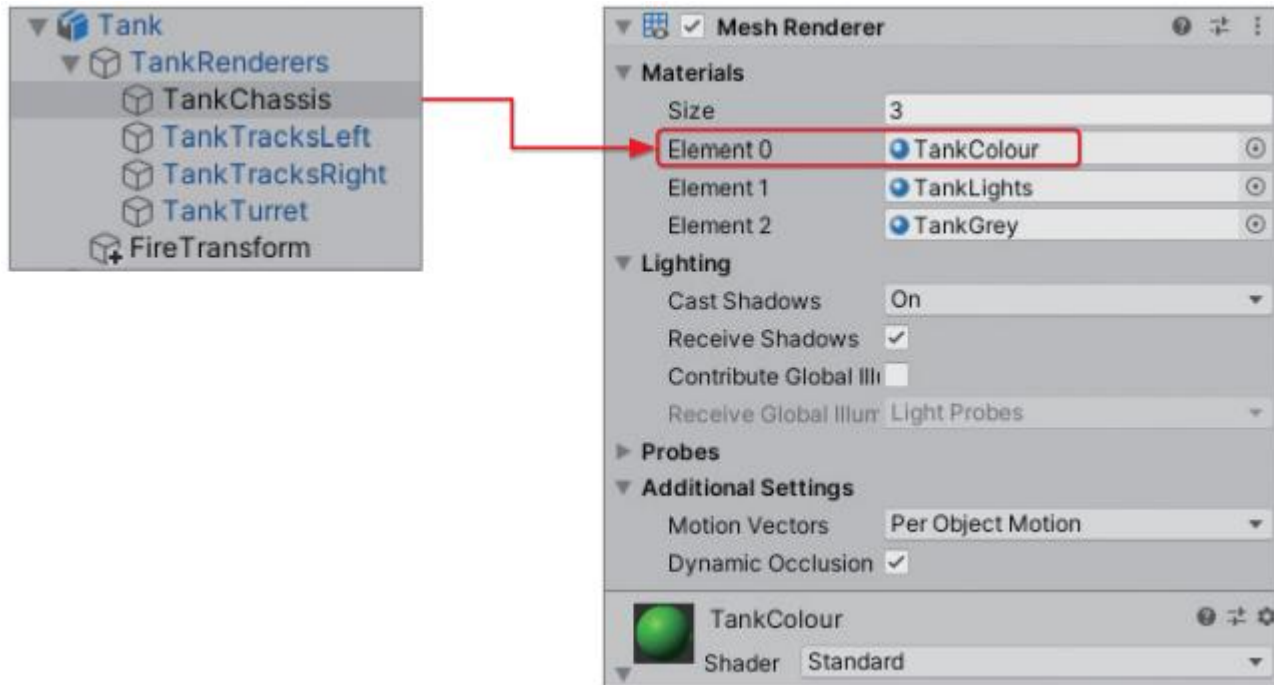
```
01 public class TankShoot : MonoBehaviour
02 {
03     public Rigidbody prefabShell;
04     public Transform fireTransform;
05     public int playerNum = 1;    // 탱크 번호
06     string fireName;            // 발사 키 이름 저장
07
08     // Start is called before the first frame update
09     void Start ()
10     {
11         fireName = "Fire" + playerNum;
12     }
13
14     // Update is called once per frame
15     void Update()
16     {
17         if (Input.GetButtonDown (fireName))
18         {
19             Fire ();
20         }
21     }
```

- 2번째 탱크 만들기
 - Tank 오브젝트 복제(Ctl-D) → Tank2로
 - Tank2의 플레이어 번호를 2로 조정



– Tank 컬러 조정

- Tank를 구성하는 오브젝트의 컬러를 다르게 지정



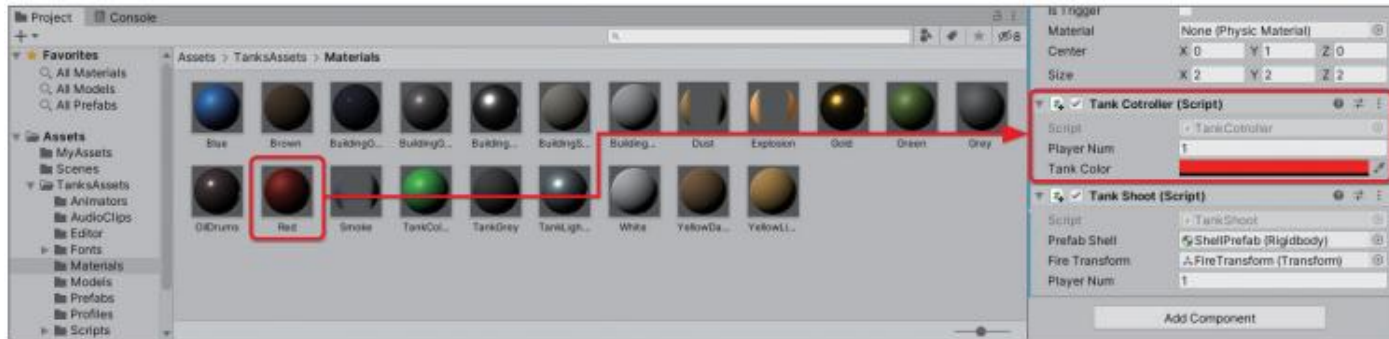
– TankController 스크립트에서 컬러 지정하도록 수정

```
01 public Color tankColor;
02
03 // Start is called before the first frame update
04 void Start()
05 {
06     mvAxisName = "Vertical" + playerNum;
07     rotAxisName = "Horizontal" + playerNum;
08
09     // Get all of the renderers of the tank.
10     MeshRenderer [ ] renderers = GetComponentsInChildren<MeshRenderer> ();
11
12     // Go through all the renderers...
13     for (int i = 0; i < renderers.Length; i++)
14     {
15         // ... set their material color to the color specific to this tank.
16         renderers [ i ].material.color = tankColor;
17     }
18 }
```

탱크의 자식 오브젝트의 렌더러 찾기

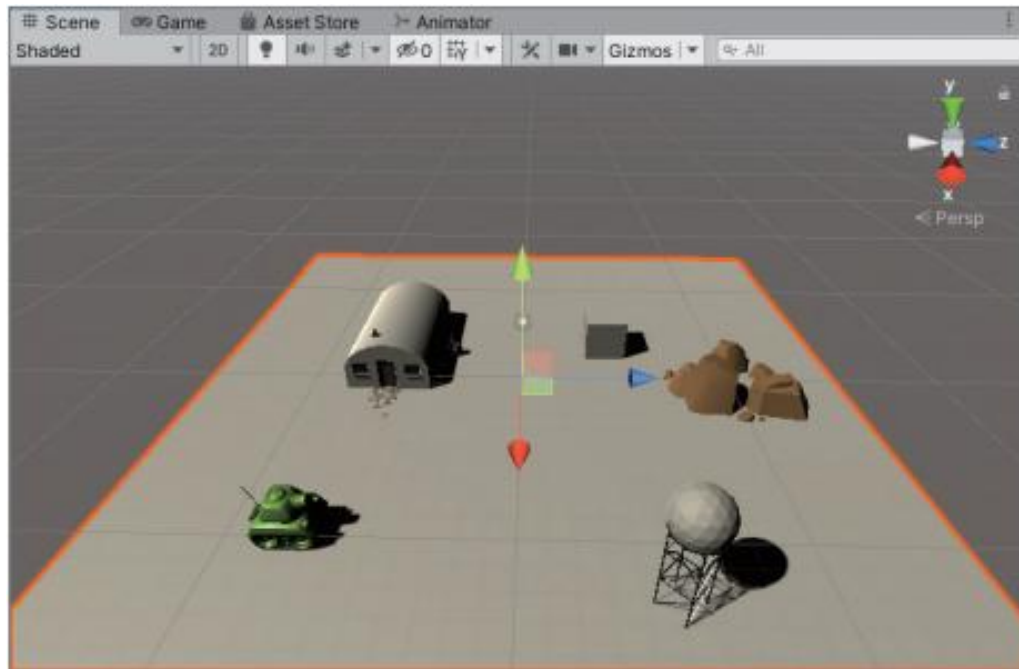
각 렌더러별로 컬러 지정하기

- Unity Editor에서 Tank와 Tank2 오브젝트를 각각 선택하여 TankController 스크립트의 전역변수인 Tank Color에 색상을 지정



- 추가 실습

- 2인용 탱크 게임에 [Models]의 다른 리소스를 추가로 배치
 - 리소스 중 일부는 포탄을 맞게 되면 폭발하도록 스크립트를 적용
 - 탱크가 포탄을 맞으면 폭발하도록 수정 (예, 3번을 맞으면)



- Asset Store에서 배경을 다운받아 적용



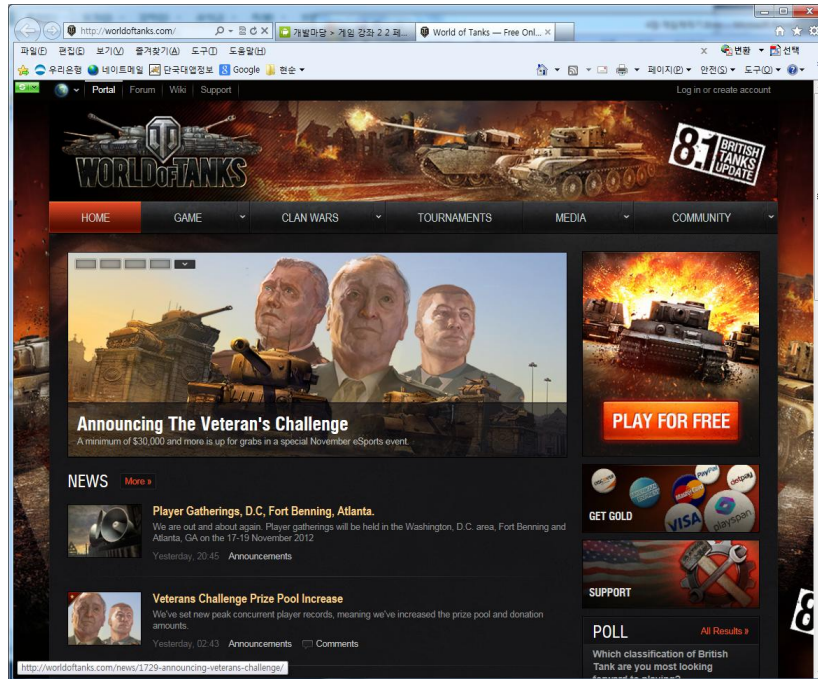


GAME 기초

Part 6. Application 개발

탱크 만들기 실습

- 탱크 게임



- **탱크 게임 분석**

- 기존의 여러 탱크 게임을 찾아 기능을 분석
- 만들고자 하는 탱크 게임의 차별적 기능을 도출
 - 게임 레벨에 따라 적군의 수를 조정
 - 적이 2방 이상 피격을 당했을 때 파괴되도록
 - Lap Timer 기능
 - 포탄 개수를 제한시키고 아이템을 획득했을 때 포탄의 수가 늘어나도록
 - 씬에서 장애물을 다양하게 제공하고 피격할 경우 장애물에 따라 점수를 획득하도록
 - ...

EXAMPLE OF TANK GAME (1)

Unity 3D Physics Based Tank Controller Testing

BoneCracker Games

구독 314

13,688

추가 공유 더보기

90 0

게시일: 2014. 5. 16.
Physics Based Tank Controller Testing

Web Demo: <http://bugra381.wix.com/bonecrackergames>

다음 동영상 자동재생

- Unity3D - Shoot Projectiles
Stuart Spence
조회수 25,263회
5:15
- TANKS! Unity Tutorial - Phase 1 of 8 - Scene Setup
Unity
조회수 43,722회
10:43
- Как создать свою игру на Unity выпуск №1
Game Create уроки по заявкам
조회수 152,105회
26:42
- Unity Tank Physics Test 2
AndersOerum
조회수 8,392회
4:20
- [Unity3D] Tank Battle Creation Kit "Physics Tank Maker"
chobiglass
조회수 6,938회
6:16
- Tank Tracks Build and Rigging Tutorial - Blender v2.69
BanthedProductions
조회수 14,753회
25:57
- Unity3D Physics Based Tank Controller Tutorial
BoneCracker Games
조회수 8,910회
17:00

<http://youtube.com/watch?v=vC7H3aB-sIA>

EXAMPLE OF TANK GAME (2)



Warrior Tank 3D Racing

Play a heart-pounding Unity game with Warrior Tank 3D Racing! This challenging online game is all about speed, heavy vehicles and the race to the top, so customize your tank and start the fight. You will first face only one opponent, but as you win more races, your adversaries will increase in number and your task will become more difficult! Have fun with this tough Unity 3D game!



Super Tank 3D Parking

For those who don't step away from extreme challenges Super Tank 3D Parking is a great unity game option to unwind and have fun! Let's see how well you can handle a massive war vehicle and get it to a full stop in the right spot. With levels that increase in difficulty, various awards and great 3D graphics, this parking unity game is the best choice for the tough!



Mars Rover Extreme Parking

Mars Rover Extreme Parking is a cool 3D Unity game in which you will have the challenge of handling a big, heavy vehicle through a series of obstacle courses. Reach the destination spot as fast as possible and stay on the track at all times. With high quality graphics and a realistic design and landscape, this online unity game is both fun and exciting to play.


Melon Dash


Family

More Hot


2016 Crowdfunder
Join 1 Back

PRE-ORDER

Got it

Learn more

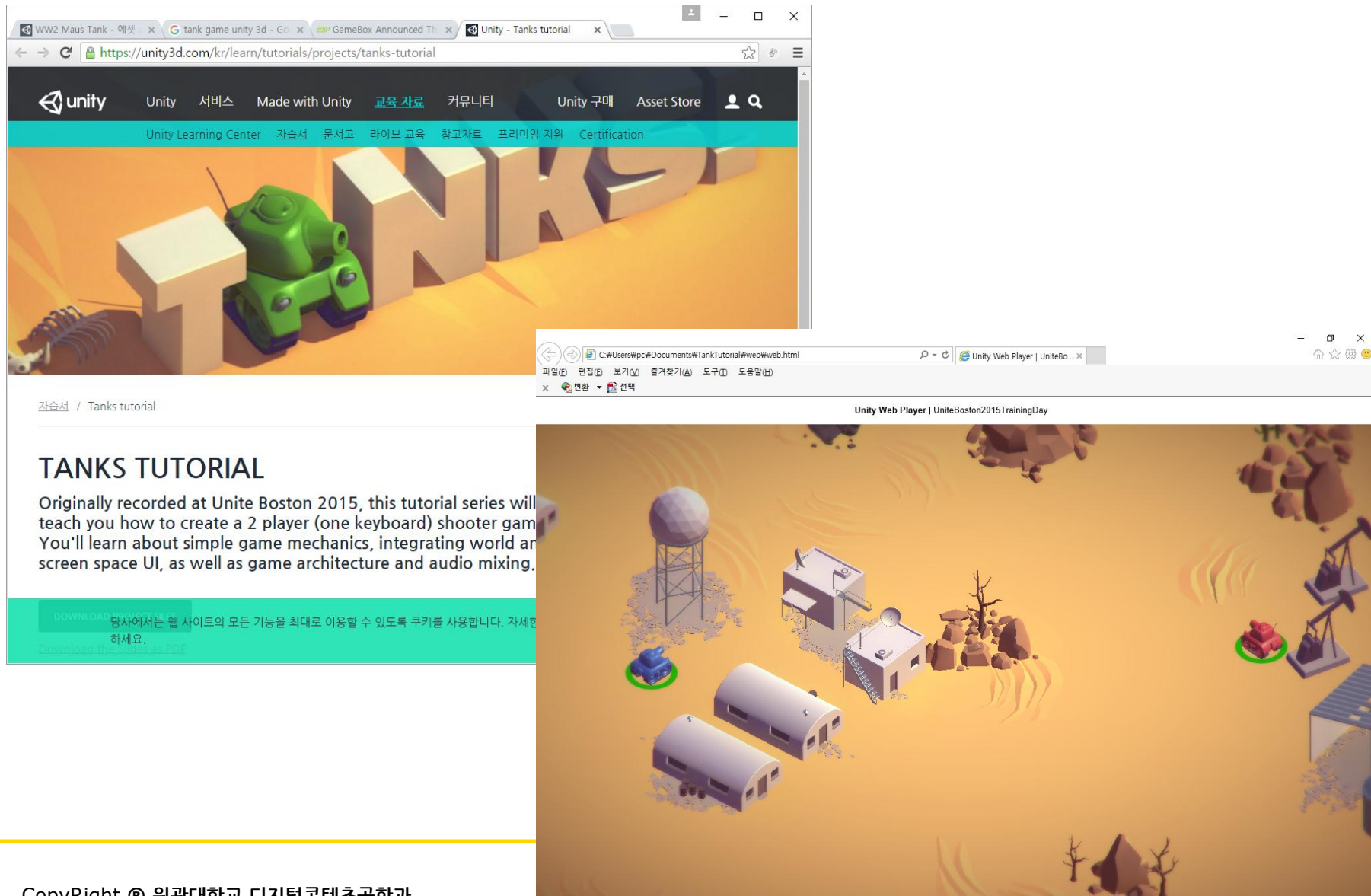
<http://www.unitygamesbox.com/unity-tank-games/>

We use cookies to improve your experience on our website. By browsing this website, you agree to our use of cookies.

Got it

Learn more

EXAMPLE OF TANK TUTORIALS (UNITY TECHNOLOGY)



WW2 Maus Tank - 예셋 x tank game unity 3d - Go x GameBox Announced Th x Unity - Tanks tutorial x

https://unity3d.com/kr/learn/tutorials/projects/tanks-tutorial

unity 서비스 Made with Unity 교육 자료 커뮤니티 Unity 구매 Asset Store

Unity Learning Center 자습서 문서고 라이브 교육 참고자료 프리미엄 지원 Certification

TANKS

자습서 / Tanks tutorial

TANKS TUTORIAL

Originally recorded at Unite Boston 2015, this tutorial series will teach you how to create a 2 player (one keyboard) shooter game. You'll learn about simple game mechanics, integrating world and screen space UI, as well as game architecture and audio mixing.

DOWNLOAD

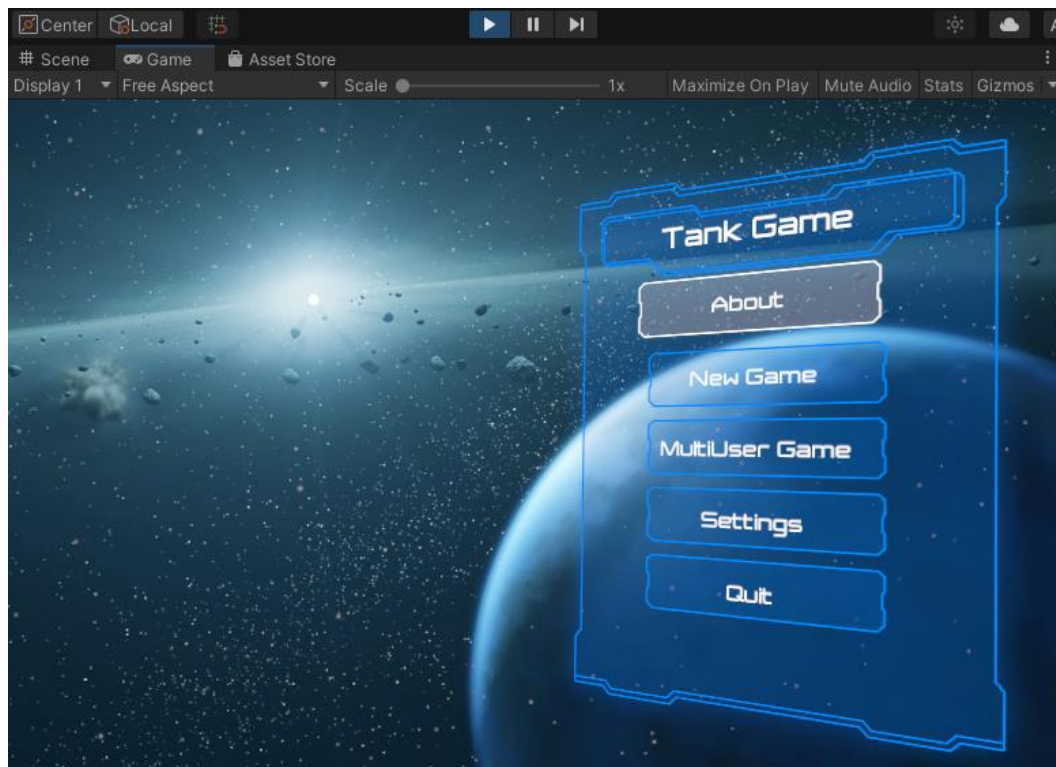
당사에서는 웹 사이트의 모든 기능을 최대로 이용할 수 있도록 쿠키를 사용합니다. 자세한 하세요.

Download the PDF

Unity Web Player | UniteBoston2015TrainingDay

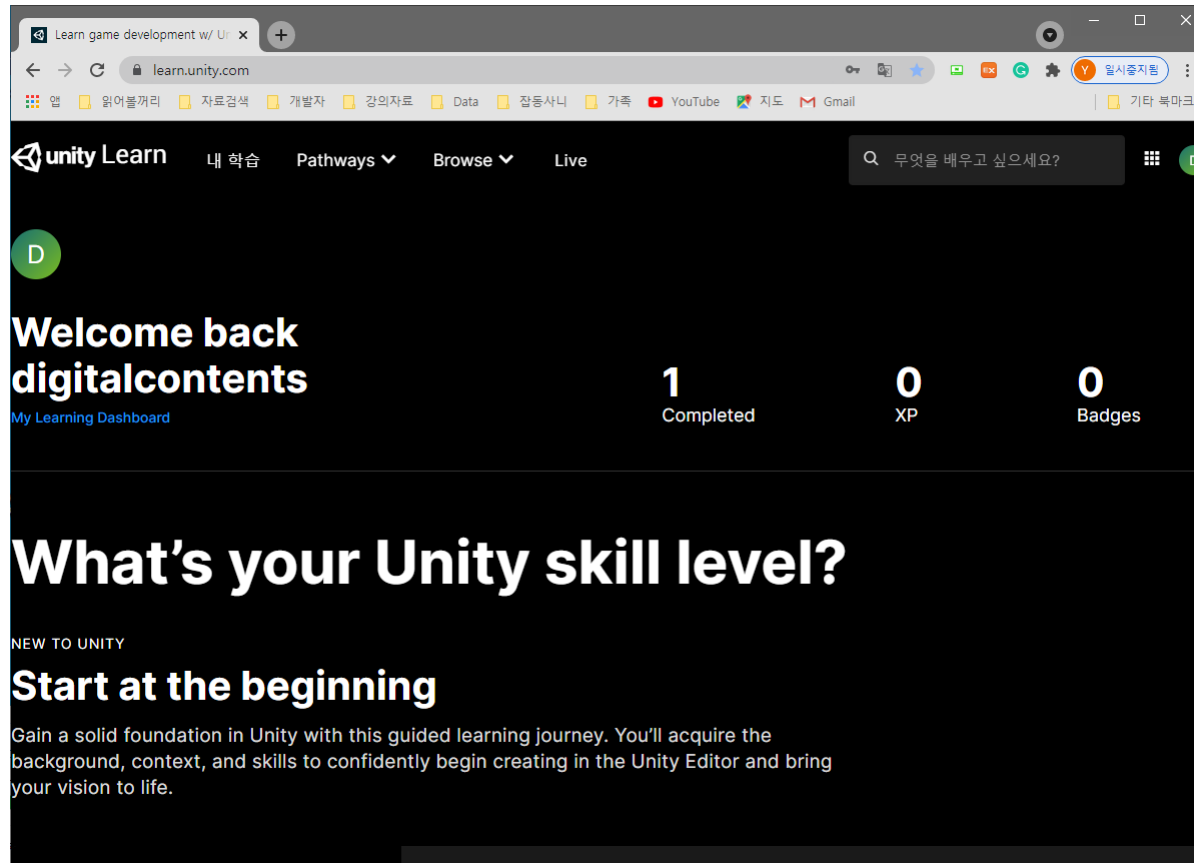
- **COMPLETE THE TANK GAME**

- 시작 화면
- 1인용 & 2인용 탱크 게임을 완성하여 윈도우용 (또는 웹용)으로 빌드





– Unity Learning (and Tutorials)



– Asset Store 에서 다운로드 (Package Manager)

